课程内容介绍：

一、UNIX、Linux系统介绍
1、UNIX系统介绍
2、Linux系统介绍
3、Linux系统的Logo
4、Minix操作系统
5、GNU工程
6、POSIX标准
7、GPL通用公共许可证
二、GNU编译工具
1、gcc常用的编译参数
2、gcc相关的文件类型
3、gcc把C语言变成可执行程序的过程
4、gcc支持的预处理指令
5、gcc预定义的宏
三、环境变量
1、什么是环境变量
2、环境变量的主要作用
设置程序的运行参数：
程序共用：
系统运行：
3、常见的环境变量：
4、查看环境变量
5、Linux系统设置环境变量
6、Windows系统设置环境变量
7、使用标准库函数设置环境变量：
四、错误处理
1、通过函数的返回值表示错误
2、通过errno表示错误
五、库文件的制作与使用
1、什么是库文件
2、静态库与动态库的区别
3、静态库的制作与使用
创建静态库：
使用静态库：
4、动态库的制件与使用
创建动态库：
使用动态库：
动态加载共享库：
5、使用库文件时的辅助工具
6、静态库的优点
使用方便：
运行速度快：
7、静态库的缺点
更新麻烦：
浪费内存：
8、动态库的缺点
运行速度慢：
程序无法执行：
9、动态库的优点
节约内存：
更新方便：
10、总结
六、进程映像
1、程序和进程

2、什么是进程映像：
3、各内存段分别存储了什么数据：
七、虚拟内存
1、什么是虚拟内存
2、用户空间与内核空间
用户空间：
内核空间：
3、进程使用虚拟内存好处有哪些：
八、内存映射
1、自动映射
2、手动映射
九、Linux系统内存管理API
1、遵循POSIX标准的内存管理API
2、Linux系统的内存管理
3、内存管理的层次关系
十、系统调用
1、什么是系统调用
2、普通函数调用与系统调用的区别
普通函数调用的步骤：
系统调用的步骤：
十一、文件与文件描述符
1、一切皆文件
2、文件描述符
3、文件描述符与文件指针：
十二、文件的创建与打开
问题1：C语言可以定义重名函数吗？
问题2：系统调用为什么可以重名？
问题3：r,w,a,r+,w+,a+分配对应哪些flags标志？
十三、文件操作
1、文件读写
2、系统IO与标准IO
3、文件位置指针
4、文件同步
5、文件描述符的状态标志
6、文件锁
文件锁的意义：
文件锁的限制：
7、复制文件描述符
十四、文件管理
1、获取文件的属性
2、文件的权限
测试文件的权限：
修改文件的权限：
3、文件的权限屏蔽码：
4、修改文件的大小
5、链接文件
硬链接文件的创建与删除：
软件链接文件的创建与读取：
十五、目录文件
1、工作目录
2、创建、删除、读取目录
十六、信号处理
1、什么是信号
2、不可靠信号与可靠信号
不可靠信号：
可靠信号：
3、信号来源和默认动作
4、捕获信号

5、发送信号的方式
通过键盘发送信号：
执行命令发送信号：
特殊事件发生时会发送信号：
调用函数发送信号：
6、暂停与休眠
7、带参的信号发送与捕获
8、计时器
系统为进程维护三个计时器：
获取/设置计时器：
十七、信号集与信号屏蔽
1、什么是信号集
2、操作信号集的相关函数：
3、信号的递送与未决
4、信号屏蔽
4、可靠与不可靠信号屏蔽的区别
十八、进程介绍
1、进程与程序
2、查看进程
简单形式：
复杂形式：
3、进程信息列表
4、父进程与子进程
5、孤儿进程与僵尸进程
6、进程标识符
十九、fork创建进程
1、父子进程谁先运行：
2、子进程是父进程的副本：
3、共享代码段、文件表：
4、子进程会继承父进程的信号处理方式：
二十、vfork和exec创建进程
1、vfork函数的特点
2、vfork与fork的区别
3、exec系列函数
4、system的实现原理
二一、进程的结束
1、进程的正常结束
1、main函数中执行return结束进程
2、调用标准C的exit函数结束进程
3、调用_exit/_Exit函数结束进程：
4、其它正常结束进程的方法
2、进程的异常终止
3、进程的资源回收
二二、进程间通信介绍
1、什么是进程间通信
2、为什么进程之间需要通信
3、那类型的问题需要多进程协同工作
进程的运行机制：
CPU密集型问题：
IO密集型问题：
二三、简单的进程间通信
1、命令行参数
2、环境变量表
4、信号
5、文件
二四、传统的进程间通信
1、有名管道：
使用函数创建：

使用命令创建：
使用管道单向通信：
数据接收者：
数据发送者：
使用管道进行进程间双向通信
进程A：
进程B：
2、匿名管道：
二五、XSI机制的进程间通信
1、XSI介绍
什么是XSI：
IPC对象：
IPC键值：
IPC描述符：
显示IPC对象命令：
删除IPC对象命令：
2、共享内存：
基本原理：
相关API：
编程模型：
3、消息队列：
基本原理：
相关API：
编程模型：
4、信号量
基本原理：
相关API：
编程模型：
二六、线程介绍
1、什么是线程
2、线程的发展简史
3、线程的调度策略
4、多线程适用的范围
5、线程的特点
轻型实体：
独立调度和分派的基本单位：
可并发执行：
共享进程资源：
6、线程与进程的区别
资源：
通信：
调度：
身份：
二七、POSIX线程库
1、POSIX线程库介绍：
2、API具体内容：
3、创建线程
4、线程回收
5、区分主、子线程
6、终止线程
7、线程分离
8、取消线程
向发送取消请求：
设置可取消状态：
设置可取消类型：
8、线程属性
1、设置线程属性：
初始化线程属性结构体：

设置具体线程属性项：
以设置好的线程属性结构体为参数创建线程：
销毁线程属性结构体：
2、获取线程属性
获取线程属性结构体：
获取具体线程属性项：
二八、线程同步
1、什么是线程同步
2、互斥锁
3、读写锁
4、死锁问题
1、什么是死锁：
2、死锁产生四大必要条件：
3、如休防止出现死锁
4、检测死锁的方法
5、原子操作
1、原子操作的优点
2、原子操作的缺点
6、信号量
1、多线程使用的信号量
2、多进程使用的信号量：
7、生产者与消费者模型
生产者：
消费者：
存储中介：
当生产快于消费：
当消费快于生产：
8、条件变量

课程内容介绍：

1. 系统环境：库、环境变量、编译器、系统特性
2. 内存管理：操作系统是如何管理内存的
3. 文件系统：文件读写、目录读写、文件属性、文件管理
4. 进程管理：多个程序同时运行，解决复杂问题
5. 信号处理：操作系统与程序之间一种通信机制
6. 进程通信：多个进程如何交互数据，这是它们协同工作的基础
7. 线程管理：就是让一个程序同时做若干件事情
8. 线程同步：让多个线程同时工作时能够不互相破坏、干扰

我们表面上的学习任务是学习如何使用操作系统的API，但核心任务就是学习操作系统的管理机制，了解操作系统的管理规则让程序能够更好的在操作系统下运行，通过大量阅读API的英文使用手册，提高自己的自学能力、英文技术文档的阅读能力。

一、UNIX、Linux系统介绍

1、UNIX系统介绍

诞生于1971年美国AT&T公司的贝尔实验室，主要开发者是丹尼斯.里奇、肯.汤普逊。

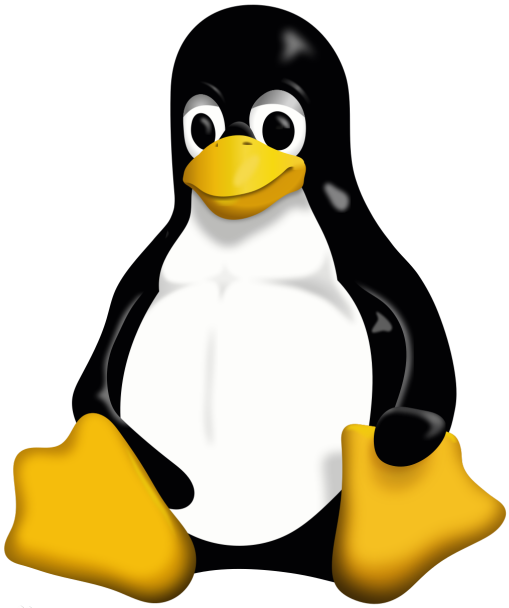
该系统的主要特点是支持多用户、多任务，并支持多种处理器架构，同时具有高安全性、高可靠性、高稳定性，既可以构建大型关键业务系统的商业服务器，也可以构建面向移动终端、手持设备、可穿戴设备的嵌入式应用。

2、Linux系统介绍

是一款类UNIX系统，免费开源，不同的发行版使用相同的内核，一般用在手机、平板、路由器、台式计算机、大型计算机、超级计算机，从严格意义上来说，Linux仅指的是操作系统内核，隶属于GNU工程，发明人叫 Linus Benedict Torvalds。

3、Linux系统的Logo

是一只企鹅，是南极的标志性动物，而目前南极不属于任何国家，为全人类所共有的，而Linux用它来当操作系统就意味这款系统属于全人类。



4、Minix操作系统

荷兰的Andrew S. Tanenbaum教授所开发的一款不包含任何UNIX源码的类UNIX系统，Linus Torvalds虽然深受Minix的启发写出了第一版本的Linux内核。

5、GNU工程

发起于1984年，由自由软件基金会提供支持，它基本原则就是共享，目的是发展出一个免费且开源的类UNIX系统，名称来自GNU's Not UNIX!的递归缩写，因为GNU的设计类似UNIX，但它不包含具著作权的Unix代码。GNU的创始人，理查德·马修·斯托曼，将GNU视为“达成社会目的技术方法”。

GNU的发展仍未完成，其中最大的问题是具有完备功能的内核尚未被开发成功。GNU的内核，称为Hurd，但是其发展尚未成熟。在实际使用上，多半使用Linux内核、FreeBSD等替代方案，作为系统核心，其中主要的操作系统是Linux的发行版。Linux操作系统包涵了Linux内核与其他自由软件项目中的GNU组件和软件，可以被称为GNU/Linux。

6、POSIX标准

可移植操作系统接口（Portable Operating System Interface，缩写为POSIX）是IEEE为要在各种UNIX操作系统上运行软件，而定义API的一系列互相关联的标准的总称。

Linux完全遵循了这个标准，所以两个操作系统的API，名字相同、参数相同、返回值相同，在Linux下编写的代码，经过稍微修改可移植到UNIX上。

7、GPL通用公共许可证

GNU通用公共许可证简称为GPL，是由自由软件基金会发行的用于计算机软件的协议证书，使用该证书的软件被称为自由软件。允许对某成果及其派生成果的重用、修改和复制，对所有人都是自由的，但不能声明做了原始工作，或声明由他人所做。

二、GNU编译工具

是GNU组织为了编译Linux内核源码而开发的一款编译工具，经过长时间的发展目前已经成为一个编译平台，能够支持多种编程语言，能在主流的操作系下使用，编译C代码的工具叫gcc，编译C++代码的工具叫g++。

1、gcc常用的编译参数

```
gcc [选项参数] 文件
    -E          # 预处理
    -S          # 生成汇编文件
    -c          # 生成目标文件
    -o          # 设置编译结果的名字
    -I          # 设置要导入的头文件的路径
    -l          # 设置要链接的库名，例如：使用sqrt、pow等数学函数时就需要链接数学库 -lm
    -L          # 设置要链接的库的路径
    -D          # 在编译时定义宏
    -g          # 编译时添加调试信息，这样编译出的程序可以用gdb调试。
    -wall       # 显示所有警告，编译器会以更严格的标准检查代码
    -werror     # 把警告当错误处理
    -std        # 指定编译器要遵循的语法标准，c89,c99,c11，当前系统默认的是c99标准。
    -pedantic   # 对不符合ANSI/ISO C语言标准的，扩展语法产生警告
```

2、gcc相关的文件类型

```
    xxx.h       # 头文件
    xxx.c       # 源文件
    xxx.i       # 预处理文件
    xxx.s       # 汇编文件
    xxx.o       # 目标文件
    xxx.h.gch   # 头文件的编译结果，用于检查自定义的头文件是否有语法错误，建议立即删除
    libxxx.a    # 静态库文件,windows系统下的静态库文件以lib结尾，例：xxx.lib
    libxxx.so   # 动态库文件,windows系统下的动态库文件以以dll结尾，例：libxxx.dll
```

3、gcc把C语言变成可执行程序的过程

```
# 1、把程序员所编写的代码进行预处理
gcc -E hello.c              # 把预处理的结果显示到屏幕上
gcc -E hello.c -o hello.i   # 会生成以.i结尾的预处理文件

# 2、把预处理的结果翻译成汇编代码
gcc -S hello.i              # 会生成以.s结尾的汇编文件

# 3、把汇编代码翻译成二进制指令
gcc -c hello.s              # 会生成以.o结尾的目标文件

# 4、把若干个文件目标文件、库文件合并成可执行文件
gcc a.o b.o c.o ...         # 默认会生成a.out可执行文件，也。
gcc a.o b.o c.o -o hello    # 可以使用-o指定可执行文件的名字
```

4、gcc支持的预处理指令

```
#include      // 将指定文件的内容插至此指令处
#include_next // 与#include一样，但从当前目录之后的目录查找，极少用
#define       // 定义宏
#undef        // 删除宏
#if           // 判定
#ifdef        // 判定宏是否已定义
#ifndef       // 判定宏是否未定义
#else         // 与#if、#ifdef、#ifndef结合使用
#elif         // else if多选分支
#endif        // 结束判定
##            // 连接宏内两个连续的字符串
#             // 将宏参数扩展成字符串字面值
#error        // 预处理时产生错误，结束预处理
#warning      // 预处理时产生警告信息
#pragma       // 提供额外信息的标准方法，可用于指定平台
#pragma GCC dependency <文件>     // 若<文件>比此文件新则产生警告
#pragma GCC poison <标识>         // 若出现<标识>则产生错误
#pragma pack(1/2/4/8)             // 按1/2/4/8字节对齐补齐
#line                             // 指定行号
```

5、gcc预定义的宏

```
void printf_macro (void)
{
    printf ("__BASE_FILE__      : %s\n", __BASE_FILE__);
    printf ("__FILE__           : %s\n", __FILE__);
    printf ("__LINE__           : %d\n", __LINE__);
    printf ("__FUNCTION__       : %s\n", __FUNCTION__);
    printf ("__func__           : %s\n", __func__);
    printf ("__DATE__           : %s\n", __DATE__);
    printf ("__TIME__           : %s\n", __TIME__);
    printf ("__INCLUDE_LEVEL__  : %d\n", __INCLUDE_LEVEL__);
    printf ("__cplusplus        : %d\n", __cplusplus);
}
```

三、环境变量

1、什么是环境变量

环境变量（environment variables）一般是指在操作系统中用来指定操作系统运行环境的一些参数，如：临时文件夹位置、系统文件夹位置等。

环境变量是在操作系统中一个具有特定名字的对象，它包含了一个或者多个应用程序所将使用到的信息。例如操作系统中的path环境变量，当要求系统运行一个程序而没有告诉它程序所在的完整路径时，系统除了在当前目录下面寻找此程序外，还应到path中指定的路径去找。用户通过设置环境变量，来更好的运行程序。

2、环境变量的主要作用

设置程序的运行参数：

环境变量相当于给系统或用户给应用程序设置的一些参数，具体起什么作用这当然和具体的环境变量相关，例如一个程序在运行时默认该使用什么语言，界面的大小，日志文件存储在什么位置，临时文件存储在什么位置，都需要通过查询操作系统的环境变量才能确定。

程序共用：

当软件A需要使用到软件B中部分功能时，但无论是操作系统还是软件A、B的作者都无论控制软件安装在什么位置，这时就可以让软件B设置环境变量告诉操作系统它安装在了哪个位置，而软件A就可以通过查询环境变量知道软件B安装在什么地方，从而调用软件B的功能。

系统运行：

用户还可以通过设置环境变量告诉操作系统一些运行参数，如设置当前系统的语言、字符编码、终端的默认大小等。

3、常见的环境变量：

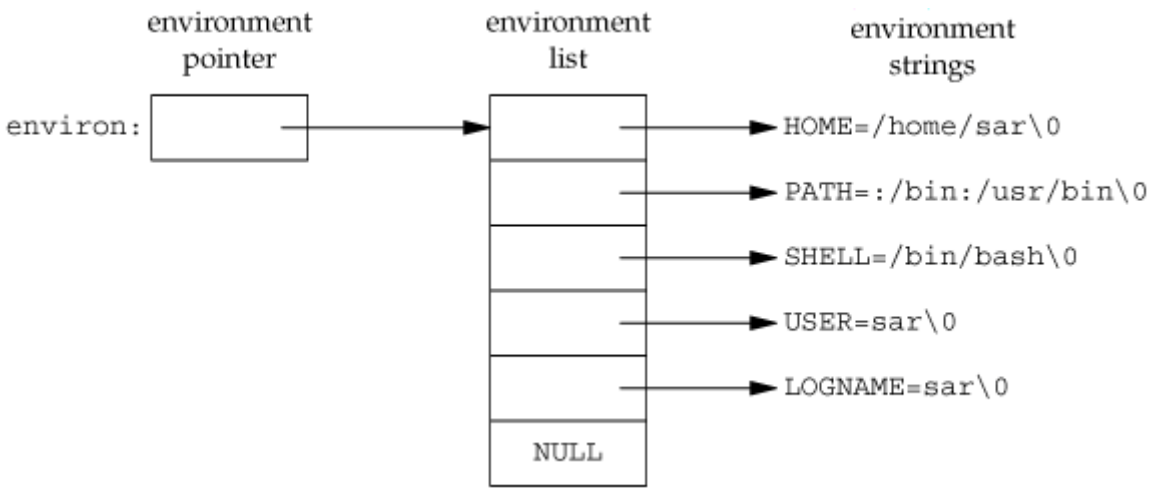
```
PS1              # 命令提示符
PATH             # 命令的搜索路径
C_INCLUDE_PATH   # 头文件的搜索路径
LIBRARY_PATH     # 库文件的搜索路径
LD_LIBRARY_PATH  # 程序执行时动态库的搜索路径
```

4、查看环境变量

1、Linux系统使用env命令查看环境变量

2、Windows系统使用set命令查看环境变量

3、在程序中查看




```c
#include <stdio.h>
// 程序在运行时父进程都会通过main函数参数传递一份环境变量表的拷贝，是一个以NULL指针结尾的字符指针数组。
int main(int argc,const char* argv[],char* environ[])
{
    for(int i=0; environ[i]; i++)
    {
        printf("%s\n",environ[i]);
    }
}
```

练习：解析出PATH环境变量的所有路径。

```c
#include <stdio.h>
#include <string.h>

void print_environ(const char *environ)
{
    // 开头= 中间用:分隔 '\0' 结尾
    for (const char *e = strchr(environ, '='); NULL != e; e = strchr(e + 1, ':'))
    {
        for (int i = 1; ':' != e[i] && '\0' != e[i]; i++)
        {
            printf("%c", e[i]);
        }
        printf("\n");
    }
}

int main(int argc, const char *argv[], char *environ[])
{
    for (int i = 0; NULL != environ[i]; i++)
    {
        if (0 == strncmp(environ[i], "PATH", 4))
        {
            print_environ(environ[i]);
        }
    }
    return 0;
}
```

5、Linux系统设置环境变量

```
# 打开配置文件：
vi ~/.bashrc            # 只对当前用户用效
vi /etc/environment     # 对所有用户有效

# 在文件末尾添加内容：
export C_INCLUDE_PATH=<环境变量的内容>
C_INCLUDE_PATH=$C_INCLUDE_PATH:<追加新的内容>

# 重新加载配置文件：
source ~/.bashrc
```

6、Windows系统设置环境变量

[Windows10系统修改环境变量的方法]

[Windows11系统修改环境变量的方法]

7、使用标准库函数设置环境变量：

```
环境变量和格式：name=value

// 根据name获得value
char *getenv(const char *name);

// 以name=value的形式设置环境变量，name不存在就添加，存在就覆盖其value
int putenv(char *string);

// 根据name设置value，注意最后一个参数表示，若name已不存在则添加，如果name已经存在是否覆盖其value由overwrite控制
// If name does exist in the  environment, then its value is changed to value if overwrite is nonzero; if overwrite is zero, then the value of name is not changed。
int setenv(const char *name, const char *value, int overwrite);

// 删除环境变量。
int unsetenv(const char *name);

// 清空环境变量，environ==NULL。
int clearenv(void);

注意：由于当前程序获得是环境变量表的拷贝(副本)，  因此在当前程序中对环境变量表进行增、删、改不会影响其它程序和操作系统，而修改后环境变量表会传给它的子进程，也就是会对子进程造成影响。
```

```
#include <stdio.h>
#include <stdlib.h>

void print_env(void)
{
    printf("-----------环境变量表start--------------\n");
    extern char **environ;
    for (int i = 0; NULL != environ && NULL != environ[i]; i++)
    {
        printf("%s\n", environ[i]);
    }
```

```
    printf("-----------环境变量表end--------------\n");
}

int main(int argc, const char *argv[])
{
    print_env();
    char *env = getenv("HEHE");
    if (NULL != env)
        printf("%s\n", env);

    printf("%s\n", putenv("HEHE=/home/zhizhen/") ? "添加失败!" : "添加成功!");
    env = getenv("HEHE");
    if (NULL != env)
        printf("%s\n", env);

    printf("%s\n", setenv("HEHE", "/home/xxxxx/", 1) ? "添加失败!" : "添加成功!");
    env = getenv("HEHE");
    if (NULL != env)
        printf("%s\n", env);

    unsetenv("HEHE");
    env = getenv("HEHE");
    if (NULL != env)
        printf("%s\n", env);

    clearenv();
    print_env();
    return 0;
}
```

四、错误处理

1、通过函数的返回值表示错误

```
// 返回合法值表示成功，返回非法值表示失败
long file_size (const char* path)
{
    FILE* fp = fopen (path, "r");
    if (NULL == fp)
        return -1;

    fseek (fp, 0, SEEK_END);
    long size = ftell (fp);

    fclose (fp);
    return size;
}

// 返回有效指针表示成功， 返回空指针(NULL/0xFFFFFFFF)表示失败
Node* query_list(Node* list,TYPE key)
{
    for(Node* n=list->head; NULL!=n; n=n->next)
    {
        if(n->data == key)
            return n;
    }
```

```
    return NULL;
}

// 返回0表示成功，非零表示失败，如：main、fseek、access等函数

// 永远成功的函数，如：menu类的函数
```

2、通过errno表示错误

errno 是记录系统的最后一次错误代码。代码是一个int型的值，在errno.h中定义。查看错误代码errno是调试程序的一个重要方法。

当调用Linux系统API函数发生异常时，一般会将errno变量赋一个整数值，不同的值表示不同的含义，可以通过查看该值推测出错的原因，在实际编程中用这一招解决了不少原本看来莫名其妙的问题。

errno在函数执行成功的情况下不会被修改，因此不能以errno非零，作为发生错误判断依据， errno是一个全局变量，其值随时可能发生变化，所以要配合函数的返回值来使用errno。

```
#include <stdio.h>
#include <errno.h>

int main ()
{
    FILE* fp = fopen ("none", "r");
    if (NULL == fp)
    {
        // 查看错误编号
        printf ("fopen: %d\n", errno);
        // 根据错误编号查看错误提示
        printf ("fopen: %s\n", strerror (errno));
        // 自动根据错误编号查看错误提示
        printf ("fopen: %m\n");
        // 自动根据错误编号查看错误提示
        perror ("fopen");
        return -1;
    }
    fclose (fp);
    return 0;
}
```

五、库文件的制作与使用

1、什么是库文件

库文件是计算机上的一类文件，提供给使用者一些开箱即用的变量、函数或类，它是若干个目标文件的集合，也可以对源码进行保密。库文件分为静态库和动态库，静态库和动态库的区别体现在程序的链接阶段。

2、静态库与动态库的区别

静态库在程序的链接阶段被复制到了程序中，动态库在链接阶段没有被复制到程序中，而是在程序运行时由系统动态加载到内存中供程序调用，这是它们最本质的区别。

3、静态库的制作与使用

创建静态库：

```
# 编译出目标文件：
gcc -c xxx1.c
gcc -c xxx2.c
...

# 把目标文件打包成静态库文件
ar -r libxxx.a x1.o x2.o ...

# ar 是一个专门控制静态库的命令
    -r # 把目录文件合并成一个静态库，如果静态库文件已经存在则更新。
    -q # 向静态库中添加目标文件
    -t # 查看静态库中有哪些目标文件
    -d # 从静态库中删除目标文件
    -x # 把静态库展开为目标文件
```

使用静态库：

```
# 方法1、直接调用，把共享库当作目标文件一样，与调用者的目标文件一起合并出可执行文件。
gcc main.c libxxx.a

# 方法2、通过设置LIBRARY_PATH环境变量来指定库的路径，通过-l参数来指定库名
gcc main.c -lxxx

# 方法3、通过gcc -L参数来指定库的路径，通过-l参数来指定库名
gcc main.c -L<PATH> -lxxx
```

4、动态库的制件与使用

创建动态库：

```
# 编译出目标文件，-fpic编译出位置无关代码，在代码中使用相对地址，这样共享库就可以加载到内存的任何位置。
gcc -c -fpic xxx1.c
gcc -c -fpic xxx2.c
...

# 把目标文件打包成共享库：
gcc -shared xxx1.o xxx2.o ... -o libxxx.so
```

使用动态库：

```
# 方法1、直接调用
gcc main.c libxxx.so

# 方法2、通过设置LIBRARY_PATH环境变量来指定库的路径，需要通过-l来指定库名
gcc main.c -lxxx


# 3、通过gcc -L参数来指定库的路径
gcc main.c -L<PATH> -lxxx

# 注意1：如果执行无法运行，需要检查操作系统是否能加载动态库，检查LD_LIBRARY_PATH环境变量
# 注意2：如果静态库和共享库同时存在，优先使用共享库，通过-static可以指定使用静态库。
```

动态加载共享库：

```
#include <dlfcn.h>
void *dlopen(const char *filename, int flag);
功能：打开共享库
filename：共享库的路径
flag：打开方式
    RTLD_LAZY：延迟加载，使用到共享库时再加载
    RTLD_NOW：立即加载
返回值：成功返回共享库的句柄，失败返回NULL

void *dlsym(void *handle, const char *symbol);
功能：通过函数名在共享库中获取函数指针
handle：共享库的句柄
symbol：函数名
返回值：函数地址，失败返回NULL

char *dlerror(void);
功能：获取错误信息

int dlclose(void *handle);
功能：卸载共享库
注意：编译时添加-ldl参数
```

```
#include <stdio.h>
#include <dlfcn.h>

int main(int argc, const char *argv[])
{
    void *handle = dlopen("libtools.so", RTLD_LAZY);
    if (NULL == handle)
    {
        printf("%s\n", dlerror());
        return -1;
    }

    void *ptr = dlsym(handle, "sex_to_str");
    if (NULL == ptr)
    {
        printf("%s\n", dlerror());
        return -1;
    }

```

```c
    const char *(*fp)(char) = ptr;
    printf("%s\n", fp('w'));

    dlclose(handle);
    return 0;
}
```

5、使用库文件时的辅助工具

```
# 查看目标文件、可执行文件、静态库、共享库文件的符号列表。
nm <flie>

# 删除目标文件、可执行文件、静态库、共享库文件中符号列表、调试信息，可以有效降低文件的大小。
strip <file>

# 查看可执行文件依赖了哪些共享库
ldd <file>

# 把二进制文件的内容转换成汇编代码，就是传说中的反汇编，但如果二进行文件进行加密机制则无法成功转换
objdump <file>
```

6、静态库的优点

使用方便：

在链接目标文件生成程序时，只需要把静态库与目标文件一起编译，编译器就会把目标文件中使用到的静态的内容拷贝到程序中，程序在运行时就不需要静态库了。

运行速度快：

静态库文件内容拷贝到程序中，因此在程序运行过程中，使用到静态库中的函数、变量时，只会在程序的内部跳转，因此使用静态库的程序要比使用动态库的程序运行速度快。

7、静态库的缺点

更新麻烦：

如果静态库中的内容发生了改变，如：版本升级，修改BUG，那么使用了相关静态库的程序就需要重新编译，如果是应用程序，用户就需要重新下载。

浪费内存：

假如有一个叫libxxx.a的静态库，a.out、b.out、c.out程序都使用使用它，那libxxx.a的内容会被分别拷贝一份到a.out、b.out、c.out，当这三个程序运行时，libxxx.a静态库文件中的内容就有三份存在于内存中，这样就有了冗余，造成内存的浪费。

8、动态库的缺点

在链接目标文件时，虽然动态库需要与使用它的目标文件一起编译，但编译器只是记录被调用的内容（函数、变量）在动态库中的位置，生成的程序中只有跳转到动态库的相关指令，当程序运行时需要把动态库一起加载到内存，当执行到动态库的相关内容时，才跳转到动态库所在的内存中执行，完成后再返回，这样会导致两个问题。

运行速度慢：

使用动态库程序需要在程序和动态库中来回跳转，因此要比使用静态库的程序运行的速度慢。

程序无法执行：

如果程序使用了动态库，当它运行时就需要系统把它使用的动态库一起加载到内存，如果系统找不到相应的动态库，那么程序就无法运行（Windows系统经常提示的xxx.dll文件缺失，程序无法运行），产生这种错误的原因有很多，如：环境变量配置错误，动态库文件存储位置错误，可执行文件拷贝到其它计算机上运行时没有一起拷贝动态库文件。

9、动态库的优点

节约内存：

使用的动态库只需被系统加载一次，不同的程序都可以使用到内存中的动态库，因此节约了很多内存，由于多个程序可共享使用一个动态库，所以动态库也叫共享库。

更新方便：

如果动态库中的函数格式没有变化(返回值、函数名、参数列表)，而只是函数中的业务逻辑代码发生变化，那么只需要重新编译动态库即可，不需要重新编译相关的可执行文件，这也是某些应用程序可以自动更新的原因。

10、总结

当一个模块不会再发生改变，并且执行速度有一些要求，适合把它封装成静态库。

随着计算机性能的不断提升，弥补了动态库运行速度慢的缺点，再加上它能节约内存、更新方便，最主要的是计算机硬件一直在升级，所以就导致大多数代码需要不断的升级，因此我们大多数情况下把模块封装成动态库。

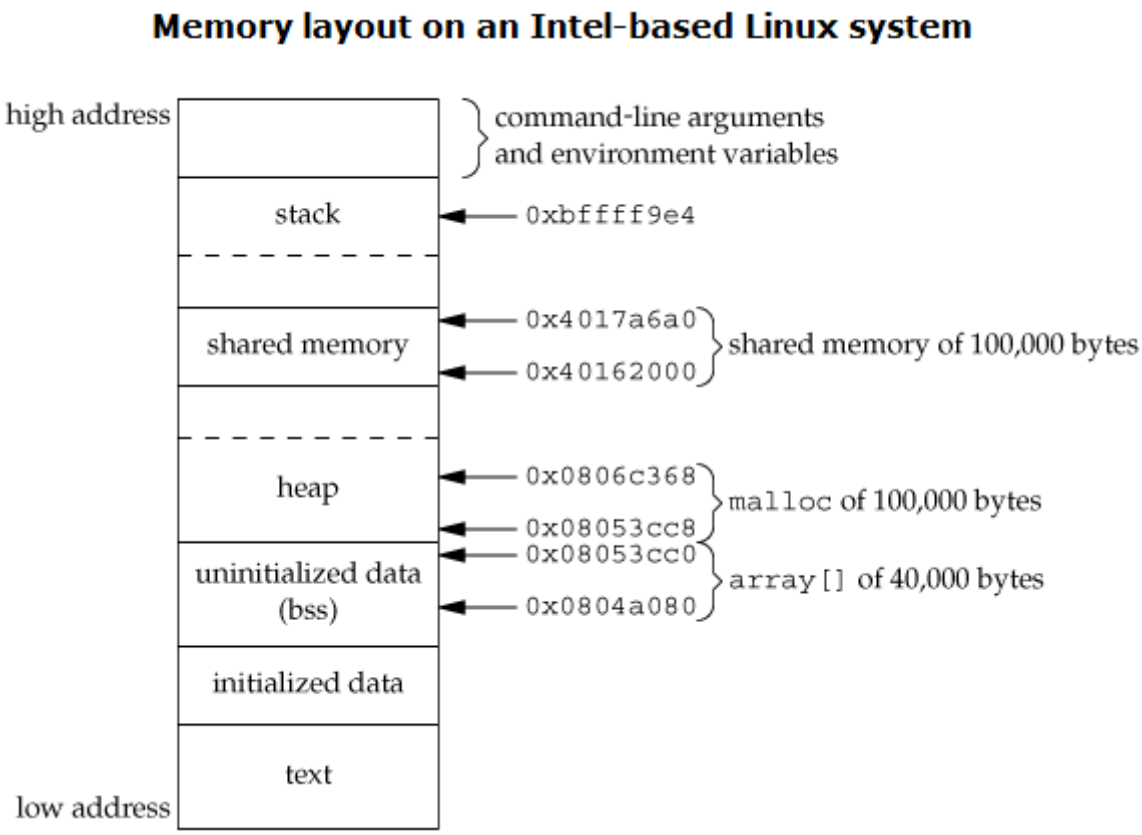
六、进程映像

1、程序和进程

程序是存储在磁盘上的可执行文件，当运行程序时系统就会把可执行文件加载到内存中运行，正在运行的程序叫进程，一个程序可以同时被执行多次，形成多个进程，每个进程互相独立，由操作系统管理。

2、什么是进程映像：

进程在内存空间中的布局就是进程映像，从低地址到高地址依次为：



3、各内存段分别存储了什么数据：

- 代码区(text)：
  里面存储的是二进制指令、字面值常量、被const修饰过的原data内存段的变量，权限为r--或r-x。
- 数据区(data)：
  里面存储的是初始化过的全局变量和静态变量。
- BSS区：
  里面存储的是未初始化的全局和静态局部变量，进程一经加载此内存段的所有字节就会被清理为0，所以全局变量和静态变量的默认值是0。
- 堆区(heap)：
  会被动态分配出去，从低地址向高地址扩展，在C语言中需要调用malloc系列函数进行管理，而malloc底层又调用了系统的API接口(brk/sbrk/mmap/munmap)。
- 栈区(stack)：
  里面存储的是非静态局部变量、块变量，包括函数的参数和返回值，从高地址向低地址扩展。
  堆区和栈区之间存在一块间隙，一方面为堆和栈的增长预留空间，同时共享库、共享内存等亦位于此。
- 命令行参数与环境变量表：
  里面存储的是命令行参数和环境变量。

```c
#include <stdio.h>
#include <stdlib.h>

const int const_global = 0; // 常全局变量
int init_global = 0;        // 初始化全局变量
int uninit_global;          // 未初始化全局变量

int main(int argc, char *argv[])
{
    const static int const_static = 0; // 常静态变量
    static int init_static = 0;        // 初始化静态变量
    static int uninit_static;          // 未初始化静态变量

```

```c
    const int const_local = 0; // 常局部变量
    int prev_local;            // 前局部变量
    int next_local;            // 后局部变量

    int *prev_heap = malloc(sizeof(int)); // 前堆变量
    int *next_heap = malloc(sizeof(int)); // 后堆变量

    const char *literal = "literal"; // 字面值常量
    extern char **environ;           // 环境变量

    printf("---- 命令行参数与环境变量 ---- <高>\n");
    printf("         环境变量：%p\n", environ);
    printf("       命令行参数：%p\n", argv);
    printf("-------------- 栈 ------------\n");
    printf("       常局部变量：%p\n", &const_local);
    printf("       前局部变量：%p\n", &prev_local);
    printf("       后局部变量：%p\n", &next_local);
    printf("-------------- 堆 ------------\n");
    printf("         后堆变量：%p\n", next_heap);
    printf("         前堆变量：%p\n", prev_heap);
    printf("------------- BSS ------------\n");
    printf(" 未初始化全局变量：%p\n", &uninit_global);
    printf(" 未初始化静态变量：%p\n", &uninit_static);
    printf("------------ 数据 ------------\n");
    printf("   初始化静态变量：%p\n", &init_static);
    printf("   初始化全局变量：%p\n", &init_global);
    printf("------------ 代码 ------------\n");
    printf("       常静态变量：%p\n", &const_static);
    printf("       字面值常量：%p\n", literal);
    printf("       常全局变量：%p\n", &const_global);
    printf("             函数：%p\n", main);
    printf("------------------------------ <低>\n");

    printf("查看/proc/%u/maps，按<回车>退出...", getpid());
    getchar();

    return 0;
}
```

七、虚拟内存

我们首先要理解一点就是操作系统不能把物理内存直接分配给进程使用，如果操作系统把物理内存直接分配给进程，这样就会面临严重的安全问题，进程可以根据自己获取到的内存地址，对指针进行运算从而访问其它进程的内存空间进行破坏，甚至影响操作系统的安全，所以操作系统的设计引入的虚拟内存的概念。

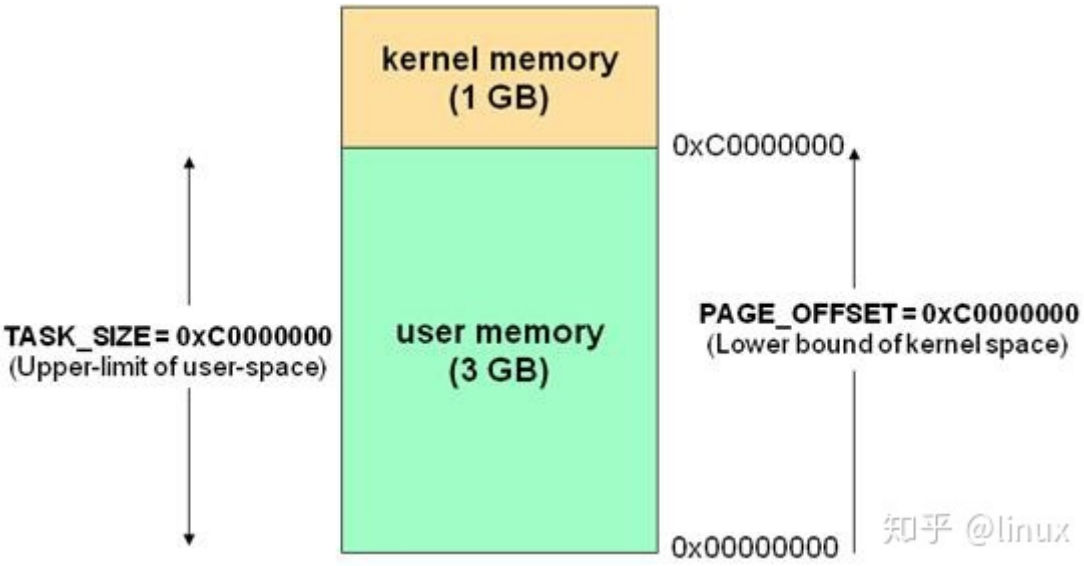
1、什么是虚拟内存

虚拟内存是操作系统内核为了对进程地址空间进行管理（process address space management）而精心设计的一个逻辑意义上的内存空间概念。

在32位系统下，操作系统规定每个进程拥有4G的虚拟内存，但这4G的内存空间并不能直接使用，也就是操作系统给进程画的饼，当进程真的需要使用内存来存储数据时，操作系统会把一部分的虚拟内存与物理内存进行映射，映射后的虚拟内存才能正常使用。

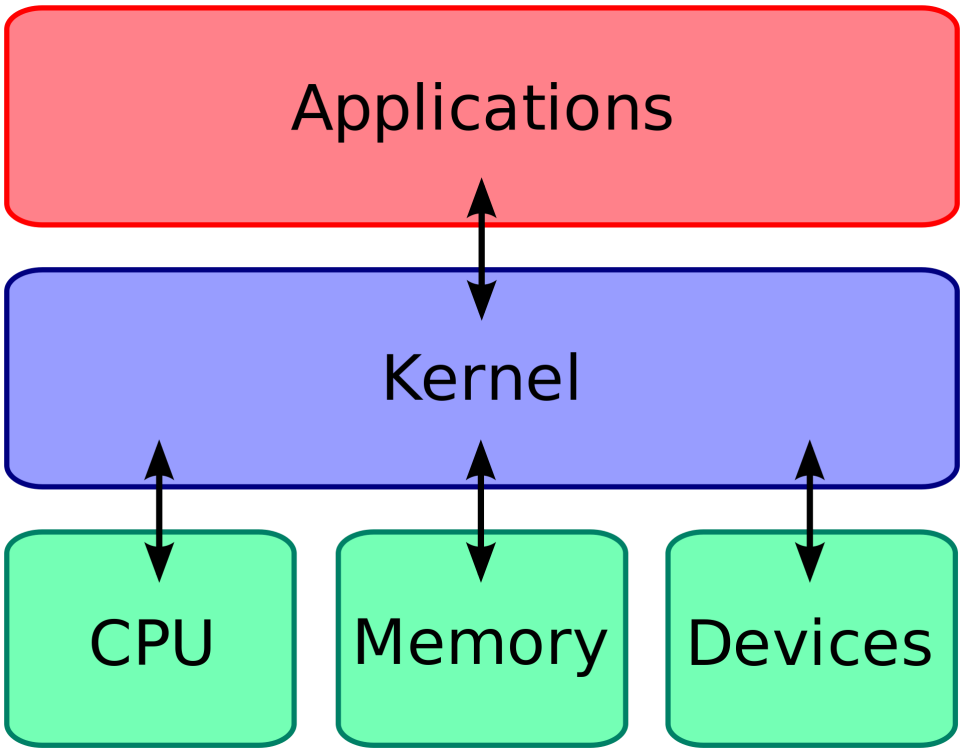
2、用户空间与内核空间

每个进程的4G虚拟内存，根据使用者的不同分为两个部分。



用户空间：

[0x00000000,0xC0000000)部分，有3GB大小，在经过操作系统的映射后，在应用程序中使用，程序中不能直接访问内核空间中的代码和数据，但可以通过系统调用进入内核态，间接地与系统内核交互。



内核空间：

[0xC0000000,0xFFFFFFFF]部分，有1GB大小，只有操作系统才能使用，里面存储着操作系统为该进程服务或者与该进程交互所需要的相关数据，内核空间由操作系统内核进行管理，不会随进程切换而改变。

用户空间对应进程，进程一切换，用户空间即随之变化，每个进程的内存空间完全独立。不同进程之间交换虚拟内存地址是毫无意义的，所以下面两个程序无法完成数据传递。

```
// 程序A
```

```c
int main(int argc, char *argv[])
{
    int num = 123456;
    printf("%lu",(unsigned long)&num);
    getchar();
}

// 程序B
int main(int argc, char *argv[])
{
    int* ptr = NULL;
    scanf("%lu",(unsigned long*)&num);
    printf("%d",*ptr);
}
```

3、进程使用虚拟内存好处有哪些：

1、操作系统为每个进程分配一个4G虚拟内存空间，可以对进程进行隔离，避免进程之间互相影响、破坏

2、操作系统把4G虚拟内存划分为用户态和内核态，可以对进程和操作系统进行隔离，避免进程对操作系统的影响破坏。

3、操作系统还可以把硬盘上的文件与虚拟内存进行映射，当物理内存不够用时，还可以使用硬盘代替，虽然速度慢了些。

注意：如果一个程序可以影响、破坏其它进程和操作系统，它就病毒了。

八、内存映射

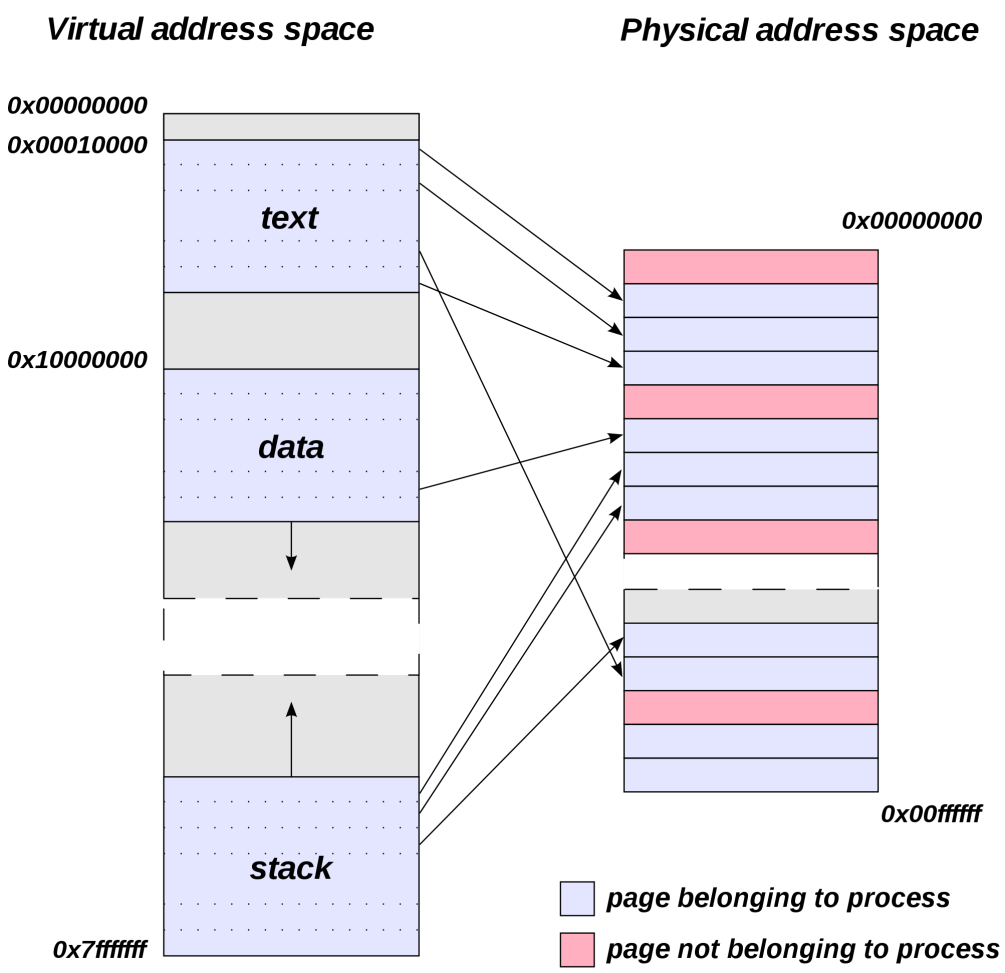
1、自动映射

当程序执行时，操作系统把它加载到内存形成进程后，会自动给该进行映射text、data、bss、statck、命令行参数表、环境变量表进行内存映射。

2、手动映射

在程序中首次使用malloc申请内存时，此时malloc手里没有堆内存可分配，也就是没有映射过的内存可供分配，malloc会向操作系统申请映射33页虚拟内存，这33页内存就归malloc管理了，之后再向malloc申请内存时，malloc会从这映射好的33页内存中分配给调用者，而malloc函数底层调用了操作系统API接口完成了映射操作。

如果程序要使用堆内存不多，达不到33页，那么使用malloc申请内存就会造成内存浪费，可以直接调用了操作系统API接口，进行更精确的内存映射。



九、Linux系统内存管理API

1、遵循POSIX标准的内存管理API

```
#include <unistd.h>

// brk和sbrk在内部维护一个指针p，指向当前堆内存最一个字节的下一个位置地址。
void *sbrk(intptr_t increment);
功能：根据参数来调整p的位置(p+/-increment)，既可以映射虚拟内存也可以取消映射。
返回值：调整前的p的位置。

int brk(void *addr);
功能：根据指针参数修改p的位置(把p设置为addr)。
返回值：成功返回0，失败返回-1。

// 注意：brk和sbrk都可以单独分配/释放内存，但一般配合使用，sbrk用户分配，brk用于释放。
// 注意：brk和sbrk只能用于对虚拟内存映射，无法做到精细化管理。
```

练习：计算出100到100000所有的素数，存储到内存中。

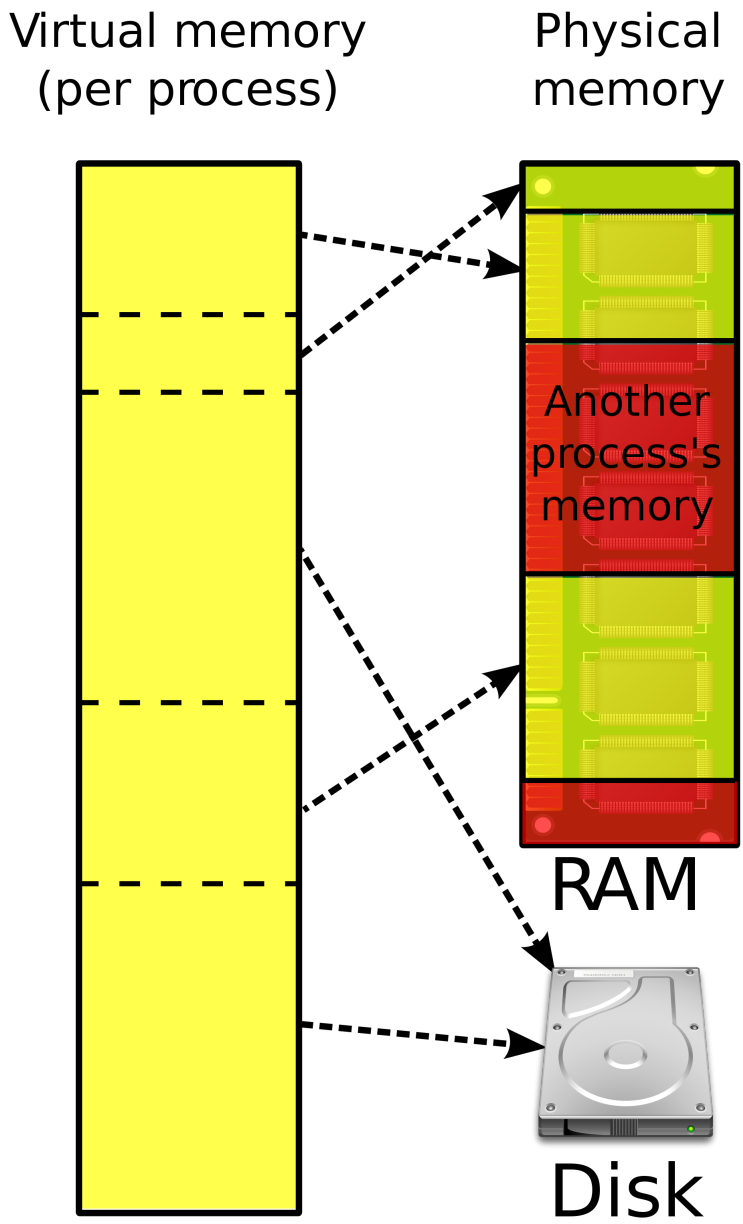
```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <unistd.h>
#include <math.h>

bool is_prime(int num)
```

```
{
    int end = sqrt(num);
    for(int i=2; i<=end; i++)
    {
        if(0 == num % i)
            return false;
    }
    return true;
}

int main(int argc,const char* argv[])
{
    int* base = sbrk(0);
    for(int i=100; i<10000; i++)
    {
        if(is_prime(i))
        {
            int* p = sbrk(4);
            *p = i;
        }
    }

    int *p = base, *end = sbrk(0);
    while(p < end)
    {
        printf("%d ",*p++);
    }
    printf("\n");
    brk(base);
    return 0;
}
```

2、Linux系统的内存管理

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,int fd, off_t offset);
功能：
    1、用户空间的虚拟内存与物理内存建立映射关系。
    2、用户空间的虚拟内存与文件建立映射关系。
addr：想要映射虚拟内存的首地址，如果是NULL则操作系统自动计算。
length：要映射的字节数，
prot：映射的内存权限
    PROT_EXEC  执行权限
    PROT_READ  读权限
    PROT_WRITE 写权限
    PROT_NONE  无权限
flags：
    MAP_SHARED 共享映射，映射的内容对其它进程是可见的，对共享区的写入，相当于输出到文件。
    MAP_PRIVATE 私有映射，映射的内容对其它进程不可见的
    MAP_DENYWRITE 拒绝其它文件写入操作
    MAP_ANON 映射的是内存而非文件
fd:文件描述符，类似于文件指针FILE
offset:映射文件时使用到的偏移值
返回值：映射成功后的虚拟内存地址，如果失败返回值为0xffffffff

```

```
int munmap(void *addr, size_t length);
功能：取消映射
addr：映射内存的首地址
length：内存的字节数
返回值：成功返回0，失败返回-1。

// 注意：系统的内存映射是以页为单位的(4096byte)
```




```
#include <stdio.h>
#include <sys/mman.h>

int main(int argc,const char* argv[])
{
    FILE* fp = fopen("test.txt","r+");
    if(NULL == fp)
    {
        perror("fopen");
        return -1;
    }
```

```

    int fd = fileno(fp);
    char* str = mmap(NULL, 100, PROT_WRITE|PROT_READ, MAP_SHARED|MAP_FILE, fd, 0);
    printf("%s\n",str);
    sprintf(str,"Hello World\n");
    munmap(str,100);
    fclose(fp);
    return 0;
}
```

3、内存管理的层次关系

```
+----+--------+----------------------------+----------+
| 用 | STL    | 自动分配/释放内存资源       | 调C++    |
|    | C++    | new/delete，构造/析构       | 调标C    |
| 户 | 标C    | malloc/calloc/realloc/free | 调POSIX  |
|    | POSIX  | brk/sbrk                   | 调Linux  |
| 层 | Linux  | mmap/munmap                | 调Kernel |
+----+--------+----------------------------+----------+
| 系 | Kernel | kmalloc/vmalloc            | 调Driver |
| 统 | Driver | get_free_page              | ...      |
| 层 | ...    | ...                        | ...      |
+----+--------+----------------------------+----------+
```

十、系统调用

1、什么是系统调用

由操作系统向应用程序提供的程序接口信息，也叫应用编程接口(Application Programming Interface，API)，它是是应用程序与操作系统之间交互的接口。

操作系统的主要功能是为管理硬件资源和为应用程序开发人员提供良好的环境，使应用程序具有更好的兼容性，为了达到这个目的，内核提供了一系列具备了一定功能的内核函数，通过一组称为系统调用（system call)的接口呈现给用户。

系统调用负责把应用程序的请求传给内核，调用相应的内核函数完成所需的处理，然后将处理结果返回给应用程序。

UNIX/Linux大分部的系统功能是通过系统调用实现的，这些系统调用被封装成了C函数的形式，但它们并不是真正的函数。

标准库函数大部分工作在用户态，一部分函数会使用系统调用进入内核态(fopen/malloc)，可以使用time命令统计程序的在用户态和内核态的运行时间:

```
time ./<可执行文件>
real    0m0.002s    # 总执行时间
user    0m0.000s    # 用户态执行时间
sys     0m0.000s    # 内核态执行时间
总执行时间 = 用户态执行时间 + 内核态执行时间 + 用户态和内核态切换消耗的时间

strace ./可执行文件名 # 追踪函数的调用过程
```

2、普通函数调用与系统调用的区别

普通函数调用的步骤：

1、调用者会先把要传递的参数压入栈内存

2、根据函数名也就是函数地址跳转到函数所在的代码段位置

3、接下来从栈内存中弹出参数

4、定义局部变量扩展栈内存并执行函数中的相关代码

5、返回执行结果

6、销毁该函数的栈内存

7、回到调用处继续执行

系统调用的步骤：

1、程序中执行到系统调用位置，触发软件中断机制

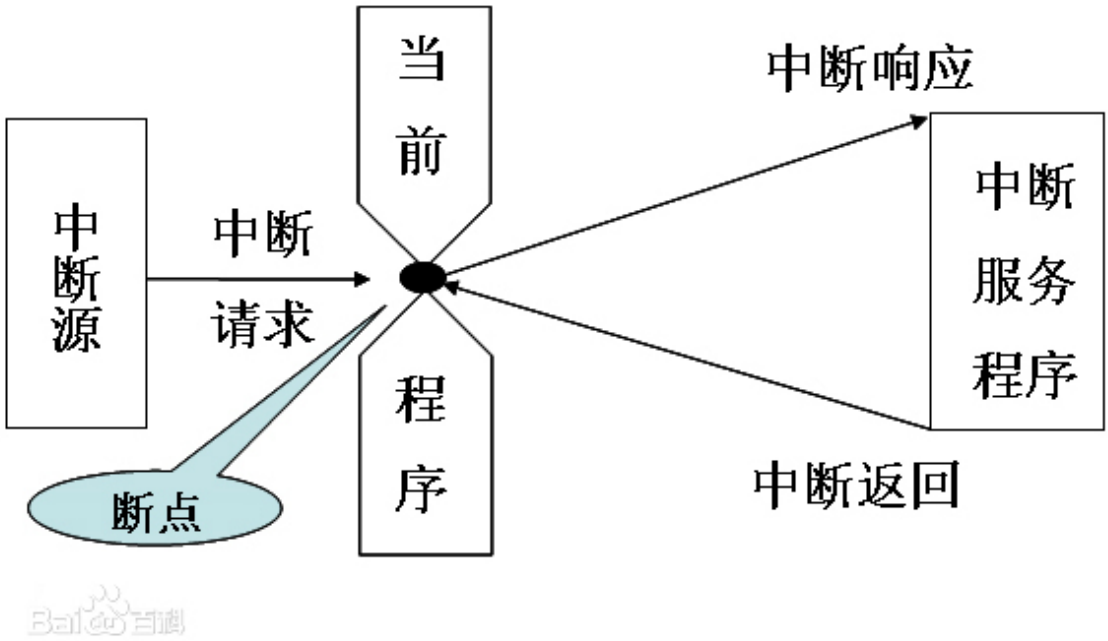
1、通过软中断机制进入内核运行状态

2、然后内核负责把参数从用户空间拷贝到内核空间

3、然后根据中断编号再执行相关操作

4、等执行完毕后，再把执行结果内核空间拷贝到用户空间

5、返回到程序再继续执行



十一、文件与文件描述符

1、一切皆文件

在UNIX和Linux系统下，把操作系统提供的服务和设备都抽象成了文件，因为这样可以给各种设备控制提供了一个简单而统一的接口，程序完全可以象访问普通磁盘文件一样，访问串行口、网络、打印机或其它设备。

在UNIX和Linux系统下中的文件具有特别重要的意义，因为它在Linux中，(几乎)一切皆文件，大多数情况下只需要使用五个基本系统调用 open/close/read/write/ioctl，即可实现对各种设备的输入和输出，Linux中的任何对象都可以被视为某种特定类型的文件，可以访问文件的方式访问之。

- 1) 目录文件
- 2) 设备文件
    - A. 控制台：/dev/console
    - B. 声卡：/dev/audio
    - C. 标准输入输出：/dev/tty
    - D. 空设备：/dev/null
- 3) 普通文件

2、文件描述符

文件描述符是一种非负的整数，表示一个打开的文件，由系统调用(open/creat)返回，在后续操作文件时可以被内核空间引用，内核默认为每个进程打开三个文件描述符：

- stdin　0 - 标准输入
- stdout 1 - 标准输出
- stderr 2 - 标准出错

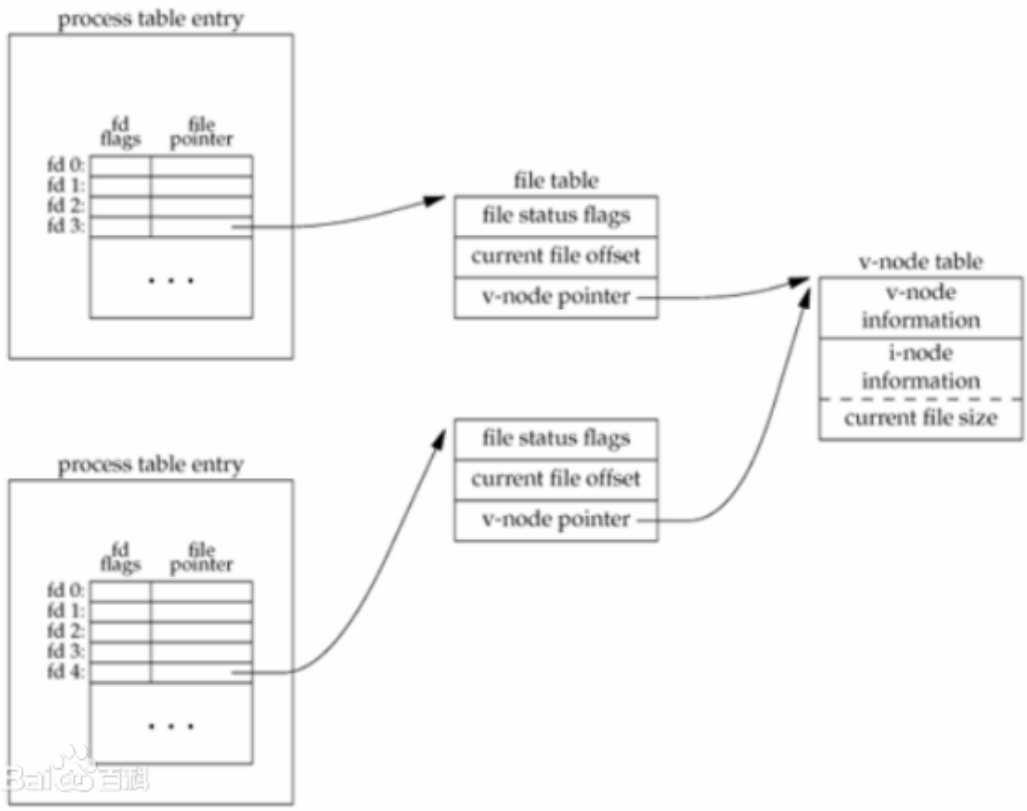
在unistd.h中被定义为如下三个宏：

```
#define STDIN_FILENO  0
#define STDOUT_FILENO 1
#define STDERR_FILENO 2
```

3、文件描述符与文件指针：

在Linux系统中打开文件后，内存中（内核区间）就会有一个内核对象，也就是记录该文件相关信息的结构体变量，但内核为了自己的安全不能把它的地址返回给用户，而且由于内核区间和用户区间的原因，返回也无法访问、使用。

而且一个进程可能会同时打开多份文件，所以操作系统就在内核区间创建了一张索引表，表的每一项都有一个指向已打开文件的内核对象，文件描述符就是索引表的主键，如果把索引表看作数组，那么文件描述符就是数组的下标，不同进程之间交换文件描述没有意义。



C语言中使用文件指针代表打开的文件，文件指针指向进程的用户区间中的一个被称为FILE结构的数据结构。FILE结构包括一个缓冲区和一个文件描述符。而文件描述符是文件描述符表的一个索引，因此从某种意义上说文件指针就是文件描述符。

```
int fileno(FILE *stream);
功能：把文件指针转换成文件描述符

FILE *fdopen(int fd, const char *mode);
功能：把文件描述符转换成文件指针
```

十二、文件的创建与打开

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int flags);
功能：打开文件
pathname：文件路径
flags：打开文件的方式
返回值：文件描述符，失败返回负值

int open(const char *pathname, int flags, mode_t mode);
功能：打开或创建文件
pathname：文件路径
flags：打开方式
mode：创建文件时的权限
返回值：文件描述符，失败返回负值

int creat(const char *pathname, mode_t mode);
功能：专门用来创建文件，但基本不使用，因为open函数完全具备它的功能。

```

```
注意：open/creat所返回的一定是当前未被使用的，最小文件描述符。
一个进程可以同时打开的文件描述符个数，受limits.h中定义的OPEN_MAX宏的限制，POSIX要求不低于16，传统UNIX是63，现代Linux是255。

flags:
    O_APPEND 打开文件后位置指针指向末尾
    O_CREAT 文件不存在时创建
    O_RDONLY 只读权限
    O_WRONLY 只写权限
    O_RDWR 读写权限
    O_TRUNC 清空文件内容
    O_EXCL 如果文件存在则创建失败
    O_NOCTTY 若pathname指向控制终端，则不将该终端作为控制终端。
    O_NONBLOCK 若pathname指向FIFO/块/字符文件，则该文件的打开及后续操作均为非阻塞模式。
    O_SYNC  write等待数据和属性，被物理地写入底层硬件后再返回。
    O_DSYNC write等待数据，被物理地写入底层硬件后再返回。
    O_RSYNC read等待对所访问区域的所有写操作，全部完成后再读取并返回。
    O_ASYNC 当文件描述符可读/写时，向调用进程发送SIGIO信号。
mode:
    S_IRWXU  00700
    S_IRUSR  00400
    S_IWUSR  00200
    S_IXUSR  00100

    S_IRWXG  00070
    S_IRGRP  00040
    S_IWGRP  00020
    S_IXGRP  00010

    S_IRWXO  00007
    S_IROTH  00004
    S_IWOTH  00002
    S_IXOTH  00001
int close (int fd);
功能：关闭文件，成功返回0，失败返回-1
```

问题1：C语言可以定义重名函数吗？

可以，但需要在不同的作用域下才可以重名。

情况1：在不同的源文件中，static函数可以与普通函数重名。

情况2：在函数内定义的函数可以与普通函数重名。

问题2：系统调用为什么可以重名？

因为系统调用不是真正的函数，而是借助软中断实现的，决定执行哪个系统调用的是中断编号，而不是名字。

问题3：r,w,a,r+,w+,a+分配对应哪些flags标志？

```
"r"      O_RDONLY
"r+"     O_RDWR
"w"      O_WRONLY|O_CREAT|O_TRUNC, 0666
"w+"     O_RDWR|O_CREAT|O_TRUNC, 0666
"a"      O_WRONLY|O_CREAT|O_APPEND, 0666
"a+"     O_RDWR|O_CREAT|O_APPEND, 0666
```

十三、文件操作

1、文件读写

```
ssize_t write(int fd, const void *buf, size_t count);
功能：写入文件内容
fd：文件描述符
buf：要写入的数据的内存首地址
count：要写入的字节数
返回值：成功写入的字节数

ssize_t read(int fd, void *buf, size_t count);
功能：读取文件内容
buf：存储数据的内存首地址
count：想读取的内存字节数
返回值：成功读取到的字节数

注意：它们与标准C的fwrite/fread很像，但更纯粹。
```

练习：以二进制形式写入1000000个整数到文件中，分别用标准文件读写和系统文件读写来完成，比较它们谁的速度更快，为什么？

```
// 标准IO
#include <stdio.h>

int main(int argc,const char* argv[])
{
    FILE* wfp = fopen("stdio.dat","w");
    if(NULL == wfp)
    {
        perror("fopen");
        return -1;
    }

    for(int i = 0; i<10000000; i++)
    {
        fwrite(&i,1,4,wfp);
    }

    fclose(wfp);
    return 0;
}
/*
time ./stdio
real    0m0.224s
user    0m0.195s
sys     0m0.028s
*/
```

```c
// 系统IO
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    int fd = open("sysio.dat",O_WRONLY|O_CREAT|O_TRUNC,0644);
    if(0 > fd)
    {
        perror("open");
        return -1;
    }

    for(int i=0; i<1000000; i++)
    {
        write(fd,&i,4);
    }

    close(fd);
    return 0;
}
/*
time ./sysio
real    0m1.137s
user    0m0.104s
sys     0m1.033s
*/
```

2、系统IO与标准IO

1、当系统调用被执行时，需要从用户态切换到内核态，执行完毕后再从内核态切换到用户态，频繁的切换就会导致性能损失。

2、标准IO在内部维护一个缓冲区(1k，1024字节)，只有在满足特定条件才会把缓冲区中的数据调用write进入写入，因此降低了系统调用的使用频率，减少用户态和内核态的来回切换次数，因此标准IO的速度比系统IO更快

3、如果想提高系统IO的速度，可以尝试维护一个更大的缓冲区，先把数据存储在缓冲区中，等缓冲区满了，再调用write写入，这样系统IO会比标准IO更快。

4、普通情况建议使用标准IO，因为比直接使用系统IO要快，如果对速度有很高的要求，可以使用系统IO+大缓冲区。

```c
// 系统IO+大缓冲区
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    int fd = open("sysio.dat",O_WRONLY|O_CREAT|O_TRUNC,0644);
    if(0 > fd)
```

```
    {
        perror("open");
        return -1;
    }

    int buf[4096];
    for(int i=0; i<1000000; i++)
    {
        buf[i%4096] = i;
        if(i%4096 == 4095)
            write(fd,buf,sizeof(buf));
    }

    close(fd);
    return 0;
}
/*
time ./sysio
real    0m0.005s
user    0m0.000s
sys     0m0.005s
*/
```

5、UNIX和Linux只有这一套读写函数，没有文本文件的读写方式，可以使用ssanf/sprintf配合缓冲区来实现文本内容的读写。

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

typedef struct Student
{
    int id;
    char name[20];
    float score;
}Student;

int main(int argc,const char* argv[])
{
    int fd = open("text.txt",O_RDWR);
    if(0 > fd)
    {
        perror("open");
        return -1;
    }
    Student stu = {10012,"hehe",365};

    char buf[256];
    sprintf(buf,"%d %s %g\n",stu.id,stu.name,stu.score);
    write(fd,buf,strlen(buf));

    close(fd);
    return 0;
}
```

练习：使用系统IO实现带覆盖检查的cp命令。

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    if(3 != argc)
    {
        puts("Use: ./cp src dest");
        return 0;
    }

    int src_fd = open(argv[1],O_RDONLY);
    if(0 > src_fd)
    {
        perror("open src");
        return -1;
    }

    int dest_fd = open(argv[2],O_WRONLY|O_CREAT|O_EXCL,0644);
    if(0 > dest_fd)
    {
        printf("%s 文件已经存在是否覆盖？(y/n)",argv[2]);
        char cmd = getchar();
        if('y' != cmd && 'Y' != cmd)
        {
            close(src_fd);
            printf("停止拷贝！\n");
            return 0;
        }
        dest_fd = open(argv[2],O_WRONLY|O_TRUNC);
        if(0 > dest_fd)
        {
            perror("open dest");
            return -1;
        }
    }

    int ret = 0;
    char buf[4096];
    while(ret = read(src_fd,buf,sizeof(buf)))
    {
        write(dest_fd,buf,ret);
    }

    close(src_fd);
    close(dest_fd);
    return 0;
}
```

3、文件位置指针

1、每个打开的文件都有一个记录读写位置的变量，它可能是整数，但习惯的称作位置指针，文件的读写操作都是从位置指针所指向地方进行的。

2、lseek可以设置文件的位置指针，与标准C不一样的是它的返回值是它调整后的位置指针，所以系统调用中没有与ftell对应的函数，因为lseek就包含fseek和ftell的功能。

```
off_t lseek(int fd, off_t offset, int whence);
功能：调整文件位置指针，用法与标准C的fseek基本一致。
offset：偏移值
whence：基础位置
    SEEK_SET    The offset is set to offset bytes.
    SEEK_CUR    The offset is set to its current location plus offset bytes.
    SEEK_END    The offset is set to the size of the file plus offset bytes.
```

3、在超过文件末尾的位置写入数据就会形成文件黑洞，黑洞不会占用磁盘空间，但会计算成文件的大小，而且也不会影响文件的读写。

练习：使用lseek实现一个计算文件大小的命令，显示出文件的字节数，超过1024字节显示KB，超过1024KB显示MB，超过1024MB显示GB

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    if(2 != argc)
    {
        puts("Use: ./filesize <name>");
        return 0;
    }

    int fd = open(argv[1],O_RDONLY);
    if(0 > fd)
    {
        printf("open %s %m\n",argv[1]);
        return -1;
    }

    size_t bytes = lseek(fd,0,SEEK_END);
    printf("%u\n",bytes);


    if(bytes > 1024*1024*1024)
        printf("%uGB %uMB %uKB %uByte\n",
            bytes/1024/1024/1024,
            bytes/1024/1024%1024,
            bytes/1024%1024,
            bytes%1024);
    else if(bytes > 1024*1024)
        printf("%uMB %uKB %uByte\n",
            bytes/1024/1024 % 1024,
            bytes/1024%1024,
```

```
            bytes%1024);
    else if(bytes > 1024)
        printf("%uKB %uByte\n",
            bytes/1024%1024,
            bytes%1024);
    else
        printf("%uByte\n",bytes);

    close(fd);
    return 0;
}
```

4、文件同步

大多数磁盘I/O都有缓冲机制，写入文件其实只是写入缓冲区，直到缓冲区满，才将其排入写队列。延迟写降低了写操作的次数，提高了写操作的效率，但可能导致磁盘文件与缓冲区数据不同步，可使用以下函数强制让磁盘缓冲区的内容同步磁盘。

```
            +-fwrite->标准库缓冲-fflush-+          sync
应用程序内存 +                          +->内核缓冲-fdatasync->磁盘(缓冲)
            +-----------write-----------+          fsync
```

```
#include <unistd.h>

void sync (void);        // 将所有被修改过的缓冲区排入写队列即返回，不等待写磁盘操作完成
int fsync (int fd);      // 只针对一个文件，且直到写磁盘操作完成才返回，成功返回0，失败返回-1。
int fdatasync (int fd); // 只同步文件数据，直到写磁盘操作完成才返回，不同步文件属性

注意：如果文件描述符的flags有SYNC相关标记，也能达到同步的效果。
```

5、文件描述符的状态标志

```
int fcntl (int fd, int cmd, long arg);
功能：设置或获取文件描述符的状态标志

cmd:
    F_GETFD - 获取文件描述符标志。
    F_SETFD - 设置文件描述符标志，目前仅定义了一个文件描述符标志位FD_CLOEXEC：
          0 - 在通过execve()函数所创建的进程中，该文件描述符依然保持打开。
          1 - 在通过execve()函数所创建的进程中，该文件描述符将被关闭。

    F_GETFL - 获取文件状态标志，不能获取O_CREAT/O_EXCL/O_TRUNC。
    F_SETFL - 追加文件状态标志，只能追加O_APPEND/O_NONBLOCK。
```

```
#include <stdio.h>
#include <fcntl.h>

void pflags(int flags)
{
    printf("文件状态标志(%08X)：", flags);

    struct
    {
        int flag;
```

```
        const char *desc;
    } flist[] = {
        O_RDONLY, "O_RDONLY",
        O_WRONLY, "O_WRONLY",
        O_RDWR, "O_RDWR",
        O_APPEND, "O_APPEND",
        O_CREAT, "O_CREAT",
        O_EXCL, "O_EXCL",
        O_TRUNC, "O_TRUNC",
        O_NOCTTY, "O_NOCTTY",
        O_NONBLOCK, "O_NONBLOCK",
        O_SYNC, "O_SYNC",
        O_DSYNC, "O_DSYNC",
        O_RSYNC, "O_RSYNC",
        O_ASYNC, "O_ASYNC"};

    size_t i;
    int first = 1;
    for (i = 0; i < sizeof(flist) / sizeof(flist[0]); i++)
    {
        if (flags & flist[i].flag)
        {
            printf("%s%s", first ? "" : " | ", flist[i].desc);
            first = 0;
        }
    }
    printf("\n");
}

int main()
{
    int fd = open("flags.txt", O_WRONLY | O_CREAT | O_TRUNC | O_ASYNC, 0644);
    if (fd == -1)
    {
        perror("open");
        return -1;
    }

    int flags = fcntl(fd, F_GETFL);
    if (flags == -1)
    {
        perror("fcntl");
        return -1;
    }

    pflags(flags); // 不能获取O_CREATE/O_EXCL/O_TRUNC

    if (fcntl(fd, F_SETFL, O_RDWR | O_APPEND | O_NONBLOCK) == -1)
    {
        perror("fcntl");
        return -1;
    }

    if ((flags = fcntl(fd, F_GETFL)) == -1)
    {
        perror("fcntl");
        return -1;
```

```
    }

    pflags(flags); // 只能追加O_APPEND/O_NONBLOCK
    close(fd);
    return 0;
}
```

6、文件锁

文件锁的意义：

当多个进程访问同一个文件时，就有可能造成文件的读写数据混乱，而解决这一问题的方法就是读写文件前给文件加锁。

文件锁的限制：

一般情况下，操作系统提供的文件锁都是劝诫锁，如果需要强制锁，则需要自己定制，也就是重新配置、编译内核。

```
#include <fcntl.h>

int fcntl (int fd, int cmd, struct flock* lock);

struct flock {
    short int l_type;   // 锁的类型：
        // F_RDLCK读锁
        // F_WRLCK写锁
        // F_UNLCK解锁
    short int l_whence; // 偏移起点：
        // SEEK_SET文件头
        // SEEK_CUR当前位置
        // SEEK_END文件尾
    off_t l_start;      // 锁区偏移，从l_whence开始
    off_t l_len;        // 锁区长度，0表示锁到文件尾
    pid_t l_pid;        // 加锁进程，-1表示自动设置
};

cmd取值：
    F_GETLK  - 测试lock所表示的锁是否可加。
        若可加则将lock.l_type置为F_UNLCK，否则通过lock返回当前锁的信息。

    F_SETLK  - 设置锁定状态为lock.l_type，
    成功返回0，失败返回-1，若因其它进程持有锁而导致失败，则errno为EACCES或EAGAIN。

    F_SETLKW - 设置锁定状态为lock.l_type，
    成功返回0，否则一直等待，除非被信号打断返回-1。
```

```
#include <stdio.h>
#include <fcntl.h>

// 打印锁信息
void plock(struct flock lock)
{
    if (lock.l_type == F_UNLCK)
        printf("没有锁。\n");
    else
    {
```

```c
        printf("%d进程", lock.l_pid);
        switch (lock.l_whence)
        {
        case SEEK_SET:
            printf("在距文件头");
            break;
        case SEEK_CUR:
            printf("在距当前位置");
            break;
        case SEEK_END:
            printf("在距文件尾");
            break;
        }

        printf("%d字节处，为%d字节加了", lock.l_start, lock.l_len);

        switch (lock.l_type)
        {
        case F_RDLCK:
            printf("读锁。\n");
            break;
        case F_WRLCK:
            printf("写锁。\n");
            break;
        }
    }
}

// 读锁测试
int rtest(int fd, off_t start, off_t len)
{
    struct flock lock;

    lock.l_type = F_RDLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = len;
    lock.l_pid = -1;

    if (fcntl(fd, F_GETLK, &lock) == -1)
        return -1;

    plock(lock);
    return 0;
}

// 写锁测试
int wtest(int fd, off_t start, off_t len)
{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = len;
    lock.l_pid = -1;

    if (fcntl(fd, F_GETLK, &lock) == -1)
```

```
        return -1;

    plock(lock);
    return 0;
}

// 加读锁
int rlock(int fd, off_t start, off_t len, int wait)
{
    struct flock lock;

    lock.l_type = F_RDLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = len;
    lock.l_pid = -1;

    return fcntl(fd, wait ? F_SETLKW : F_SETLK, &lock);
}

// 加写锁
int wlock(int fd, off_t start, off_t len, int wait)
{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = len;
    lock.l_pid = -1;

    return fcntl(fd, wait ? F_SETLKW : F_SETLK, &lock);
}

// 解锁
int ulock(int fd, off_t start, off_t len)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = start;
    lock.l_len = len;
    lock.l_pid = -1;

    return fcntl(fd, F_SETLK, &lock);
}

int main()
{
    int fd = open("lock.txt", O_RDWR);
    if (fd == -1)
    {
        perror("open");
        return -1;
    }

    // 对CDEF做读锁测试
```

```c
    printf("对CDEF做读锁测试。");
    if (rtest(fd, 2, 4) == -1)
    {
        printf("失败：%m\n");
        return -1;
    }

    // 对CDEF加读锁
    printf("对CDEF加读锁");
    if (rlock(fd, 2, 4, 0) == -1)
        printf("失败：%m\n");
    else
    {
        printf("成功！\n");
        ulock(fd, 2, 4);
    }

    // 对CDEF做写锁测试
    printf("对CDEF做写锁测试。");
    if (wtest(fd, 2, 4) == -1)
    {
        printf("失败：%m\n");
        return -1;
    }

    // 对CDEF加写锁
    printf("对CDEF加写锁");
    if (wlock(fd, 2, 4, 0) == -1)
        printf("失败：%m\n");
    else
    {
        printf("成功！\n");
        ulock(fd, 2, 4);
    }

    // 对KLMN做读锁测试
    printf("对KLMN做读锁测试。");
    if (rtest(fd, 10, 4) == -1)
    {
        printf("失败：%m\n");
        return -1;
    }

    // 对KLMN加读锁
    printf("对KLMN加读锁");
    if (rlock(fd, 10, 4, 0) == -1)
        printf("失败：%m\n");
    else
    {
        printf("成功！\n");
        ulock(fd, 10, 4);
    }

    // 对KLMN做写锁测试
    printf("对KLMN做写锁测试。");
    if (wtest(fd, 10, 4) == -1)
    {
        printf("失败：%m\n");
```

```c
        return -1;
    }

    // 对KLMN加写锁
    printf("对KLMN加写锁");
    if (wlock(fd, 10, 4, 0) == -1)
        printf("失败：%m\n");
    else
    {
        printf("成功！\n");
        ulock(fd, 10, 4);
    }

    printf("等待KLMN上的写锁被解除...\n");

    // 对KLMN加写锁
    printf("对KLMN加写锁");
    if (wlock(fd, 10, 4, 1) == -1)
        printf("失败：%m\n");
    else
    {
        printf("成功！\n");
        ulock(fd, 10, 4);
    }

    close(fd);
    return 0;
}
```

进程A：读锁 进程B：读锁　共享

进程A：写锁 进程B：读锁　互斥

进程A：读锁 进程B：写锁　互斥

进程A：写锁 进程B：写锁　互斥

7、复制文件描述符

```c
int dup(int oldfd);
功能：复制文件描述符
oldfd：已经打开了的文件描述符
返回值：系统返回一个新的文件描述述newfd，当前可用文件描述中的最小数值

int dup2(int oldfd, int newfd);
功能：按指定的newfd复制oldfd文件描述符，如果newfd已经被占用，则把它关闭重新复制。
返回值：成功返回0，失败返回-1。

int fcntl (int fd, int cmd, long arg);
功能：复制fd为不小于arg的文件描述符，若arg文件描述符已用，该函数会选择比arg大的最小未用值，而非如dup2函数那样关闭之。
cmd：取值为F_DUPFD

注意：当对oldfd和newfd进行读写时，访问的是同一个文件。
```

复制文件描述符的意义：

复制文件描述符就让一个内核对象对应多个文件描述符，它的意义在于实现文件的回调访问，可以实现很多神奇的操作，如命令重定向就是基于该功能实现的。

练习：

1、实现cat命令的功能

2、实现cat - 代替 cat >

3、实现cat + 代替 cat >>

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    if(2 != argc && 4 != argc)
    {
        printf("Use:./cat <file> [-|+ file]");
        return 0;
    }

    int fd = open(argv[1],O_RDONLY);
    if(0 > fd)
    {
        perror("open");
        return -1;
    }

    if(4 == argc)
    {
        int dest_fd = -1;
        if('-' == argv[2][0])
        {
            // 以清空方式打开文件 >
            dest_fd = open(argv[3],O_WRONLY|O_CREAT|O_TRUNC,0644);
            if(0 > dest_fd)
            {
                perror("open dest");
                return -1;
            }
        }
        else if('+' == argv[2][0])
        {
            // 以追加方式打开文件 >>
            dest_fd = open(argv[3],O_WRONLY|O_CREAT|O_APPEND,0644);
            if(0 > dest_fd)
            {
                perror("open dest");
                return -1;
            }
        }
        else
        {
            printf("Use:./cat <file> [-|+ file]");
```

```
            return 0;
        }

        // 把文件描述符与1复制
        dup2(dest_fd,1);
        close(dest_fd);
    }

    int ret = 0;
    char buf[4096];
    while(ret = read(fd,buf,sizeof(buf)))
    {
        write(1,buf,ret);
    }

    close(fd);
    return 0;
}
```

十四、文件管理

1、获取文件的属性

```
int stat(const char *path, struct stat *buf);
功能：根据文件的路径来获取文件的属性

int fstat(int fd, struct stat *buf);
功能：根据文件描述符来获取文件的属性

int lstat(const char *path, struct stat *buf);
功能：获取链文件的属性

struct stat {
    dev_t     st_dev;    文件的设备ID
    ino_t     st_ino;    文件的节点号
    mode_t    st_mode;   文件的类型和权限
    nlink_t   st_nlink;  硬链接数量
    uid_t     st_uid;    属主ID(文件的拥有者的用户ID)
    gid_t     st_gid;    属组ID(文件的拥有者的用户组ID)
    dev_t     st_rdev;   特殊设备ID
    off_t     st_size;   文件的总字节数
    blksize_t st_blksize; I/O块数
    blkcnt_t  st_blocks;  以512字节为一块，占多少块
    time_t    st_atime;    最后访问时间
    time_t    st_mtime;    最后的修改时间
    time_t    st_ctime;    最后状态修改时间
};

st_mode
    S_IFSOCK    套接字文件
    S_IFLNK     软链接文件
    S_IFREG     普通文件
    S_IFBLK     块设备文件
    S_IFDIR     目录
    S_IFCHR     字符设备文件
    S_IFIFO     管道文件
    S_ISUID     属主沾滞位标志(一般合并在执行权限上)
```

```
    S_ISGID     属组沾滞位标志
    S_ISVTX     其它沾滞位标志

    S_IRWXU     属主的读写执行权限
    S_IRUSR     属主的读权限
    S_IWUSR     属主的写权限
    S_IXUSR     属主的执行权限
    S_IRWXG     属组的读写执行权限
    S_IRGRP     属组的读权限
    S_IWGRP     属组的写权限
    S_IXGRP     属组的执行权限
    S_IRWXO     其它的读写执行权限
    S_IROTH     其它的读权限
    S_IWOTH     其它的写权限
    S_IXOTH     其它的执行权限

帮助判断文件类型的宏函数:
    S_ISREG(m)  普通文件
    S_ISDIR(m)  目录文件
    S_ISCHR(m)  字符设备文件
    S_ISBLK(m)  块设备文件
    S_ISFIFO(m) 管道文件
    S_ISLNK(m)  软链接文件
    S_ISSOCK(m) 套接字文件
```

```c
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>

const char *mtos(mode_t m)
{
    static char s[11];

    if (S_ISDIR(m))
        strcpy(s, "d");
    else if (S_ISLNK(m))
        strcpy(s, "l");
    else if (S_ISBLK(m))
        strcpy(s, "b");
    else if (S_ISCHR(m))
        strcpy(s, "c");
    else if (S_ISSOCK(m))
        strcpy(s, "s");
    else if (S_ISFIFO(m))
        strcpy(s, "p");
    else
        strcpy(s, "-");

    strcat(s, m & S_IRUSR ? "r" : "-");
    strcat(s, m & S_IWUSR ? "w" : "-");
    strcat(s, m & S_IXUSR ? "x" : "-");

    strcat(s, m & S_IRGRP ? "r" : "-");
    strcat(s, m & S_IWGRP ? "w" : "-");
    strcat(s, m & S_IXGRP ? "x" : "-");

```

```c
    strcat(s, m & S_IROTH ? "r" : "-");
    strcat(s, m & S_IWOTH ? "w" : "-");
    strcat(s, m & S_IXOTH ? "x" : "-");

    if (m & S_ISUID)
        s[3] = (s[3] == 'x' ? 's' : 'S');
    if (m & S_ISGID)
        s[6] = (s[6] == 'x' ? 's' : 'S');
    if (m & S_ISVTX)
        s[9] = (s[9] == 'x' ? 't' : 'T');
    return s;
}

const char *ttos(time_t t)
{
    static char s[20];

    struct tm *lt = localtime(&t);
    sprintf(s, "%04d-%02d-%02d %02d:%02d:%02d",
            lt->tm_year + 1900, lt->tm_mon + 1, lt->tm_mday,
            lt->tm_hour, lt->tm_min, lt->tm_sec);
    return s;
}

int main(int argc, char *argv[])
{
    if (argc < 2)
        goto usage;

    struct stat st;
    if (stat(argv[1], &st) == -1)
    {
        perror("stat");
        return -1;
    }

    printf("            设备ID：%u\n", st.st_dev);
    printf("          i节点号：%u\n", st.st_ino);
    printf("             模式：%s\n", mtos(st.st_mode));
    printf("         硬链接数：%u\n", st.st_nlink);
    printf("            属主ID：%u\n", st.st_uid);
    printf("            属组ID：%u\n", st.st_gid);
    printf("       特殊设备ID：%u\n", st.st_rdev);
    printf("         总字节数：%u\n", st.st_size);
    printf("      I/O块字节数：%u\n", st.st_blksize);
    printf("占用块(512字节)数：%u\n", st.st_blocks);
    printf("     最后访问时间：%s\n", ttos(st.st_atime));
    printf("     最后修改时间：%s\n", ttos(st.st_mtime));
    printf(" 最后状态改变时间：%s\n", ttos(st.st_ctime));
    return 0;
}
```

2、文件的权限

测试文件的权限：

```
#include <unistd.h>

int access (const char* pathname, int mode);
pathname：文件的路径
mode:
    R_OK/W_OK/X_OK 测试调用进程对该文件，是否可读/可写/可执行
    F_OK，测试该文件是否存在
返回值：成功返回0，失败返回-1。
```

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        fprintf(stderr, "用法：%s <文件>\n", argv[0]);
        return -1;
    }

    printf("文件%s", argv[1]);
    if (access(argv[1], F_OK) == -1)
        printf("不存在(%m)。\n");
    else
    {
        if (access(argv[1], R_OK) == -1)
            printf("不可读(%m)，");
        else
            printf("可读，");

        if (access(argv[1], W_OK) == -1)
            printf("不可写(%m)，");
        else
            printf("可写，");

        if (access(argv[1], X_OK) == -1)
            printf("不可执行(%m)。\n");
        else
            printf("可执行。\n");
    }

    return 0;
}
```

修改文件的权限：

```
#include <sys/stat.h>

int chmod (const char* path,mode_t mode);
int fchmod (int fd, mode_t mode);

mode为以下值的位或(直接写八进制整数形式亦可，如07654)
```

```
------------------------------
S_IRUSR          - 属主可读
S_IWUSR          - 属主可写
S_IXUSR          - 属主可执行
------------------------------
S_IRGRP          - 属组可读
S_IWGRP          - 属组可写
S_IXGRP          - 属组可执行
------------------------------
S_IROTH          - 其它可读
S_IWOTH          - 其它可写
S_IXOTH          - 其它可执行
```

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>

int main(void)
{
    int fd = open("chmod.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        perror("open");
        return -1;
    }

    // if (fchmod (fd, 0765) == -1) {
    if (-1 == fchmod(fd,S_ISUID | S_ISGID | S_ISVTX |
                    S_IRUSR | S_IWUSR |
                    S_IRGRP | S_IXGRP |
                    S_IROTH))
    {
        perror("fchmod");
        return -1;
    }

    close(fd);
    return 0;
}
```

3、文件的权限屏蔽码：

当使用creat、open创建文件时，无论权限设置为什么都会创建成功，但操作系统会根据权限屏蔽码对用户设置的权限进行过滤，文件最终的权限是系统过滤后的权限。

```
可以用umask命令查看/修改当前shell的文件权限屏蔽码：
# umask
0022

# umask 0033
# umask
0033

#include <sys/stat.h>
mode_t umask (mode_t cmask);
功能：为当前进程设置文件权限掩码，并返回旧的权限屏蔽码，此函数永远成功。
注意：chmod函数和命令都不受权限屏蔽码的影响。
```

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>

int main(void)
{
    // mode_t old = umask (0333);
    mode_t old = umask(
        S_IWUSR | S_IXUSR |
        S_IWGRP | S_IXGRP |
        S_IWOTH | S_IXOTH);

    // 设权限屏蔽码以后，此进程所创建的文件，都不会有屏蔽字所包含的权限。
    int fd = open("umask.txt", O_RDWR | O_CREAT | O_TRUNC, 0777);
    if (fd == -1)
    {
        perror("open");
        return -1;
    }

    close(fd);
    umask(old);

    return 0;
}
```

4、修改文件的大小

```c
#include <unistd.h>

int truncate (const char* path,off_t length);
int ftruncate (int   fd, off_t length);
// 成功返回0，失败返回-1，截短丢弃，加长添零
```

```c
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>

int main()
{
    const char *text = "Hello, World !";
```

```c
    size_t size = strlen(text) + 1;

    int fd = open("trunc.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        perror("open");
        return -1;
    }

    if (ftruncate(fd, size) == -1)
    {
        perror("ftruncate");
        return -1;
    }

    void *map = mmap(NULL, size, PROT_READ | PROT_WRITE,
                     MAP_SHARED /*MAP_PRIVATE*/, fd, 0);
    if (map == MAP_FAILED)
    {
        perror("mmap");
        return -1;
    }

    memcpy(map, text, size);
    printf("%s\n", map);

    munmap(map, size);
    close(fd);
    return 0;
}
```

5、链接文件

硬链接文件的创建与删除：

```c
#include <unistd.h>

int link (const char* path1,const char* path2);
功能：创建硬链接文件，对应ln命令的功能

int unlink (const char* path);
功能：删除文件的硬链接，只有当文件的硬链接数降为0时，文件才会真正被删除。

int remove (const char* pathname);
功能：目前没有发现与unlink有什么区别，remove底层调用的就是unlink函数。
注意：若该文件正在被某个进程打开，其内容直到该文件被关闭才会被真正删除。
```

软件链接文件的创建与读取：

```
#include <unistd.h>

int symlink (const char* oldpath, const char* newpath);
功能：创建软链接文件，目标文件可以不存在，对应的是ln -s命令的功能

ssize_t readlink (const char* restrict path,char* restrict buf,size_t
      bufsize);
功能：获取软链接文件本身的内容，open函数打开的是链接目标，不能打开软链接文件本身。

注意：硬链接只是一个文件名，即目录中的一个条目，它链接的是文件的内容，被链接的文件不存在时，
硬链接文件依然能正常使用，而软链接则是一个独立的文件，其内容是另一个文件的路径信息，被链接的
文件不存在时，软链接文件无法再被open函数打开，目录文件只能创建软链接。
```

十五、目录文件

1、工作目录

工作目录指的是当前所在的目录，它是相对路径的起点，操作文件时，如果只提供文件名而没有路径信息，则操作的就是工作目录下的文件，一般可执行文件的存储位置就是进程的默认工作目录。

```
#include <unistd.h>

int chdir (const char* path);
int fchdir (int fildes);
功能：修改工作目录，功能相当于cd命令

char* getcwd (char*  buf,size_t size);
功能：获取当前进程的工作目录，功能相当于pwd命令
```

2、创建、删除、读取目录

```
#include <sys/stat.h>

int mkdir (const char* path,mode);
功能：创建空白目录，目录必须有执行权限才可进入

#include <unistd.h>
int rmdir ( const char* path // 目录路径);
功能：删除目录，但只能删除空目录

#include <sys/types.h>
#include <dirent.h>

DIR* opendir (const char* name);
DIR* fdopendir (int fd);
功能：打开目录文件
返回值：成功返回目录流指针，失败返回NULL。

struct dirent* readdir (DIR* dirp);
功能：从目录流中读取一个条目
返回值：成功返回下一个目录条目结构体的指针，到达目录尾或失败返回NULL。

struct dirent {
    ino_t          d_ino;       // i节点号
```

```
    off_t           d_off;       // 下一条目的偏移量，注意是磁盘偏移量，而非内存地址偏移
    unsigned short d_reclen;    // 记录长度
    unsigned char  d_type;      // 文件类型
    char           d_name[256]; // 文件名
};
d_type取值：
    DT_DIR     - 目录
    DT_REG     - 普通文件
    DT_LNK     - 软链接
    DT_BLK     - 块设备
    DT_CHR     - 字符设备
    DT_SOCK    - Unix域套接字
    DT_FIFO    - 有名管道
    DT_UNKNOWN - 未知

void rewinddir(DIR *dirp);
功能：复位目录流

long telldir(DIR *dirp);
功能：获取目录流当前位置

void seekdir(DIR *dirp, long loc);
功能：设置目录流当前位置

int closedir (DIR* dirp);
功能：关闭目录文件
成功返回0，失败返回-1。
```

```c
#include <stdio.h>
#include <dirent.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        fprintf(stderr, "用法：%s <目录>\n", argv[0]);
        return -1;
    }

    DIR *dp = opendir(argv[1]);
    if (!dp)
    {
        perror("opendir");
        return -1;
    }

    for (struct dirent *de = readdir(dp); de; de = readdir(dp))
    {
        printf("[%010d %010d] ", de->d_ino, de->d_off);

        switch (de->d_type)
        {
        case DT_DIR:
            printf("        目录：");
            break;
```

```c
        case DT_REG:
            printf("    普通文件：");
            break;
        case DT_LNK:
            printf("      软链接：");
            break;
        case DT_BLK:
            printf("      块设备：");
            break;
        case DT_CHR:
            printf("    字符设备：");
            break;
        case DT_SOCK:
            printf("Unix域套接字：");
            break;
        case DT_FIFO:
            printf("    有名管道：");
            break;
        default:
            printf("        未知：");
            break;
        }

        printf("%s\n", de->d_name);
    }

    closedir(dp);
    return 0;
}
```

十六、信号处理

1、什么是信号

信号是UNIX、类UNIX以及其他POSIX兼容的操作系统中进程间通讯的一种有限制的方式。它是一种异步的通知机制，用来提醒进程一个事件已经发生。

当一个信号发送给一个进程，操作系统中断了进程正常的控制流程，此时，任何非原子操作都将被中断。如果进程定义了信号的处理函数，那么它将被执行，否则就执行默认动作。

```
# 执行kill -l命令可以显示出当前系统可能产生的所有信号
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL      10) SIGUSR1
11) SIGSEGV      12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT    17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN      22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM    27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS       34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4   39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9   44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14  49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11  54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
```

```
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

2、不可靠信号与可靠信号

不可靠信号：

- 那些建立在早期机制上的信号被称为“不可靠信号”，小于SIGRTMIN(34)的信号都是不可靠信号。
- 非实时信号，不支持排队，可能会丢失，同一个信号产生多次，进程可能只收到一次该信号。
- 进程每次处理完这些信号后，对相应信号的响应被自动恢复为默认动作，除非显示地通过signal函数重新设置一次信号处理程序，但现在已经升级了，只有极个别人系统还仍然保持这样。

可靠信号：

- 位于[SIGRTMIN(34),SIGRTMAX(64)]区间的信号都是可靠信号。
- 实时信号，支持排队，信号每产生一次就处理一次，因此可靠信号不会丢失。
- 无论可靠信号还是不可靠信号，都可以通过sigqueue/sigaction函数发送/安装，以获得比其早期版本kill/signal函数更可靠的使用效果。

3、信号来源和默认动作

信号	进程的默认动作	产生条件
SIGHUP(1)	终止	终端挂起或者控制进程终止产生此信号
SIGINT(2)	终止	用户按中断键(Ctrl+C)，产生此信号，并送至前台进程组的所有进程
SIGQUIT(3)	终止+core	用户按退出键(Ctrl+\)，产生此信号，并送至前台进程组的所有进程
SIGILL(4)	终止+core	进程执行了一条非法硬件指令，通常是因为可执行文件本身出现错误
SIGTRAP(5)	终止+core	可以实现自定义的硬件故障，常用于gdb调试时产生，如函数没返回值
SIGABRT(6)	终止+core	调用abort函数，产生此信号
SIGBUS(7)	终止+core	总线错误，一般产生的原因是错误的内存访问
SIGFPE(8)	终止+core	表示一个算术运算异常，例如除以0、浮点溢出等
SIGKILL(9)	终止	必须死信号，不能被捕获或忽略，常用于杀死进程
SIGUSR1(10)	终止	用户自定义信号，用于进程间通信
SIGSEGV(11)	终止+core	段错误信号，试图访问未分配的内存，或向没有写权限的内存写入数据
SIGUSR2(12)	终止	用户自定义信号，用于进程间通信
SIGPIPE(13)	终止	写管道时读进程已终止，或SOCK_STREAM类型套接字时连接已断开，均产生此信号
SIGALRM(14)	终止	alarm函数设置的计时器到期，setitimer函数设置的时间到期，产生此信号
SIGTERM(15)	终止	程序结束信号, 与SIGKILL不同的是该信号可以被阻塞和处理，通常用来要求程序自己正常退出
SIGSTKFLT(16)	终止	表示数学协处理器发生栈故障
SIGCHLD(17)	忽略	在一个进程终止或停止时，将此信号发送给其父进程
SIGCONT(18)	继续/忽略	向处于停止状态的进程发送此信号，令其继续运行
SIGSTOP(19)	停止进程	不能被捕获或忽略，停止一个进程
SIGTSTP(20)	停止进程	用户按停止键(Ctrl+Z)，产生此信号，并送至前台进程组的所有进程
SIGTTIN(21)	停止	后台进程组中的进程试图从控制终端读数据，产生此信号
SIGTTOU(22)	停止	后台进程组中的进程试图往控制终端写数据，产生此信号

信号	进程的默认动作	产生条件
SIGURG(23)	忽略	从网络上接收到紧急带外数据，产生此信号
SIGXCPU(24)	终止+core	当进程的运行时间超过了其CPU时间限制，产生此信号
SIGXFSZ(25)	终止+core	当进程企图扩大文件以至于超过文件大小资源限制，产生此信号
SIGVTALRM(26)	终止	以setitimer函数设置的虚拟间隔时间到期，产生此信号
SIGPROF(27)	终止	以setitimer函数设置的虚拟梗概统计间隔时间到期，产生此信号
SIGWINCH(28)	忽略	终端窗口大小改变，产生此信号
SIGIO(29)	终止	表示出现一个异步I/O事件
SIGPWR(30)	终止	电源失效，关机时产生此信号
SIGSYS(31)	终止+core	指示一个无效的系统调用

4、捕获信号

```
#include <signal.h>

typedef void (*sighandler_t) (int);
sighandler_t signal(int signum,sighandler_t handler);
功能：注册信号处理函数
signum  - 信号码，也可使用系统预定义的常量宏，如SIGINT等。
handler - 信号处理函数指针或以下常量：
    SIG_IGN: 忽略该信号
    SIG_DFL: 默认处理
返回值：成功返回原来的信号处理函数指针或SIG_IGN/SIG_DFL常量，失败返回SIG_ERR。
```

在某些UNIX系统上，通过signal函数注册的信号处理函数只一次有效，即内核每次调用信号处理函数前，会将对该信号的处理自动恢复为默认方式，为了获得持久有效的信号处理，可以在信号处理函数中再次调用signal函数，重新注册一次。

```
void sigint (int signum)
{
    ...
    signal (SIGINT, sigint);
}

int main (void)
{
    ...
    signal (SIGINT, sigint);
    ...
}
```

SIGKILL/SIGSTOP信号不能被忽略，也不能被捕获，普通用户只能给自己的进程发送信号，root用户可以给任何进程发送信号。

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void sigint (int signum)
{
    printf ("%u进程：收到SIGINT信号！\n", getpid ());
}

void sigkill (int signum)
{
    printf ("收到SIGKILL信号！\n");
}

int main ()
{
    /*
    // 忽略信号
    if (signal (SIGINT, SIG_IGN) == SIG_ERR)
    {
        perror ("signal");
        return -1;
    }
    if (signal (SIGKILL, SIG_IGN) == SIG_ERR)
    {
        perror ("signal");
        return -1;
    }

    if (signal (SIGKILL, sigkill) == SIG_ERR)
    {
        perror ("signal");
        return -1;
    }
    */
    if (signal (SIGINT, sigint) == SIG_ERR)
    {
        perror ("signal");
        return -1;
    }

    printf ("%u进程：我在运行，按<Ctrl+C>没用，按<Ctrl+\\>终止...\n",getpid ());
    for (;;);

    return 0;
}
```

5、发送信号的方式

通过键盘发送信号：

给当前终端下的进程发送信号：

Ctrl+C 终端中断SIGINT(2)
Ctrl+\ 终端退出SIGQUIT(3)
Ctrl+Z 终端暂停SIGTSTP(20)

执行命令发送信号：

```
kill <-信号> 进程号
killall <-信号> 进程名
注意：如果省略了信号，则默认发送SIGTERM(15)信号
```

特殊事件发生时会发送信号：

当进程执行某些非法操作时，被操作系统发现，操作系统就会向进程发送相关信号，如果：段错误SIGSEGV(11)、浮点数例外SIGFPE(8)、总线错误SIGBUS(7)。

调用函数发送信号：

```
#include <signal.h>

int kill (pid_t pid, int sig);
功能：向指定的进程发送sig信号
pid > 0 - 向pid进程发送sig信号。
pid = 0 - 向同进程组的所有进程发送信号。
pid =-1 - 向所有进程发送信号，前提是调用进程有向其发送信号的权限。
pid <-1 - 向绝对值等于pid的进程组发信号。


int raise (int sig);
功能：向调用进程自身发送sig信号。

void abort(void);
功能：向调用进程向自身发送SIGABRT信号

unsigned int alarm (unsigned int seconds);
功能：使内核在seconds秒之后，向调用进程发送SIGALRM(14)闹钟信号，SIGALRM信号的默认处理是终止进程。
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main (void)
{
    pid_t pid;
    printf("请输入进程号：");
    scanf("%u",&pid);

    printf ("进程：我是进程%u，我要向%u进程发送SIGINT信号...\n",
        getpid(), pid);

    if (kill(pid, SIGINT) == -1)
```

```
    {
        perror ("kill");
        return -1;
    }
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void sigint (int signum)
{
    printf ("%u进程：收到SIGINT信号，即将退出。\n", getpid ());
    exit(0);
}

int main ()
{
    if (signal (SIGINT, sigint) == SIG_ERR)
    {
        perror ("signal");
        return -1;
    }

    printf ("%u进程：向自己发送SIGINT信号...\n", getpid ());
    if (raise (SIGINT) == -1)
    {
        perror ("kill");
        return -1;
    }

    for(;;);
    return 0;
}
```

```
#include <stdio.h>
#include <time.h>
#include <unistd.h>
#include <signal.h>

void sigalrm (int signum)
{
    time_t t = time (NULL);
    struct tm* lt = localtime (&t);

    printf ("\r%02d:%02d:%02d", lt->tm_hour, lt->tm_min, lt->tm_sec);
    alarm (1);
}

void sigint(int signum)
{
    printf("取消闹钟！\n");
    // seconds取0表示取消之前设过且尚未超时的定时
    alarm(0);
```

```c
}

int main (void)
{
    signal (SIGALRM, sigalrm);
    signal(SIGINT,sigint);

    alarm(10);
    sleep(3);

    // 若之前已设过定时且尚未超时，则调用该函数会重新设置定时，并返回之前定时的剩余时间
    printf("%u\n",alarm(5));

    for (;;);
    return 0;
}
```

6、暂停与休眠

```
#include <unistd.h>
int pause (void);
功能：使调用进程暂停执行，进入睡眠状态，直到有信号终止该进程或被有信号被捕获,相当于没有时间限制的sleep函数。
```

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void sigint (int signum)
{
    printf ("%u进程：收到SIGINT信号！\n", getpid ());
    // 信号处理函数并从中返回以后，pause函数才会返回
    for(int i=3; i>0; i--)
    {
        printf("\r%d",i);
        fflush(stdout);
        sleep(1);
    }
}

int main ()
{
    signal (SIGINT, sigint);
    printf ("%u进程：我是进程，大睡无期...\n", getpid ());
    // 该函数要么不返回，要么被信号中断返回-1
    printf("%d\n",pause ());
    printf ("%u进程：我被唤醒了，即将退出。\n", getpid ());
    sleep(1);
}
```

```
#include <unistd.h>
unsigned int sleep (unsigned int seconds);
功能：使调用进程睡眠seconds秒，除非有信号终止该进程或被捕获,相当于有时间限制的pause函数。

int usleep (useconds_t usec);
功能：使调用进程睡眠usec微秒，除非有信号终止该进程或被捕获。
```

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void sigint (int signum)
{
    printf ("%u进程：收到SIGINT信号！\n", getpid ());
    // 信号处理函数并从中返回以后，sleep函数才会返回
    for(int i=3; i>0; i--)
    {
        printf("\r%d",i);
        fflush(stdout);
        sleep(1);
    }
}

int main ()
{
    signal (SIGINT, sigint);
    printf ("%u进程：我是进程，我要睡10秒...\n", getpid());
    // 该函数要么返回0(睡够)，要么被信号中断返回剩余秒数
    printf("%u进程：我被唤醒了,实际睡了%d，即将退出。\n", getpid (),10-sleep(10));
}
```

7、带参的信号发送与捕获

```
#include <signal.h>

int sigaction (int signum, const struct sigaction* act,struct sigaction* oldact);
功能：设置信号的处理方案
signum：信号码
act：信号处理方式
oldact：原信号处理方式,可为NULL

struct sigaction {
    void (*sa_handler)(int); // 信号处理函数指针1
    void (*sa_sigaction)(int,siginfo_t*,void*); // 信号处理函数指针2
    sigset_t sa_mask; // 信号掩码
    int sa_flags; // 信号处理标志
    void (*sa_restorer)(void); // 保留，NULL
};

sa_flags可为以下值的位或：
    SA_ONESHOT/SA_RESETHAND 执行完一次信号处理后，即对此信号的处理恢复为默认，这也是老版本signal函数的缺省行为。
    SA_NODEFER/SA_NOMASK 在信号处理函数的执行过程中，不屏蔽这个正在被处理的信号。
    SA_NOCLDSTOP 若signum参数取SIGCHLD，则当子进程暂停时，不通知父进程。
    SA_RESTART 系统调用一旦被signum参数所表示的信号中断，会自行重启。
```

```
    SA_SIGINFO 使用信号处理函数指针2，通过该函数的第二个参数，提供更多信息。

typedef struct siginfo {
    pid_t si_pid;   // 发送信号的PID
    sigval_t si_value; // 信号附加值(需要配合sigqueue函数)
    ...
}siginfo_t;

typedef union sigval {
    int sival_int;
    void* sival_ptr;
}sigval_t;

int sigqueue (pid_t pid, int sig,const union sigval value);
功能：向pid进程发送sig信号，附加value值(整数或指针)。
```

在默认情况下，在信号处理函数的执行过程中，会自动屏蔽这个正在被处理的信号，而对于其它信号则不会屏蔽，通过sigaction::sa_mask成员可以人为指定。

在信号处理函数的执行过程中，需要加入进程信号掩码中的信号，并在信号处理函数执行完之后，自动解除对这些信号的屏蔽。

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void sigint1 (int signum)
{
    printf ("收到SIGINT信号！睡眠中...\n");
    sleep (3);
    printf ("睡醒了。\n");
}

void sigint2 (int signum, siginfo_t* si, void* pv)
{
    printf ("收到发自%u进程的SIGINT信号！\n", si -> si_pid);
    printf("附加的数据是：%d\n",si->si_value.sival_int);
    //printf("%s\n",si->si_value.sival_ptr);
}

int main ()
{
    printf("我是进程%u\n",getpid());
    struct sigaction act = {};
    act.sa_handler = sigint1;
    act.sa_sigaction = sigint2;
    act.sa_flags = SA_SIGINFO;

    sigaction (SIGINT, &act, NULL);
    for (;;) pause ();

    return 0;
}
```

```
#include <sys/wait.h>

int main (void)
```

```
{
    pid_t pid;
    printf("请输入进程号：");
    scanf("%u",&pid);

    sigval_t sv;
    sv.sival_int = 6666;
    // sv.sival_ptr = "hehehe";

    if (sigqueue (pid, SIGINT, sv) == -1)
    {
        perror ("sigqueue");
        return -1;
    }
    return 0;
}
```

8、计时器

系统为进程维护三个计时器：

- 真实计时器：程序运行的实际时间。
- 虚拟计时器：程序运行在用户态所消耗的时间。
- 实用计时器：程序运行在用户态和内核态所消耗的时间之和。

  实际时间(真实计时器) = 用户时间(虚拟计时器) + 内核时间 + 睡眠时间，用指定的初始间隔和重复间隔为进程设定好计时器后，该计时器就会定时地向进程发送时钟信号。

获取/设置计时器：

```
#include <sys/time.h>

int getitimer (int which,struct itimerval* curr_value);
功能：获取当前进程的计时器设置。

int setitimer (int which,const struct itimerval* new_value,struct itimerval* old_value);
功能：设置计时器。
which       - 指定哪个计时器，取值：
    ITIMER_REAL: 真实计时器；
    ITIMER_VIRTUAL: 虚拟计时器；
    ITIMER_PROF: 实用计时器。
new_value   - 新的设置。
old_value   - 旧的设置(可为NULL)。

struct itimerval {
    struct timeval it_interval;
    // 重复间隔(每两个时钟信号的时间间隔)，取0将使计时器在发送第一个信号后停止
    struct timeval it_value;
    // 初始间隔(从调用setitimer函数到第一次发送时钟信号的时间间隔)，取0将立即停止计时器
};

struct timeval {
    long tv_sec;  // 秒数
    long tv_usec; // 微秒数，不能超过1000000
};
```

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/time.h>

void sigalrm(int signum)
{
    printf("闹钟响了...\n");
}

int main(int argc,const char* argv[])
{
    signal(SIGALRM,sigalrm);
    struct itimerval timer = {{1,500000},{3,500000}};
    setitimer(ITIMER_REAL,&timer,NULL);

    for(;;) pause();

    return 0;
}
```

十七、信号集与信号屏蔽

1、什么是信号集

一种专门存储信号的数据类型sigset_t，有128个字节，每个字节代表一个信号。

2、操作信号集的相关函数：

```c
#include <signal.h>

int sigfillset (sigset_t* set);
功能：将信号集set中的全部信号位置1

int sigemptyset (sigset_t* set);
功能：将信号集set中的全部信号位清0

int sigaddset (sigset_t* set, int signum);
功能：将信号集set中与signum对应的位置1

int sigdelset (sigset_t* set, int signum);
功能：将信号集set中与signum对应的位清0

int sigismember (const sigset_t* set, int signum);
功能：若信号集set中与signum对应的位为1，则返回1，否则返回0。
```

```c
#include <stdio.h>
#include <signal.h>

 void showsig(sigset_t* setp)
 {
    for(int i=0; i<128; i++)
    {
        printf("sig=%d set=%hhu|",i,((unsigned char*)setp)[i]);
        if(0 == (i+1)%4)
            printf("\n");
```

```c
    }
    printf("----------------------------\n");
 }

int main ()
{
    sigset_t set;
    unsigned char* ptr = (void*)&set;
    printf ("%u\n", sizeof (set));

    // 填满信号集
    sigfillset(&set);
    // 清空信号集
    sigemptyset(&set);
    // 向信号集中添加信号
    sigaddset(&set, SIGINT);
    sigaddset(&set, SIGQUIT);
    // 从信号集中删除信号
    sigdelset(&set, SIGQUIT);
    showsig(&set);


    // 测试信号集中是否有某个信号
    if (sigismember (&set, SIGINT))
        printf ("There is SIGINT.\n");
    else
        printf ("There isn't SIGINT.\n");

    if (sigismember (&set, SIGQUIT))
        printf ("There is SIGQUIT.\n");
    else
        printf ("There isn't SIGQUIT.\n");

    return 0;
}
```

3、信号的递送与未决

1、当信号产生时，系统内核会在其所维护的进程表中，为特定的进程设置一个与该信号相对应的标志位，这个过程称为递送(delivery)。

2、信号从产生到完成递送之间存在一定的时间间隔，处于这段时间间隔中的信号状态，称为未决(pending)。

4、信号屏蔽

1、每个进程都有一个信号掩码(signal mask，它实际上是一个信号集)，其中包括了所有需要被屏蔽的信号。

2、当进程执行诸如更新数据库等敏感任务时，可能不希望被某些信号中断，这时可以暂时屏蔽(注意不是忽略)这些信号，使其滞留在未决状态，待任务完成以后，再回过头来处理这些信号。

3、在信号处理函数的执行过程中，这个正在被处理的信号总是处于信号掩码中。

4、可以通过sigprocmask函数，检测和修改调用进程的信号掩码，也可以通过sigpending函数，获取调用进程当前处于未决状态的信号集。

4、可靠与不可靠信号屏蔽的区别

1、对于不可靠信号，通过sigprocmask函数设置信号掩码以后，相同的被屏蔽信号只会屏蔽第一个，并在恢复信号掩码后被递送，其余的则直接忽略掉。

2、而对于可靠信号，则会在信号屏蔽时按其产生的先后顺序排队，一旦恢复信号掩码，这些信号会依次被信号处理函数处理。

```
#include <signal.h>

int sigprocmask (int how, const sigset_t* set,
    sigset_t* oldset);
功能：修改当前进程的信号掩码
how  - 修改信号掩码的方式，可取以下值：
    SIG_BLOCK: 新掩码是当前掩码和set的并集(将set加入信号掩码)；
    SIG_UNBLOCK: 新掩码是当前掩码和set补集的交集(从信号掩码中删除set)；
    SIG_SETMASK: 新掩码即set(将信号掩码设为set)。
set  - 为NULL则忽略。
oldset - 备份以前的信号掩码，NULL则不备份。

int sigpending (sigset_t* set);
功能：获取当前进程处于未决状态的信号集。
```

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void load_data(void)
{
    // 定义信号集
    sigset_t set,oldset;
    // 清空信号集
    sigemptyset(&set);
    // 往信号集中添加信号
    sigaddset(&set,SIGINT);
    sigaddset(&set,34);
    // 屏蔽信号并备份旧的信号屏蔽码
    sigprocmask(SIG_SETMASK,&set,&oldset);
    for(int i=0; i<10; i++)
    {
        printf("正在加载第%d条数据\n",i+1);
        sleep(1);
    }

    // 取消屏蔽
    sigprocmask(SIG_SETMASK,&oldset,NULL);
}

void sig34(int signum)
{
    printf("当前进程收到了34信号...\n");
}

void sigint(int signum)
{
    printf("用户按了Ctrl+C...\n");
}
```

```c

int main(int argc,const char* argv[])
{
    printf("我是进程%u\n",getpid());
    signal(SIGINT,sigint);
    signal(34,sig34);
    load_data();
    return 0;
}
```

十八、进程介绍

1、进程与程序

程序是存储在磁盘上的可执行文件， 里面包含着可执行的机器指令和数据的静态实体，进程是处于活动状态计算机程序，也就是说进程就是运行中的程序，但一个运行着的程序，可能由多个进程组成，一般情况下一个正在运行着的程序只有一个默认进程，但可以通过系统调用创建若干个子进程。

2、查看进程

简单形式：

ps 命令以简略方式显示当前用户所有控制终端的进程信息。

复杂形式：

ps axuw 命令以列表形式显示进程详细信息
a - 所有用户所有控制终端的进程
x - 包括无控制终端的进程
u - 以详尽方式显示
w - 以更大列宽显示

3、进程信息列表

- USER/UID: 进程属主。
- PID: 进程ID。
- %CPU/C: CPU使用率。
- %MEM: 内存使用率。
- VSZ: 占用虚拟内存大小(KB)。
- RSS: 占用物理内存大小(KB)。
- TTY: 终端次设备号，“?”表示无控制终端，如后台进程。
- STAT/S: 进程状态。可取如下值：
    - O - 就绪，等待被调度。
    - R - 运行，Linux下没有O状态，就绪状态也用R表示。
    - S - 可唤醒睡眠。系统中断，获得资源，收到信号，都可被唤醒，转入运行状态。
    - D - 不可唤醒睡眠。只能被wake_up系统调用唤醒。
    - T - 暂停。收到SIGSTOP信号转入暂停状态， 收到SIGCONT信号转入运行状态。
    - W - 等待内存分页(2.6内核以后被废弃)。
    - X - 死亡。不可见。
    - Z - 僵尸。已停止运行，但其父进程尚未回收相关资源。
    - < - 高优先级。
    - N - 低优先级。
    - L - 有被锁到内存中的分页。实时进程和定制IO。

    - s - 会话首进程。
    - l - 多线程化的进程。
    - /+ - 在前台进程组中。
- START/STIME: 进程开始时间。
- TIME: 进程运行时间。
- COMMAND/CMD: 进程指令。
- F: 进程标志，可由下列值取和：
    - 1 - 通过fork产生但是没有exec。
    - 4 - 拥有超级用户特权。
- PPID: 父进程ID。
- NI: 进程nice值，-20到19，可通过系统调用或命令修改。
- PRI: 进程优先级。
  静态优先级 = 80 + nice，60到99，值越小优先级越高。
  内核在静态优先级的基础上，根据进程的交互性计算得到实际(动态)优先级，以体现对IO消耗型进程的奖励，和对处理器消耗型进程的惩罚。
- ADDR: 内核进程的内存地址。普通进程显示“-”。
- SZ: 占用虚拟内存页数。
- WCHAN: 进程正在等待的内核函数或事件。
- PSR: 进程被绑定到哪个处理器。

4、父进程与子进程

1、一个进程可以创建另一个进程，创建者叫做父进程，被创建都叫子进程。

2、父进程启动子进程后，子进程在操作系统的调度下与其父进程同时运行。

5、孤儿进程与僵尸进程

1、子进程先于父进程结束，子进程向父进程发送SIGCHLD(17)信号，父进程回收子进程的相关资源。

2、父进程先于子进程结束，子进程成为孤儿进程，同时被孤儿院进程收养，即成为孤儿院进程的子进程，早期的孤儿院是init，现在的孤儿院是upstart。

3、子进程先于父进程结束，但父进程没有回收子进程的相关资源，该子进程即成为僵尸进程。

6、进程标识符

1、每个进程都有一个以非负整数表示的唯一标识，即进程ID，简称PID。

2、进程ID在任何时刻都是唯一的，但可以重用，当一个进程结束时，其进程ID就可以被分配其它新创建的进程。

3、延迟重用，当进程结束后，它的进程ID并不会被操作系统立即重新分配出去，而隔一段时间再重新分配出去。

```
#include <stdio.h>
#include <unistd.h>

pid_t getpid(void);
功能：获取当进程的ID
pid_t getppid(void);
功能：获取当前进程的父进程ID

int main ()
{
    printf ("我是进程：%u，我爸爸是：%u\n", getpid(),getppid());
    return 0;
}
```

十九、fork创建进程

```
#include <unistd.h>
pid_t fork(void);
功能：创建一个子进程
返回值：失败返回-1，成功返回两次
父进程：返回子进程的pid
子进程：返回0

注意：总进程数或实际用户ID所拥有的进程数，超过系统限制，该函数将失败。
```

```
#include <stdio.h>
#include <unistd.h>

int main ()
{
    pid_t pid = fork ();
    printf("pid=%u\n",pid);
}
```

1、父子进程谁先运行：

函数调用后父子进程各自继续运行，其先后顺序不确定，某些实现可以保证子进程先返回执行。

```
#include <stdio.h>
#include <unistd.h>

int main ()
{
    printf ("%u进程：我要调用fork()了...\n", getpid ());
    pid_t pid = fork ();
    if (pid == -1)
    {
        perror ("fork");
        return -1;
    }

    // 用返回值的不同，可以分别为父子进程编写不同的处理分支。
    if (pid == 0)
        printf ("%u进程：我是%u进程的子进程。\n", getpid (),getppid ());
    else
```

```
        printf ("%u进程：我是%u进程的父进程。\n", getpid (), pid);

    sleep (1);
    return 0;
}
```

2、子进程是父进程的副本：

使用fork创建的子进程获得父进程data段、bss段、heap段、stack段、I/O流缓冲区的拷贝。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global = 100;
int main ()
{
    printf("I/O流缓冲区演示，");

    int local = 200;
    char* heap = (char*)malloc (256 * sizeof (char));
    sprintf (heap, "ABC");

    printf ("父进程：%d %d %s\n", global, local, heap);

    pid_t pid = fork ();
    if (pid == -1)
    {
        perror ("fork");
        return -1;
    }

    if (pid == 0)
    {
        global++;
        local++;
        sprintf (heap, "XYZ");
        printf ("子进程：%d %d %s\n", global, local, heap);
        free (heap);
    }
    else
    {
        sleep (1);
        printf ("父进程：%d %d %s\n", global, local, heap);
        free (heap);
    }

    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>

int main ()
{
    printf ("I/O流缓冲区演示：");
```

```c
    getchar();


    pid_t pid = fork ();
    if (pid == -1)
    {
        perror ("fork");
        return -1;
    }

    if (pid == 0)
    {
        int a, b, c;
        scanf ("%d%d%d", &a, &b, &c);
        printf ("子进程：%d %d %d\n", a, b, c);
    }
    else
    {
        int a, b, c;
        scanf ("%d%d%d", &a, &b, &c);
        printf ("父进程：%d %d %d\n", a, b, c);
    }

    return 0;
}
// I/O流缓冲区演示：<字符>1 2 3
```

3、共享代码段、文件表：

fork之前的代码只有父进程执行，fork之后的代码父子进程都有机会执行，受代码逻辑的控制而进入不同分支。

函数调用后，父进程的文件描述符表(进程级)也会被复制到子进程中，二者共享同一个文件表(内核级)。

```c
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

void write_str(int fd,const char* str)
{
    int ret = write(fd,str,strlen(str));
    printf("%u write %d 字节的数据%m\n",getpid(),ret);
}

int main ()
{
    int fd = open ("test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (0 > fd)
    {
        perror ("open");
        return -1;
    }

    write_str(fd,"Hello, World !");

```

```
    pid_t pid = fork ();
    if (pid == 0)
        write_str(fd,"Linux");
    else
        write_str(fd,"UNIX");

    close (fd);
    return 0;
}
```

4、子进程会继承父进程的信号处理方式：

通过fork创建的子进程，之所能继承父进程的信号处理方式，是因为子进程与父进程共享代码段，否则是无法继承的。

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigint(int signum)
{
    printf("我是进程%u,我收到了%d信号\n",getpid(),signum);
}

int main(int argc,const char* argv[])
{
    //signal(SIGINT,sigint);
    signal(SIGINT,SIG_IGN);

    pid_t pid = fork();
    if(0 == pid)
    {
        printf("我是子进程%u,我爸爸是%u\n",getpid(),getppid());
        for(;;) pause();
    }
    else
    {
        printf("我是父进程%u,我儿子是%u\n",getpid(),pid);
        for(;;) pause();
    }
    return 0;
}
```

练习：实现出孤儿进程与僵尸进程，然后使用ps aux命令查看关确认。

```
// 孤儿进程
#include <stdio.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    pid_t pid = fork();
    if(0 == pid)
    {
        for(;;)
        {
```

```
            sleep(1);
            printf("我是进程%u,我爸爸是%u\n",getpid(),getppid());
        }
    }
    else
    {
        sleep(1);
        printf("我是进程%u,我儿子是%u\n",getpid(),pid);
        return 0;
    }
}

// 僵局进程
#include <stdio.h>
#include <unistd.h>

int main(int argc,const char* argv[])
{
    pid_t pid = fork();
    if(0 == pid)
    {
        printf("我是进程%u,我爸爸是%u\n",getpid(),getppid());
        return 0;
    }
    else
    {
        for(;;) pause();
    }
}
```

二十、vfork和exec创建进程

```
pid_t vfork(void);
功能：创建子进程，返回值与fork相同
```

1、vfork函数的特点

1、当调用vfork函数后，父进程会进入阻塞状态，子进程先返回。

2、子进程先临时使用父进程的相关资源，然后继续调用exec系列函数加载一个可执行文件，启动另一个进程替换自己。

3、等子进程调用完exec系列函数后，子进程才真正创建完毕后，父进行才会解除阻塞返回。

4、如果子进程不调用exec系列函数，会有两种情况：

情况1：子进程一直不结果，则父进程就一直陷入阻塞状态，无法返回。

情况1：子进程直接结束并释放相关资源，但此时子进程使用的是父进程的相关资源，然后父进程返回，此时会就产生段错误，因为它的相关资源已经被子进程释放了。

5、vfork不能单独使用创建子进程，必须与exec系列函数配合使用。

2、vfork与fork的区别

1、vfork调用之后，子进程先返回，而fork调用后，父子进程不一定谁先返回。

2、vfork不会复制父进程的相关资源，而是加载其它的可执行文件进行替换。

3、exec系列函数

```
这一系列函数的功能就是配合vfork创建子进程。
int execl(const char *path, const char *arg, ...);
path：程序的路径
arg：命令行参数

int execlp(const char *file, const char *arg, ...);
file：只需要程序的名字，系统会去PATH环境变量的路径中来查找这个程序。
arg：命令行参数

int execle(const char *path, const char *arg,..., char * const envp[]);
envp：环境变量表，创建子进程时传递父进程的环境变量表。

int execv(const char *path, char *const argv[]);
argv：命令参数以指针数组的形式提供，以NULL作为结尾

int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[],char *const envp[]);
```

4、system的实现原理

```
#include <stdlib.h>
int system (const char* command);
功能：
    执行command，成功返回command对应进程的终止状态，失败返回-1。
    若command取NULL，返回非零表示shell可用，返回0表示shell不可用。
```

该函数的实现，调用了vfork、exec和waitpid等函数，其返回值：

- 如果调用vfork或waitpid函数出错，则返回-1。
- 如果调用exec函数出错，则在子进程中执行exit(127)。
- 如果都成功，则返回command对应进程的终止状态(由waitpid的status输出参数获得)。

使用system函数而不用vfork+exec的好处是，system函数针对各种错误和信号都做了必要的处理。

二一、进程的结束

1、进程的正常结束

1、main函数中执行return结束进程

```
int main (...)
{
    ...
    return x;
}

等价于:
int main (...)
{
    ...
    exit (x);
}
```

2、调用标准C的exit函数结束进程

```
#include <stdlib.h>
void exit (int status);
```

1、该函数一旦调用就不会返回，其父进程调用wait/waitpid函数可获得status的低8位数据。

2、进程退出之前先调用所有事先通过atexit/on_exit函数注册的函数，冲刷并关闭所有仍处于打开状态的标准I/O流，删除所有通过tmpfile函数创建的文件。

3、可用EXIT_SUCCESS/EXIT_FAILURE常量宏作参数，表示进程是否正常结束。

4、该函数的底层调用了_exit/_Exit函数。

```
#include <stdlib.h>

int atexit (void (*function) (void));
function - 函数指针，指向进程退出前需要被调用的函数,该函数既没有返回值也没有参数。

int on_exit (void (*function) (int, void*), void* arg);
function - 函数指针，指向进程退出前需要被调用的函数,该函数没有返回值但有两个参数:
           第一参数来自exit函数的status参数，第二个参数来自on_exit函数的arg参数。
arg      - 任意指针，将作为第二个参数被传递给function所指向的函数。
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void atexit_fp(void)
{
    printf("我要结束了...!\n");
}

void on_exit_fp(int status,void* ptr)
{
    printf("我要结束了...,status=%d,%s\n",status,(char*)ptr);
}

int main(int argc,const char* argv[])
{
    // 谁先登记的谁后执行
    on_exit(on_exit_fp,"十八年后又是一条好汉");
    atexit(atexit_fp);
```

```

    for(int i=0; i<3; i++)
    {
        printf("我是进程%u\n",getpid());
        sleep(1);
    }

    return 0; // 等价于exit(0);
}
```

3、调用_exit/_Exit函数结束进程：

```
#include <unistd.h>
void _exit (int status);

该函数有一个完全等价的标准C版本：
#include <stdlib.h>
void _Exit (int status);
```

1、该函数一旦调用就不会返回，其父进程调用wait/waitpid函数可获得status的低8位数据。

2、进程退出之前，先关闭所有仍处于打开状态的文件描述符，将其所有子进程托付给孤儿院收养(init/upstart)，向父进程递送SIGCHILD信号。

```
#include <sys/stat.h>
#include <fcntl.h>

pid_t pid;
void sigchld(int signum)
{
    printf("我是进程%u,我儿子%u死了\n",getpid(),pid);
}

int main(int argc,const char* argv[])
{
    signal(SIGCHLD,sigchld);

    pid = fork();
    if(0 == pid)
    {
        sleep(3);
        _exit(123);
    }
    else
    {
        for(;;) pause();
    }

    return 0;
}
```

4、其它正常结束进程的方法

1、进程的最后一个线程执行完毕。

2、进程的最后一个线程调用pthread_exit函数。

2、进程的异常终止

1、进程收到某些信号，他杀。

2、进程自己调用abort函数，产生了SIGABRT信号，自杀。

3、进程的最后一个线程收到"取消"操作，并作出响应。

4、进程如果是异常结束的，on_exit/atexit登记的函数不会执行，缓冲区也不会冲刷，但依然会给父进程发送SIGCHLD信号并关闭所有打开状态的文件描述符。

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void atexit_fp(void)
{
    printf("我要结束了...!\n");
}

void on_exit_fp(int status,void* ptr)
{
    printf("我要结束了...,status=%d,%s\n",status,(char*)ptr);
}

void sigchld(int signum)
{
    printf("我儿子死了!\n");
}

int main(int argc,const char* argv[])
{
    signal(SIGCHLD,sigchld);
    pid_t pid = fork();
    if(0 == pid)
    {
        // 谁先登记的谁后执行
        on_exit(on_exit_fp,"十八年后又是一条好汉");
        atexit(atexit_fp);

        printf("hehe");
        kill(getpid(),SIGINT);
    }

    for(;;) pause();
}

```

3、进程的资源回收

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
功能：以阻塞状态等待子进程结束，并回收它的相关资源，只要有一个子进程结束它就返回，如果没有子进程就立即返回。
status：获取子进程的退出状态码，是输出型的参数。
返回值：成功返回子进程的PID，失败返回-1。

子进程的终止状态码status可借助sys/wait.h中定义的参数宏解析查看：
    WIFEXITED()      - 子进程是否正常终止
    WIFSIGNALED()    - 子进程是否异常终止，正常结束结果是0，异常结束结果是1
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main ()
{
    pid_t pid = fork ();
    if (pid == 0)
    {
        int status = 0x12345678;
        printf ("子进程：我是%u进程。我要以%#x状态退出。\n",
            getpid (), status);
        //kill(getpid(),9);
        sleep(3);
        return status;
    }

    printf ("父进程：我要等待子进程...\n");
    int status;
    pid = wait (&status);
    printf ("父进程：发现%u进程以%#x状态退出了。\n", pid,WEXITSTATUS (status));
    printf("%d\n",WIFSIGNALED(status));
    return 0;
}
```

```
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>

int main ()
{
    for (int i = 0; i < 5; i++)
    {
        pid_t pid = fork ();
        if (pid == 0)
        {
            printf ("子进程：我是%u进程。我要退出了。\n", getpid ());
            sleep(i);
            return 0;
        }
```

```
    }

    for (;;)
    {
        printf ("父进程：我要等待子进程...\n");

        int status;
        pid_t pid = wait(&status);
        if (pid == -1)
        {
            printf ("父进程：已经没有子进程可等了，走喽！\n");
            break;
        }
        printf ("父进程：发现%u进程退出了。\n", pid);
    }
    return 0;
}
```

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *status, int options);
功能：以阻塞状态等待子进程结束，与wait不同的是它可以指定等待哪个进程。
pid：要等待的子进程PID
    <-1 等待abs(pid)进程组中的任意子进程结束
    =-1 等待任意子进程结束，此时功能与wait等价。
    =0  等待同组的任意进程结束。
    >0  此时它代表着一个指定的子进程。
status：获取进程的结束状态码。
    WIFSTOPPED(status)      - 子进程是否处于暂停
    WIFCONTINUED(status)    - 子进程是否在暂停之后继续运行
options：
    0 表示忽略此参数，功能与wait一样。
    WNOHANG 如果没有子进程结束立即返回。
    WUNTRACED 如果有子进程暂停也立即返回
    WCONTINUED 如果有子进程从暂停状态切换为继续状态，立即返回。
返回值：成功返回子进程的PID，失败返回-1。
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main ()
{
    pid_t cpid[3];
    for (int i = 0; i < sizeof (cpid) / sizeof (cpid[0]); i++)
    {
        cpid[i] = fork ();
        if (cpid[i] == 0)
        {
            printf ("子进程：我是%u进程。我要退出了。\n", getpid ());
            return 0;
        }
    }

    for (i = 0; i < sizeof (cpid) / sizeof (cpid[0]); i++)
```

```
    {
        printf ("父进程：我要等待%u进程...\n", cpid[i]);

        pid_t pid = waitpid (cpid[i], 0, 0);
        if (pid == -1)
        {
            perror ("waitpid");
            return -1;
        }
        printf ("父进程：发现%u进程退出了。\n", pid);
    }
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>

int main (void)
{
    for (int i = 0; i < 3; i++)
    {
        pid_t pid = fork ();
        if (pid == 0)
        {
            printf ("子进程：我是%u进程。我要退出了。\n", getpid ());
            return 0;
        }
    }

    for (;;)
    {
        printf ("父进程：我要等待子进程...\n");
        pid_t pid = waitpid (-1, 0, WNOHANG);
        if (pid == -1)
        {
            printf ("父进程：已经没有子进程可等了，走喽！\n");
            break;
        }

        if (pid)
            printf ("父进程：发现%u进程退出了。\n", pid);
        else
            printf ("父进程：没发现子进程退出，干点儿别的...\n");
    }
    return 0;
}
```

二二、进程间通信介绍

1、什么是进程间通信

进程间通信（Interprocess communication 也叫IPC）：指两个或多个进程之间进行数据交互的过程。

2、为什么进程之间需要通信

由于进程采用的是虚拟空间+用户态/内核态机制，所以就导致进程与进程之间、进程与内核之间是互相独立的，如果想让多个进程协同工作解决复杂问题，就需要进程之间进行通信。

3、那类型的问题需要多进程协同工作

进程的运行机制：

从人类的感知来讲，所有进程是并行执行，但实际情况是所以进程轮流使用CPU，只是轮转的速度比较快，人类感受不到，CPU的一个内核就是一个运算单位，可以供一个进行使用（这也是CPU厂商不断追求CPU内核的数量原因）,CPU内核越多，同时执行的进程就越多。

CPU密集型问题：

主要特点是需要进行大量的计算，消耗CPU资源，比如计算圆周率、对视频进行高清解码等等，全靠CPU的运算能力，负责运算的进程越多，竞争的到执行时间就越长。但是任务越多，花在任务切换的时间就越多，CPU执行任务的效率就越低，是一种损人利已的做法。所以，要最高效地利用CPU，计算密集型任务同时进行的数量应当等于CPU的核心数。

IO密集型问题：

主要涉及到网络、磁盘IO的任务都是IO密集型任务，这类任务的特点是CPU消耗很少，任务的大部分时间都在等待IO操作完成（因为IO的速度远远低于CPU和内存的速度）。对于IO密集型任务，任务越多，CPU效率越高，但也有一个限度，因为创建进程也需要相关的资源，比如代码段、栈、文件描述符表、环境变量表。

所以处理CPU密集型程序时可以选择多进程实现，有效的利用多核提升效率；而IO密集型的由于99%的时间都花在IO上，花在CPU上的时间很少，所以多线程也能提高很大效率。

二三、简单的进程间通信

1、命令行参数

情况1：在终端执行程序时，给子进程传递命令行参数。

情况2：使用vfork+exec创建进程时，给子进程传递命令行参数。

只能在父子进程之间，在父进行创建子进程时使用，只能由父进行传递给子进程，单向通信，并且只能传递一些简单的字符串数据。

2、环境变量表

情况1：使用fork创建子进程时，子进程会拷贝一份父进程的环境变量表。

情况2：使用vfork+exec创建进程时，给子进程传递一份父进程的环境变量表，子进程拿到环境表后会进行拷贝。

只能在父子进程之间，父进行创建子进程时使用，只能由父进行传递给子进程，单向通信，并且只能传递一些简单的字符串数据。

4、信号

情况1：使用kill向指定的进程发送信号进行通信。

情况2：使用sigqueue向指定的进程发送信号，也可以附带一些简单的数据。

可以在任意进程之间进行通信，但也只能是互相告知某某事件发生了，即使用能使用sigqueue带一些数据，但只有能传递一个int类型整数，只有通过fork创建的子进程才能传递内存地址。

5、文件

情况1：使用文件+信号，让任意两个进程之间传递大量数据，但要各自控制好位置指针，协调好读写时间。

情况2：使用文件+文件锁，也可以让任意两个进程之间传递大量数据，但文件锁的操作比较麻烦，也有可能陷入死锁的情况。

单纯的使用文件进行通信，无法协调读取和写入的时间，空间造成文件内的数据混乱，所以必须配合使用信号或文件锁。

二四、传统的进程间通信

管道是UNIX系统最古老的进程间通信方式，是一种特殊的文件读写机制。

当进程从管道文件读取数据时，如果管道中没有数制则进程会进入阻塞状态，直到读取到数据才返回，所以就不需要使用信号、文件锁协调读写时间。

管道中的数据一旦读取完毕就会消失，不需要管理文件位置指针，所以管道文件通信比直接使用文件进行通信要方便很多。

1、有名管道：

在文件系统中创建出一个实体的管道文件，然后再使用系统I/O相关API进行相关操作。

使用函数创建：

```
int mkfifo(const char *pathname, mode_t mode);
功能：创建一个管道文件
pathname：管道文件的路径
mode：管道文件的权限
```

使用命令创建：

```
mkfifo <filo>
# echo hello > fifo
# cat fifo
```

步骤	进程A	函数	进程B	步骤
1	创建管道	mkfifo	----	
2	打开管道	open	打开管道	1
3	读写管道	read/write	读写管道	2
4	关闭管道	close	关闭管道	3
5	删除管道	unlink	----	

使用管道单向通信:

数据接收者:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo.h"

void sigfunc(int signum)
{
    printf("收到%d信号，停止通信!\n",signum);
    unlink(FIFO_PATH);
    exit(0);
}

int main(int argc,const char* argv[])
{
    for(int i=1; i<65; i++)
    {
        signal(i,sigfunc);
    }

    // 创建管道文件
    if(mkfifo(FIFO_PATH,0644))
    {
        perror("mkfile");
        return -1;
    }

    // 打开管道文件
    int fd = open(FIFO_PATH,O_RDONLY);
    if(0 > fd)
    {
        perror("open");
        return -1;
    }

    char buf[4096];
    for(;;)
    {
        // 读取数据
        int ret = read(fd,buf,sizeof(buf));
        if(0 == strcmp("quit",buf))
            break;

        printf("从管道读取%d个字节数据，内容是:%s\n",ret,buf);
    }

    printf("通信结束! \n");

    // 关闭管道文件
    close(fd);
```

```

    // 删除管道文件
    unlink(FIFO_PATH);
    return 0;
}
```

数据发送者:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo.h"

int main(int argc,const char* argv[])
{
    // 打开管道文件
    int fd = open(FIFO_PATH,O_WRONLY);
    if(0 > fd)
    {
        perror("open");
        return -1;
    }

    char buf[4096];
    for(;;)
    {
        printf("请输入要发送的数据: ");
        scanf("%s",buf);

        // 写入数据
        int ret = write(fd,buf,strlen(buf)+1);
        if(0 == strcmp("quit",buf))
            break;

    }

    printf("通信结束! \n");
    // 关闭管道文件
    close(fd);

    return 0;
}
```

使用管道进行进程间双向通信

```c
#ifndef FIFO_H
#define FIFO_H

#define AtoB_PATH   "./atob_fifo"
#define BtoA_PATH   "./btoa_fifo"

#endif//FIFO_H
```

进程A:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo.h"

void sigfunc(int signum)
{
    printf("收到%d信号，停止通信!\n",signum);
    unlink(AtoB_PATH);
    unlink(BtoA_PATH);
    exit(0);
}

int main(int argc,const char* argv[])
{
    // 注册信号处理函数
    for(int i=1; i<65; i++)
    {
        signal(i,sigfunc);
    }

    // 创建管道文件
    if(mkfifo(AtoB_PATH,0644) || mkfifo(BtoA_PATH,0644))
    {
        perror("mkfile");
        return -1;
    }

    // 打开管道文件
    int atob_fd = open(AtoB_PATH,O_WRONLY);
    int btoa_fd = open(BtoA_PATH,O_RDONLY);
    if(0 > atob_fd || 0 > btoa_fd)
    {
        perror("open");
        return -1;
    }

    char buf[4096];
    for(;;)
    {
        // 读取数据
        int ret = read(btoa_fd,buf,sizeof(buf));
```

```c
        if(0 == strcmp("quit",buf))
            break;

        printf("从管道读取%d个字节数据，内容是:%s\n",ret,buf);
        printf("请输入要返回的数据：");
        scanf("%s",buf);

        // 写入数据
        ret = write(atob_fd,buf,strlen(buf)+1);
        if(0 == strcmp("quit",buf))
            break;
    }

    printf("通信结束！\n");

    // 关闭管道文件
    close(atob_fd);
    close(btoa_fd);

    // 调用信号处理函数
    abort();
}
```

进程B：

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo.h"

int main(int argc,const char* argv[])
{
    // 打开管道文件
    int atob_fd = open(AtoB_PATH,O_RDONLY);
    int btoa_fd = open(BtoA_PATH,O_WRONLY);
    if(0 > atob_fd || 0 > btoa_fd)
    {
        perror("open");
        return -1;
    }

    char buf[4096];
    for(;;)
    {
        printf("请输入要发送的数据：");
        scanf("%s",buf);

        // 写入数据
        int ret = write(btoa_fd,buf,strlen(buf)+1);
        if(0 == strcmp("quit",buf))
            break;

```

```
        // 读取数据
        ret = read(atob_fd,buf,sizeof(buf));
        if(0 == strcmp("quit",buf))
            break;

        printf("从管道读取%d个字节数据，内容是:%s\n",ret,buf);

    }

    printf("通信结束！\n");

    // 关闭管道文件
    close(atob_fd);
    close(btoa_fd);

    return 0;
}
```

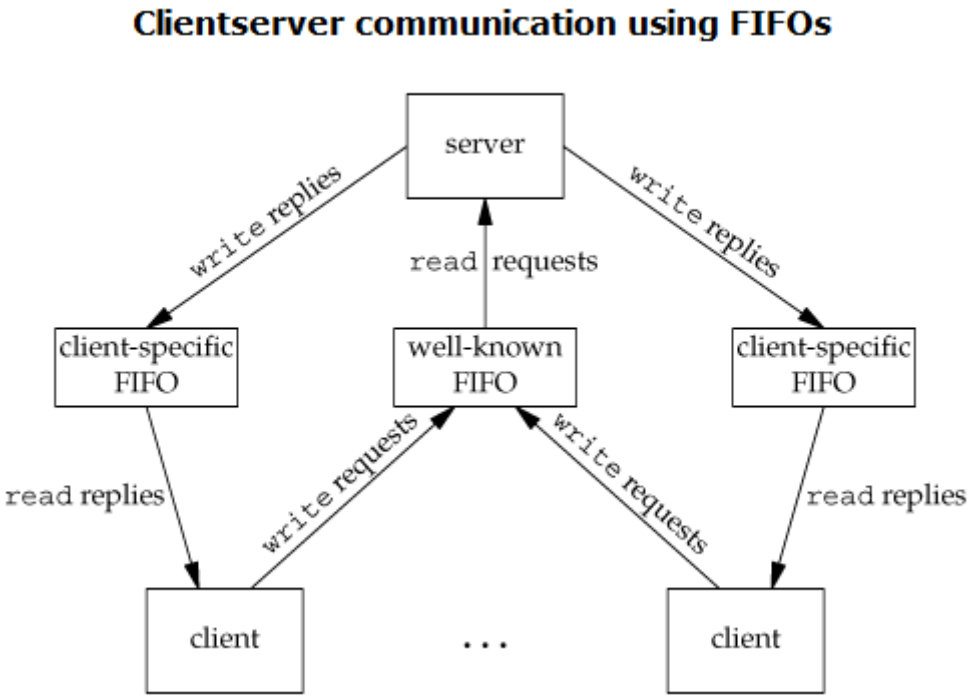
2、匿名管道：

只在内核中创建出管道文件对象并返回该对象的文件描述符，然后再使用系统I/O相关API进行相关操作。匿名管道不会在文件系统中显示，在创建时不需要提供路径，也不占用磁盘空间，而只使用内核空间内存存储数据，关闭文件描述符后自动回收。

```
#include <unistd.h>
int pipe (int pipefd[2]);
功能：创建出管道文件对象并返回该对象的文件描述符
pipefd：存储文件描述符的数组，其中pipefd[0]用于读，pipefd[1]用于写。
返回值：成功返回0，失败返回-1。

使用步骤：
    1、调用该函数在内核中创建管道文件，并通过其输出参数，获得分别用于读和写的两个文件描述符
    2、调用fork函数，创建子进程，子进程会拷贝pipefd数组。
    3、写数据的进程关闭读端(pipefd[0])，读数据的进程关闭写端(pipefd[1])
    4、父子进程传输数据
    5、父子进程分别关闭自己的文件描述符
```

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
int ctop_fds[2];
int ptoc_fds[2];

void child_process(void)
{
    // 使用ctop写所以关掉ctop的读，使用ptoc读所以关掉ptoc的写
    close(ctop_fds[0]);
    close(ptoc_fds[1]);

    char buf[4096];
    for(;;)
    {
        printf("进程%u请输入要发送的数据:",getpid());
        scanf("%s",buf);

        int ret = write(ctop_fds[1],buf,strlen(buf)+1);
```

```c
        if(0 == strcmp("quit",buf))
            break;

        ret = read(ptoc_fds[0],buf,sizeof(buf));
        printf("进程%u,读取到%d字节的数据，内容是：%s\n",getpid(),ret,buf);
        if(0 == strcmp("quit",buf))
            break;
    }
    close(ctop_fds[1]);
    close(ptoc_fds[0]);
}

void parent_process(void)
{
    // 使用ctop读所以关掉ctop的写，使用ptoc写所以关掉ptoc的读
    close(ctop_fds[1]);
    close(ptoc_fds[0]);

    char buf[4096];
    for(;;)
    {
        int ret = read(ctop_fds[0],buf,sizeof(buf));
        printf("进程%u,读取到%d字节的数据，内容是：%s\n",getpid(),ret,buf);
        if(0 == strcmp("quit",buf))
            break;

        printf("进程%u请输入要发送的数据:",getpid());
        scanf("%s",buf);

        ret = write(ptoc_fds[1],buf,strlen(buf)+1);
        if(0 == strcmp("quit",buf))
            break;
    }
    close(ctop_fds[0]);
    close(ctop_fds[1]);
}

int main(int argc,const char* argv[])
{
    if(pipe(ctop_fds) || pipe(ptoc_fds))
    {
        perror("pipe");
        return -1;
    }

    if(0 == fork())
        child_process();
    else
        parent_process();
    return 0;
}
```



二五、XSI机制的进程间通信

1、XSI介绍

什么是XSI：

X/Open国际联盟有限公司是一个欧洲基金会，它的建立是为了向UNIX环境提供标准，XSI是X/Open System Interface的缩写，也就是X/Open设计的系统接口。

X/Open的主要的目标是促进对UNIX系统、接口、网络和应用的开放式系统协议的制定。它还促进在不同的UNIX环境之间的应用程序的互操作性，以及支持对电气电子工程师协会对UNIX的可移植操作系统接口规范。

IPC对象：

它用于进程间通信的XSI-IPC内核对象，就是像匿名管道、文件内核对象一样，使用XSI方式进行进程间通信时，系统会在内核中创建一个XSI-IPC内核对象，让需要通信的进程共同访问。

XSI-IPC只存在于内核空间只，不在文件系统显示，通过IPC键进行创建、获取。

IPC键值：

用于创建XSI-IPC内核对象的凭证，是一个无符号整数，相当于IPC对象的名字，类似于文件名。
在创建创建XSI-IPC内核对象可以由创建者自定义，类似于给文件取名，但所有XSI-IPC内核对象都在同一个作用域下，所以有重名的风险，所以操作系统提供一个自动生成IPC键值的API。

```
#include <sys/types.h>
#include <sys/ipc.h>

key_t ftok(const char *pathname, int proj_id);
功能：自动生成一个IPC键值
pathname：当前项目的路径，必须一个真实存在的路径，它靠的是路径的位置，而不是字符串。
proj_id：当前项目的编号
注意：只要提供相同的路径和项目编号，就可以获得相同的IPC键值。
```

IPC描述符：

类似于文件描述符，是一个非负整数，它是IPC内核对象的给用户空间访问凭证，然后这个对象会以整数的形式提供给各个进程。

显示IPC对象命令：

```
ipcs -m - 显示共享内存(m: memory)
ipcs -q - 显示消息队列(q: queue)
ipcs -s - 显示信号量(s: semphore)
ipcs -a - 显示所有IPC对象(a: all)
```

删除IPC对象命令：

```
ipcrm -m ID - 删除共享内存
ipcrm -q ID - 删除消息队列
ipcrm -s ID - 删除信号量
```

2、共享内存：

基本原理：

在内核中开辟一内存，可以让其它进程的虚拟地址与它进行映射，这样就达到多个进程共享一块内存的目的，当一个进程向这块内存写数据时，其它进了也都可以读取到，这样就达到了通信的目的。

注意：这种通信方式不存在数据的复制，是最快的进程间通信方式。

使用方式：

当一个进程向共享内存写入数据后，内核不会通知其它进程，进程从共享内存读取数据时，无法分辨是否是通信进程新写入的数据，所以有以下三种固定的使用方式。

读写：一个进程只负责写入数据，别一个进程只负责读取数据，只要负责读的进程使用的是最新即可，这是单向通信的模式。

轮询：配合闹钟或定时器，每隔一段时间读取一个。

中断：配合信号，进程只要往里面写入数据就给对方发送一个信号，其它进程如果读取了数据也给对方发送一个已读取的信号。

相关API：

```
#include <sys/ipc.h>
#include <sys/shm.h>

int shmget(key_t key, size_t size, int shmflg);
功能：创建/获取一块内核中共享内存
key：IPC键值，相当于文件名
size：共享内存的字节数，建议取内存页字节数的整数倍，默认一页内存4096字节。
    若希望创建共享内存，则必需指定size参数，若只为获取已有的共享内存，则size参数可取0。
shmflg：
    0           - 表示该参数无效，获取共享内存，size的值也无效。
    IPC_CREAT   - 创建共享内存
    IPC_EXCL    - 互斥，如果共享内存已存在则创建失败
    mode        - 共享内存的权限，创建共享内存时一定要加
返回值：成功返回共享内存描述符，失败返回-1。
```

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
功能：加载共享内存(把进程中的虚拟地址与内核中的共享内存建立映射关系)
shmid：IPC描述符
shmaddr：虚拟地址，也可以为NULL，操作系统会自动分配。
shmflg：
      0          - 以读写方式使用共享内存。
      SHM_RDONLY - 以只读方式使用共享内存。
      SHM_RND    - 只在shmaddr参数非NULL时起作用，对shmaddr向下取内存页的整数倍，作为映射地址。
返回值：成功返回映射后的虚拟地址，并且内核将该共享内存的加载计数加1，失败返回-1。

int shmdt(const void *shmaddr);
功能：卸载共享内存，也就是取消映射

int shmctl(int shmid, int cmd, struct shmid_ds *buf);
功能：销毁共享内存、获取共享内存的属性、设置共享内存的属性
shmid：IPC描述符
cmd：
    IPC_STAT    获取共享内存的属性
    IPC_SET     设置共享内存的属性，uid，gid，mode可以设置。
    IPC_RMID    并非真正删除共享内存，只是做一个删除标记，禁止其被继续加载，但已有加载依然保留，只有当该共享内存的加载计数为0时，才真正被删除。

struct shmid_ds {
    struct ipc_perm shm_perm;   // 所有者及权限
    size_t shm_segsz;           // 共享内存的大小，以字节为单位
    time_t shm_atime;           // 最后加载时间
    time_t shm_dtime;           // 最后卸载时间
    time_t shm_ctime;           // 最后修改时间
    pid_t shm_cpid;             // 创建者的PID
    pid_t shm_lpid;             // 最后加载/卸载的者PID
    shmatt_t shm_nattch;        // 加载计数器
};

struct ipc_perm
{
   key_t          __key; // 键值
   uid_t          uid;   // 有效属主ID
   gid_t          gid;   // 有效属组ID
   uid_t          cuid;  // 有效创建者ID
   gid_t          cgid;  // 有效创建组ID
   unsigned short mode;  // 权限字
   unsigned short __seq; // 序列号
};
```

编程模型：

步骤	进程A	函数	进程B	步骤
1	创建共享内存	shmget	获取共享内存	1
2	加载共享内存	shmat	加载共享内存	2
3	使用共享内存	...	使用共享内存	3
4	卸载共享内存	shmdt	卸载共享内存	4
5	销毁共享内存	shmctl	----	

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <signal.h>

int shmfd;
void* shmptr;
pid_t* pids;
char* buf;

void shm_exit(void)
{
    // 卸载共享内存
    shmdt(shmptr);
    perror("shmdt");

    // 删除共享内存
    shmctl(shmfd,IPC_RMID,NULL);
    perror("shmctl");

    // 结束进程
    exit(0);
}

void sig_write(int signum)
{
    printf("请输入要发送的数据：");
    scanf("%s",buf);
    kill(pids[1],SIGUSR1);
    if(0 == strcmp("quit",buf))
        shm_exit();
}


void sig_read(int signum)
{
    printf("读取到数据：%s\n",buf);
    if(0 == strcmp("quit",buf))
        shm_exit();
    sig_write(0);
}

```

```c
// 首先运行
int main(int argc,const char* argv[])
{
    signal(SIGUSR1,sig_read);
    signal(SIGUSR2,sig_write);

    // 创建共享内存
    shmfd = shmget(ftok(".",1),4096,IPC_CREAT|IPC_EXCL|0600);
    if(0 > shmget)
    {
        perror("shmget");
        return -1;
    }

    // 加载共享内存
    shmptr = shmat(shmfd,NULL,0);
    if(-1 == (int)shmptr)
    {
        perror("shmat");
        return -1;
    }

    // 双向互相告知进程号
    pids = shmptr;
    pids[0] = getpid();
    buf = shmptr+8;

    // 进程休眠
    for(;;) pause();
    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <signal.h>

int shmfd;
void* shmptr;
pid_t* pids;
char* buf;

void shm_exit(void)
{
    // 卸载共享内存
    shmdt(shmptr);
    perror("shmdt");

    // 结束进程
    exit(0);
}

void sig_write(int signum)
{
```

```c
    printf("请输入要发送的数据：");
    scanf("%s",buf);
    kill(pids[0],SIGUSR1);
    if(0 == strcmp("quit",buf))
        shm_exit();
}


void sig_read(int signum)
{
    printf("读取到数据：%s\n",buf);
    if(0 == strcmp("quit",buf))
        shm_exit();
    sig_write(0);
}

// 后运行
int main(int argc,const char* argv[])
{
    signal(SIGUSR1,sig_read);
    signal(SIGUSR2,sig_write);

    // 获取共享内存
    shmfd = shmget(ftok(".",1),0,0);
    if(0 > shmget)
    {
        perror("shmget");
        return -1;
    }

    // 加载共享内存
    shmptr = shmat(shmfd,NULL,0);
    if(-1 == (int)shmptr)
    {
        perror("shmat");
        return -1;
    }

    // 双向互相告知进程号
    pids = shmptr;
    pids[1] = getpid();
    buf = shmptr+8;

    kill(pids[0],SIGUSR2);

    // 进程休眠
    for(;;) pause();

    return 0;
}
```

3、消息队列：

基本原理：

是由系统内核维护的一个链式队列，每个一条消息由类型、数据、数据长度组成。

和管道类似，但它可以双向通信，还可以按类型接收消息，而不是顺序。

相关API：

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgget(key_t key, int msgflg);
功能：以key参数为键值 创建或获取消息队列
msgflg：
    0          - 获取，不存在即失败。
    IPC_CREAT - 创建，不存在即创建，已存在即获取，除非...
    IPC_EXCL  - 排斥，已存在即失败。
    mode       - 消息队列的权限
返回值：成功返回消息队列标识符，失败返回-1

int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
功能：向消息队列发送数据

msgp：向一个包含消息类型和消息内容的内存块，以下是内存块的格式
    struct msgbuf
    {
        long mtype;      // 数据的长度，注意不包括消息类型。
        char mtext[n];   // 消息内容，也可以使用柔性数组
    };
msgsz：消息内容的字节数，不包含消息类型
msgflg：
    0 当消息队列中没有足够的空闲空间时，此函数会阻塞。
    IPC_NOWAIT 当消息队列中没有足够的空闲空间时，此函数不会阻塞，而是返回-1。

ssize_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp,int msgflg);
功能：从消息队列接收数据
msgp：消息内存块的首地址，消息由消息类型+消息数据组成，也叫消息数据缓冲区
msgsz：消息数据缓冲区的字节数，如果实际消息的字符数大于msgsz，则立即返回-1
msgtyp：
    >0 - 要接收的消息类型
    =0 - 返回消息队列中的第一条消息。
    <0 - 返回消息队列中类型小于等于msgtyp的绝对值的消息，如果有多个，则返回类型最小者。
msgflg：
    MSG_EXCEPT 返回消息队列中第一个类型不为msgtyp的消息，此时应 msgtyp>0
    IPC_NOWAIT 如果没有满足条件的消息，立即返回
    MSG_NOERROR 只读取该消息的前msgsz字节，如果有剩余则丢弃
返回值：返回接收到的消息的字节数

int msgctl(int msqid, int cmd, struct msqid_ds *buf);
功能：销毁消息队列、获取消息队列的属性、设置消息队列的属性
cmd取值：
    IPC_STAT - 获取消息队列的属性，通过buf参数输出
    IPC_SET  - 设置消息队列的属性，通过buf参数输入，仅以下四个属性可设置：
         msqid_ds::msg_perm.uid
         msqid_ds::msg_perm.gid、
         msqid_ds::msg_perm.mode
         msqid_ds::msg_qbytes
```

```
    IPC_RMID - 立即删除消息队列同，此时所有阻塞在对该消息队列的，msgsnd和msgrcv函数调用，都会立即返回失败。

struct msqid_ds {
    struct ipc_perm msg_perm;     // 权限信息
    time_t          msg_stime;    // 随后发送时间
    time_t          msg_rtime;    // 最后接收时间
    time_t          msg_ctime;    // 最后改变时间
    unsigned long   __msg_cbytes; // 消息队列中的字节数
    msgqnum_t       msg_qnum;     // 消息队列中的消息数
    msglen_t        msg_qbytes;   // 消息队列能容纳的最大字节数
    pid_t           msg_lspid;    // 最后发送进程PID
    pid_t           msg_lrpid;    // 最后接收进程PID
};

struct ipc_perm {
    key_t          __key; // 键值
    uid_t          uid;   // 有效属主ID
    gid_t          gid;   // 有效属组ID
    uid_t          cuid;  // 有效创建者ID
    gid_t          cgid;  // 有效创建组ID
    unsigned short mode;  // 权限字
    unsigned short __seq; // 序列号
};
```

编程模型：

步骤	进程A	函数	进程B	步骤
1	创建消息队列	msgget	获取消息队列	1
2	发送接收消息	msgsnd/msgrcv	发送接收消息	2
3	销毁消息队列	msgctl	----	

```
#ifndef MSG_H
#define MSG_H

#define MSG_MAX (256)

typedef struct Msg
{
    long type;
    char msg[MSG_MAX];
}Msg;

/*
#define LOGIN_TYPE      1
#define REGISTER_TYPE   2
#define DESTORY_TYPE    3
*/

#endif//MSG_H
```

```
#include <stdio.h>
```

```c
#include <string.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>
#include "msg.h"

int main(int argc,const char* argv[])
{

    int msgfd = msgget(ftok(".",1),IPC_CREAT|IPC_EXCL|0644);
    if(0 > msgget)
    {
        perror("msgget");
        return -1;
    }

    Msg msg;
    for(;;)
    {
        // 从消息队列读取消息
        printf("等待消息...\n");
        int ret = msgrcv(msgfd,&msg,sizeof(msg.msg),1,0);
        printf("recv byte:%d type:%ld msg:%s\n",ret,msg.type,msg.msg);
        if(0 == strcmp("quit",msg.msg))
            break;

        printf("请输入要返回的内容: ");
        scanf("%s",msg.msg);

        msg.type = 2;
        ret = msgsnd(msgfd,&msg,strlen(msg.msg)+1,0);
        perror("msgsnd");
        if(0 == strcmp("quit",msg.msg))
            break;
    }

    msgctl(msgfd,IPC_RMID,NULL);
    perror("msgctl");

    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>
#include "msg.h"

int main(int argc,const char* argv[])
{

    int msgfd = msgget(ftok(".",1),0);
    if(0 > msgget)
    {
```

```
        perror("msgget");
        return -1;
    }

    Msg msg;
    for(;;)
    {
        printf("请输入要发送的内容：");
        scanf("%s",msg.msg);
        msg.type = 1;

        int ret = msgsnd(msgfd,&msg,strlen(msg.msg)+1,0);
        perror("msgsnd");
        if(0 == strcmp("quit",msg.msg))
            break;

        // 从消息队列读取消息
        printf("等待消息...\n");
        ret = msgrcv(msgfd,&msg,sizeof(msg.msg),2,0);
        printf("recv byte:%d type:%ld msg:%s\n",ret,msg.type,msg.msg);
        if(0 == strcmp("quit",msg.msg))
            break;

    }
    return 0;
}
```

4、信号量

基本原理：

所谓信号量就是内核中维护的一个全局变量，被当作计数器，用于限制多个进程对有限共享资源的访问。

1、假定操作系统有n个共享资源，则需要把信号量的值设置为n。

2、有m个进程需要以独占方式使用k个共享资源，且 m*k > n。

3、每个进程在使用共享资源前要尝试执行 信号量-k 操作。

4、如果 信号量-k >= 0，说明剩余的共享资源数量够用，当进程获得该资源后，需执行信号量 -= k，然后尽情的使用共享资源，当不再使用该资源时，需执行信号量 += k。

5、如果信号量 -k < 0，说明剩余的共享资源数量不够用，进程就会阻塞等待该资源，直到信号量-k >= 0，进程被唤醒，执行步骤4；

注意：进程之间的共享资源几乎没有，所以很少用到。

相关API：

```
int semget (key_t key, int nsems, int semflg);
功能：以key参数为键值 创建或获取信号量集
nsems：参数表示集合中的信号量数，或获取已有的信号量集合则nsems取0
semflg取值：
      0          - 获取，不存在即失败
      IPC_CREAT  - 创建，不存在即创建，已存在则获取
      IPC_EXCL   - 排斥，已存在即失败
      mode       - 消息队列的权限
返回值：成功返回信号量集合标识，失败返回-1。
```

```

int semop (int semid, struct sembuf* sops,unsigned nsops);
功能：对semid参数所标识的信号量集合
sops：struct sembuf结构数组
nsops：结构数组的长度
struct sembuf {
    unsigned short sem_num; // 信号量下标
    short          sem_op;  // 操作数
        // 若sem_op大于0，则将其加到第sem_num个信号量上，以表示对资源的释放；
        // 若sem_op小于0，则将其加到第sem_num个信号量上，以表示对资源的获取；
    short          sem_flg; // 操作标记
        // 0 若第sem_num个信号量的计数值不够减(信号量不能为负)，则进程会阻塞，直到该信号量够减为止，以表示对资源的等待；
        // IPC_NOWAIT，则当第sem_num个信号量的计数值不够减时，此函数不会阻塞，而是返回-1，以便在等待资源的同时还可做其它处理；
        // 若sem_op等0，则直到第sem_num个信号量的计数值为0时才返回，除非sem_flg包含IPC_NOWAIT位。
};

int semctl (int semid, int semnum, int cmd);
int semctl (int semid, int semnum, int cmd,union semun arg);
功能：获取信号量属性、设置信号量属性、删除信号量
cmd取值：
    IPC_STAT - 获取信号量集合的属性，通过arg.buf输出。
    IPC_SET  - 设置信号量集合的属性，通过arg.buf输入，仅以下三个属性可设置：
        semid_ds::sem_perm.uid
        semid_ds::sem_perm.gid
        semid_ds::sem_perm.mode
    IPC_RMID立即删除信号量集合。
         此时所有阻塞在对该信号量集合的，semop函数调用，都会立即返回失败。
    GETALL   - 获取信号量集合中每个信号量的计数值，通过arg.array输出。
    SETALL   - 设置信号量集合中每个信号量的计数值，通过arg.array输入。
    GETVAL   - 获取信号量集合中，第semnum个信号量的计数值，通过返回值输出。
    SETVAL   - 设置信号量集合中，第semnum个信号量的计数值，通过arg.val输入。
semnum：
    只有针对信号量集合中具体某个信号量的操作，才会使用semnum参数。
    针对整个信号量集合的操作，会忽略semnum参数。

union semun {
    int              val;   // Value for SETVAL
    struct semid_ds* buf;   // Buffer for IPC_STAT, IPC_SET
    unsigned short*  array; // Array for GETALL, SETALL
    struct seminfo*  __buf; // Buffer for IPC_INFO
};

struct semid_ds {
    struct ipc_perm sem_perm;  // Ownership and permissions
    time_t          sem_otime; // Last semop time
    time_t          sem_ctime; // Last change time
    unsigned short  sem_nsems; // No. of semaphores in set
};

struct ipc_perm {
    key_t          __key; // 键值
    uid_t          uid;   // 有效属主ID
    gid_t          gid;   // 有效属组ID
    uid_t          cuid;  // 有效创建者ID
    gid_t          cgid;  // 有效创建组ID
```

```
    unsigned short mode;  // 权限字
    unsigned short __seq; // 序列号
};

struct  seminfo {
    int semmap; /* Number of entries in semaphoremap; unused within kernel */
    int semmni; /* Maximum number of semaphore sets */
    int semmns; /* Maximum number of semaphores in all semaphore sets */
    int semmnu; /* System-wide maximum number of undo structures; unused within kernel */
    int semmsl; /* Maximum number of semaphores in a set */
    int semopm; /* Maximum number of operations for semop(2) */
    int semume; /* Maximum number of undo entries per process; unused within kernel */
    int semusz; /* Size of struct sem_undo */
    int semvmx; /* Maximum semaphore value */
    int semaem; /* Max. value that can be recorded for semaphore adjustment (SEM_UNDO) */
};

```

编程模型:

步骤	进程A	函数	进程B	步骤
1	创建信号量	semget	获取信号量	1
2	初始信号量	semctl	----	
3	加减信号量	semop	加减信号量	2
4	销毁信号量	semctl	----	

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <unistd.h>

int semfd;
void write_pid(int sec)
{
    struct sembuf sops[1] = {{0,-1,0}};
    semop(semfd,sops,1);

    int val = semctl(semfd,0,GETVAL);
    printf("子进程:%u 我骑了一头小毛驴,还有%d头\n",getpid(),val);

    sleep(sec);
    sops[0].sem_op = 1;
    semop(semfd,sops,1);

    val = semctl(semfd,0,GETVAL);
    printf("子进程:%u 我还了一头小毛驴,还有%d头\n",getpid(),val);
}
```

```c

int main(int argc,const char* argv[])
{
    // 创建信号量
    semfd = semget(ftok(".",1),1,IPC_CREAT|0600);
    if(0 > semfd)
    {
        perror("semget");
        return 0;
    }


    // 初始化信号量
    struct sembuf sops[1] = {{0,5,0}};
    semop(semfd,sops,1);
    perror("semop");
    printf("父进程：我前5头小毛驴!\n");

    for(int i=0; i<10; i++)
    {
        pid_t pid = fork();
        if(0 == pid)
        {
            write_pid(i);
            return 0;
        }
    }

    int status;
    pid_t child_pid;
    while(-1 != (child_pid = wait(&status)))
    {
        printf("子进程:%u 结束 返回值：%d\n",child_pid,WEXITSTATUS (status));
    }

    semctl(semfd,0,IPC_RMID);
    perror("semctl");
    return 0;
}
```

二六、线程介绍

1、什么是线程

线程是操作系统能够进行运算执行的最小单位，它被包含在进程之中，是进程中的实际运作单位。一条线程指的是进程中一个单一顺序的控制流，一个进程中可以并发多个线程，每条线程并行执行不同的任务。

2、线程的发展简史

60年代，在操作系统中能拥有资源和独立运行的基本单位是进程。

随着计算机技术的发展，进程出现了很多弊端：

一是由于进程是资源拥有者，创建、撤消与切换存在较大的时空开销，因此需要引入轻型进程；

二是由于对称多处理机出现，可以满足多个运行单位，而多个进程并行开销过大。

因此在80年代，出现了能独立运行的基本单位——线程（Threads）。

3、线程的调度策略

线程是独立调度和分派的基本单位，有三种不同的调试策略：

1、线程可以为操作系统内核调度的内核线程，如Win32线程；

2、由用户进程自行调度的用户线程，如Linux平台的POSIX Thread；

3、由内核与用户进程进行混合调度，如Windows 7的线程。

4、多线程适用的范围

一个进程可以有很多线程，每条线程并行执行不同的任务。
在多核或多CPU，或支持Hyper-threading的CPU上使用多线程程序设计的好处是显而易见，即提高了程序的执行吞吐率。

在单CPU单核的计算机上，使用多线程技术，可以把进程中负责I/O处理、人机交互而常被阻塞的部分与密集计算的部分分开来执行，原因就是线程占用的资源少，被阻塞时不浪费资源。

5、线程的特点

轻型实体：

线程中的实体基本上不拥有系统资源，只是有一点必不可少的、能保证独立运行的资源。线程的实体包括用于指示被执行指令序列的程序计数器、局部变量、状态参数和返回地址。

线程是动态概念，它的动态特性由线程控制块TCB（Thread Control Block）描述，包括以下信息：

1、线程状态

2、当线程不运行时，被保存的现场资源

3、一组执行堆栈

4、存放每个线程的局部变量主存区

5、访问同一个进程中的主存和其它资源

独立调度和分派的基本单位：

在多线程OS中，线程是能独立运行的基本单位，因而也是独立调度和分派的基本单位。由于线程很“轻”，故线程的切换非常迅速且开销小（在同一进程中的）。

可并发执行：

在一个进程中的多个线程之间，可以并发执行，甚至允许在一个进程中所有线程都能并发执行；同样，不同进程中的线程也能并发执行，充分利用和发挥了处理机与外围设备并行工作的能力。

共享进程资源：

在同一进程中的各个线程，都可以访问该进程的用户空间，此外，还可以访问进程所拥有的已打开文件、定时器、信号量机构等，线程可以共享该进程所拥有的资源。所以线程之间互相通信不必调用内核。

6、线程与进程的区别

资源：

进程采用虚拟空间+用户态/内核态机制，所以就导致进程与进程之间是互相独立的，各自的资源不可见。

在同一进程中的各个线程都可以共享该进程所拥有的资源。

通信：

由于进程之间是互相独立的，需要使用各种IPC通信机制，保障多个进程协同工作。

同一进程中的各个线程共享该进程所拥有的资源，线程间可以直接读写进程数据段来进行通信，但需要线程同步和互斥手段的辅助，以保证数据的一致性。

调度：

无论系统采用什么样的线程调试策略，线程上下文切换都比进程上下文切换要快得多。

身份：

进程是个资源单位，线程是个执行单位，并且线程是进程的一部分，线程需要进程安身立命，进程也需要线程当牛做马。

二七、POSIX线程库

1、POSIX线程库介绍：

POSIX线程（POSIX Threads，常被缩写为pthread）是POSIX的线程标准，定义了创建和操纵线程的一套API。
实现POSIX 线程标准的库常被称作pthread，一般用于Unix-likePOSIX 系统，如Linux、Solaris。但是Microsoft Windows上的实现也存在，例如直接使用Windows API实现的第三方库pthread-w32。

2、API具体内容：

pthread定义了一套C语言的类型、函数与常量，它以pthread.h头文件和一个接口库libpthread.so，gcc和g++编译器没有默认链接该库，需要程序员使用 -l pthread 参数进行手动链接。

pthread API中大致共有100个函数调用，全都以"pthread_"开头，并可以分为四类：

1、线程管理，如创建线程，等待线程，查询线程状态等。

2、互斥锁，有创建、摧毁、锁定、解锁、设置属性等操作

3、条件变量，有创建、摧毁、等待、通知、设置与查询属性等操作

4、使用了互斥锁的线程间的同步管理。

3、创建线程

```
int pthread_create (pthread_t* restrict thread,
                    const pthread_attr_t* restrict attr,
                    void* (*start_routine) (void*),
                    void* restrict arg);

thread        - 线程ID，输出型参数。我们目前使用的Linux中pthread_t即unsigned long int
attr          - 线程属性，NULL表示缺省属性，如果没有特殊需求，一般写NULL即可
start_routine - 线程入口函数指针，参数和返回值的类型都是void*
                启动线程本质上就是调用一个函数，只不过是在一个独立的线程中调用的，函数返回即线程结束
```

```
arg             - 传递给线程过程函数的参数
返回值：成功返回0，失败返回错误码，但不会修改全局的错误变量，也就是无法使用perror获取错误原因。

注意：
    1、restrict: C99引入的编译优化指示符，提高重复解引用同一个指针的效率。
    2、应设法保证在线程过程函数执行期间，其参数所指向的目标持久有效。
```

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

void* run(void* arg)
{
    for(int i=0; i<10; i++)
    {
        printf("%d\n",i);
    }
}

int main(int argc,const char* argv[])
{
    pthread_t tid;
    int errno = pthread_create(&tid,NULL,run,NULL);
    printf("%s\n",strerror(errno));
    // pthread库的函数，不会影响errno全局变量，所以无法使用perror显示错误原因，它们会返回错误码，再使用strerror把错误码转换成错误原因。
    // 当资源耗尽时创建线程就会失败，大约350线程

    return 0;
    // main函数中的return status等价于exit(status),可以把整个进程都杀死，是因为整个进程死了，子进程才死的，而不是因为主线程死了，子线程才死的。子线程的死亡与主线程没有关系，主线程无法影响子线程。
}

```

4、线程回收

```
int pthread_join (pthread_t thread, void** retval);
功能：等待thread参数所标识的线程结束，并回收相关资源，如果thread线程没有结束则阻塞
retval：获得线程正常结束时的返回值，是输出型的参数，用于获取线程入口函数的返回值。
返回值：成功返回0，失败返回错误码

从线程过程函数中返回值的方法：
    1、线程过程函数将所需返回的内容放在一块内存中，返回该内存的地址，保证这块内存在函数返回，即线程结束，以后依然有效；
    2、若retval参数非NULL，则pthread_join函数将线程入口函数所返回的指针，拷贝到该参数所指向的内存中；
    3、线程入口函数所返回的指针指向text、data、bss内存段的数据，如果指向heap内存段，则还需保证在用过该内存之后释放之。
```

```c
#include <string.h>
#include <unistd.h>
```

```c
#include <pthread.h>

void* run1(void* arg)
{
    for(int i=0; i<5; i++)
    {
        printf("%d\n",i);
    }
}

void* run2(void* arg)
{
    pthread_join(*(pthread_t*)arg,NULL);
    for(int i=5; i<10; i++)
    {
        printf("%d\n",i);
    }
}

int main(int argc,const char* argv[])
{
    pthread_t tid1,tid2;
    pthread_create(&tid1,NULL,run1,NULL);
    pthread_create(&tid2,NULL,run2,&tid1);

    pthread_join(tid2,NULL);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#define PAI 3.14159

void* thread_area (void* arg)
{
    double r = *(double*)arg;
    /*
    double s;
    s = PAI * r * r;
    return &s; 如果返回的是局部变量的地址，则pthread_join函数获取到的返回值是NULL
    */

    /*
    static double s;
    s = PAI * r * r;
    return &s;
    */

    double* s = malloc (sizeof (double));
    *s = PAI * r * r;
    return s;
}

int main ()
```

```
{

    double r;
    scanf ("%lf", &r);

    pthread_t tid;
    // 可以传递栈内存的地址，但有安全隐患
    int error = pthread_create (&tid, NULL, thread_area, &r);
    if (error)
    {
        fprintf (stderr, "pthread_create: %s\n", strerror (error));
        return -1;
    }

    double* s;
    if ((error = pthread_join (tid, (void**)&s)) != 0)
    {
        fprintf (stderr, "pthread_join: %s\n", strerror (error));
        return -1;
    }

    printf ("s = %g\n", *s);
    free (s);

    return 0;
}
```

5、区分主、子线程

```
pthread_t pthread_self (void);
成功返回调用线程的ID，不会失败。

int pthread_equal (pthread_t t1, pthread_t t2);
功能：若参数t1和t2所标识的线程ID相等，则返回非零，否则返回0。

注意：某些实现的pthread_t不是unsigned long int类型，可能是结构体类型，无法通过“==”判断其相等性。
```

```
#include <stdio.h>
#include <string.h>
#include <pthread.h>

pthread_t g_main;

void* ismain (void* arg)
{
  //if (pthread_self () == g_main)
    if (pthread_equal (pthread_self (), g_main))
        printf ("我是主线程！\n");
    else
        printf ("我不是主线程！\n");
}

int main ()
{
    g_main = pthread_self ();
```

```

    ismain (NULL);

    pthread_t tid;
    int error = pthread_create (&tid, NULL, ismain, NULL);
    if (error)
    {
        fprintf (stderr, "pthread_create: %s\n", strerror (error));
        return -1;
    }

    if ((error = pthread_join (tid, NULL)) != 0)
    {
        fprintf (stderr, "pthread_join: %s\n", strerror (error));
        return -1;
    }

    return 0;
}
```

6、终止线程

方法1：从线程入口函数中return。

方法2：调用pthread_exit函数。

```
void pthread_exit (void* retval);
retval - 和线程过程函数的返回值语义相同。
```

注意：在任何线程中调用exit函数都将终止整个进程。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#define PAI 3.14159

void* thread_area (void* arg)
{
    double r = *(double*)arg;
    double* s = malloc (sizeof (double));
    //exit (0);
    *s = PAI * r * r;
    pthread_exit (s); // <==> return s;
    *s = 2 * PAI * r;
    return s;
}

int main ()
{
    printf ("r = ");
    double r;
    scanf ("%lf", &r);

    pthread_t tid;
    int error = pthread_create (&tid, NULL, thread_area, &r);
```

```
    if (error)
    {
        fprintf (stderr, "pthread_create: %s\n", strerror (error));
        return -1;
    }

    double* s;
    if ((error = pthread_join (tid, (void**)&s)) != 0)
    {
        fprintf (stderr, "pthread_join: %s\n", strerror (error));
        return -1;
    }

    printf ("s = %g\n", *s);
    free (s);

    return 0;
}
```

7、线程分离

同步方式(非分离状态)：创建线程之后主线程调用pthread_join函数等待其终止，并释放线程资源。

异步方式(分离状态)：无需创建者等待，线程终止后自行释放资源。

```
int pthread_detach (pthread_t thread);
功能：使thread参数所标识的线程进入分离(DETACHED)状态。
返回值：成功返回0，失败返回错误码。
```

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

void* thread_proc (void* arg)
{
    for (int i = 0; i < 200; i++)
    {
        putchar ('-');
        usleep (50000);
    }
    return NULL;
}

int main ()
{
    pthread_t tid;
    int error = pthread_create (&tid, NULL, thread_proc, NULL);
    if (error)
    {
        fprintf (stderr, "pthread_create: %s\n", strerror (error));
        return -1;
    }

    if ((error = pthread_detach (tid)) != 0)
    {
        fprintf (stderr, "pthread_detach: %s\n", strerror (error));
```

```
        return -1;
    }
    /*
    if ((error = pthread_join (tid, NULL)) != 0)
    {
        fprintf (stderr, "pthread_join: %s\n", strerror (error));
        return -1;
    }
    */
    for (int i = 0; i < 200; i++)
    {
        putchar ('+');
        usleep (100000);
    }

    printf ("\n");
    return 0;
}
```

8、取消线程

向发送取消请求：

```
int pthread_cancel (pthread_t thread);
功能：该函数只是向线程发出取消请求，并不等待线程终止。

缺省情况下，线程在收到取消请求以后，并不会立即终止，而是仍继续运行，直到其达到某个取消点。
在取消点处，线程检查其自身是否已被取消了，并做出相应动作。
```

设置可取消状态：

```
int pthread_setcancelstate (int state,int* oldstate);
成功返回0，并通过oldstate参数输出原可取消状态(若非NULL)，失败返回错误码。

state取值：
   PTHREAD_CANCEL_ENABLE  - 接受取消请求(缺省)。
   PTHREAD_CANCEL_DISABLE - 忽略取消请求。
```

设置可取消类型：

```
int pthread_setcanceltype (int type, int* oldtype);

成功返回0，并通过oldtype参数输出原可取消类型
(若非NULL)，失败返回错误码。

type取值：
   PTHREAD_CANCEL_DEFERRED      - 延迟取消(缺省)。
       被取消线程在接收到取消请求之后并不立即响应，
        而是一直等到执行了特定的函数(取消点)之后再响应该请求。
   PTHREAD_CANCEL_ASYNCHRONOUS - 异步取消。
       被取消线程可以在任意时间取消，不是非得遇到取消点才能被取消。
       但是操作系统并不能保证这一点。
```

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>
#include <pthread.h>

void elapse (void)
{
    size_t i;
    for (i = 0; i < 800000000; ++i);
}

void* thread_proc (void* arg)
{
    /*
    // 设置当前线程忽略其它线程的取消请求，默认情况线程会接收取消请求
    int error = pthread_setcancelstate (PTHREAD_CANCEL_DISABLE, NULL);
    if (error)
    {
        fprintf (stderr, "pthread_setcancelstate: %s\n", strerror (error));
        exit (EXIT_FAILURE);
    }
    */

    /*
    设置当前线程的取消类型为立即取消，默认情况是延时取消，到达取消点之后才会响应取消
    int error = pthread_setcanceltype (PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    if (error)
    {
        fprintf (stderr, "pthread_setcanceltype: %s\n", strerror (error));
        exit (EXIT_FAILURE);
    }
    */
    for (;;)
    {
        printf ("线程：子在川上曰，逝者如斯夫。\n");
        elapse ();
    }

    return NULL;
}

int main ()
{
    printf ("按<回车>取消线程...\n");

    pthread_t tid;
    int error = pthread_create (&tid, NULL, thread_proc, NULL);
    if (error)
    {
        fprintf (stderr, "pthread_create: %s\n", strerror (error));
        return -1;
    }

    getchar ();

    if ((error = pthread_cancel (tid)) != 0)
    {
        fprintf (stderr, "pthread_cancel: %s\n", strerror (error));
        exit (EXIT_FAILURE);
    }
```

```c

    printf ("已发送取消请求，等待线程终止...\n");

    if ((error = pthread_join (tid, NULL)) != 0)
    {
        fprintf (stderr, "pthread_join: %s\n", strerror (error));
        return -1;
    }

    printf ("线程已终止。\n");
    return 0;
}
```

8、线程属性

```c
int pthread_create (pthread_t* restrict thread,
                    const pthread_attr_t* restrict attr,
                    void* (*start_routine) (void*),
                    void* restrict arg);
//创建线程函数的第二个参数即为线程属性，传空指针表示使用缺省属性。

typedef struct {
    // 分离状态
    int detachstate;
       // PTHREAD_CREATE_DETACHED - 分离线程。
       // PTHREAD_CREATE_JOINABLE(缺省) - 可汇合线程。

    // 竞争范围
    int scope;
       // PTHREAD_SCOPE_SYSTEM - 在系统范围内竞争资源。
       // PTHREAD_SCOPE_PROCESS(Linux不支持) - 在进程范围内竞争资源。


    // 继承特性
    int inheritsched;
       // PTHREAD_INHERIT_SCHED(缺省) - 调度属性自创建者线程继承。
       // PTHREAD_EXPLICIT_SCHED - 调度属性由后面两个成员确定。


    // 调度策略
    nt schedpolicy;
        // SCHED_FIFO - 先进先出策略。
            // 没有时间片。
            // 一个FIFO线程会持续运行，直到阻塞或有高优先级线程就绪。
            // 当FIFO线程阻塞时，系统将其移出就绪队列，待其恢复时再加到同优先级就绪队列的末尾。
            // 当FIFO线程被高优先级线程抢占时，它在就绪队列中的位置不变。
            // 因此一旦高优先级线程终止或阻塞，被抢占的FIFO线程将会立即继续运行。
        // SCHED_RR - 轮转策略。
            // 给每个RR线程分配一个时间片，一但RR线程的时间片耗尽，系统即将移到就绪队列的末尾。
            // SCHED_OTHER(缺省) - 普通策略。
            // 静态优先级为0。任何就绪的FIFO线程或RR线程，都会抢占此类线程。

    // 调度参数
    struct sched_param schedparam;
        // struct sched_param {
```

```
        //     int sched_priority; /* 静态优先级 */
        // };


    // 栈尾警戒区大小(字节)  缺省一页(4096字节)。
    size_t guardsize;

    // 栈地址
    void* stackaddr;

    // 栈大小(字节)
    size_t stacksize;
} pthread_attr_t;

不要手动读写该结构体，而应调用pthread_attr_set/get函数设置/获取具体属性项。
```

1、设置线程属性：

初始化线程属性结构体：

```
pthread_attr_t attr = {}; // 不要使用这种方式
int pthread_attr_init (pthread_attr_t* attr);
```

设置具体线程属性项：

```
int pthread_attr_setdetachstate (pthread_attr_t* attr,int detachstate);
int pthread_attr_setscope (pthread_attr_t* attr,int scope);
int pthread_attr_setinheritsched (pthread_attr_t* attr,int inheritsched);
int pthread_attr_setschedpolicy (pthread_attr_t* attr,int policy);
int pthread_attr_setschedparam (pthread_attr_t* attr,const struct sched_param* param);
int pthread_attr_setguardsize (pthread_attr_t* attr,size_t guardsize);
int pthread_attr_setstackaddr (pthread_attr_t* attr,void* stackaddr);
int pthread_attr_setstacksize (pthread_attr_t* attr,size_t stacksize);
int pthread_attr_setstack (pthread_attr_t* attr,void* stackaddr, size_t stacksize);
```

以设置好的线程属性结构体为参数创建线程：

```
int pthread_create (pthread_t* restrict thread,
                    const pthread_attr_t* testrict attr,
                    void* (*start_routine) (void*),
                    void* restrict arg);
```

销毁线程属性结构体：

```
int pthread_attr_destroy (pthread_attr_t* attr);
```

2、获取线程属性

获取线程属性结构体：

```
int pthread_getattr_np (pthread_t thread,pthread_attr_t* attr);
```

获取具体线程属性项：

```
int pthread_attr_getdetachstate (pthread_attr_t* attr,int* detachstate);
int pthread_attr_getscope (pthread_attr_t* attr,int* scope);
int pthread_attr_getinheritsched (pthread_attr_t* attr,int* inheritsched);
int pthread_attr_getschedpolicy (pthread_attr_t* attr,int* policy);
int pthread_attr_getschedparam (pthread_attr_t* attr,struct sched_param* param);
int pthread_attr_getguardsize (pthread_attr_t* attr,size_t* guardsize);
int pthread_attr_getstackaddr (pthread_attr_t* attr,void** stackaddr);
int pthread_attr_getstacksize (pthread_attr_t* attr,size_t* stacksize);
int pthread_attr_getstack (pthread_attr_t* attr,void** stackaddr, size_t* stacksize);
以上所有函数成功返回0，失败返回错误码。
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define __USE_GNU
#include <pthread.h>

int printattrs (pthread_attr_t* attr)
{
    printf("------- 线程属性 -------\n");

    int detachstate;
    int error = pthread_attr_getdetachstate (attr, &detachstate);
    if (error)
    {
        fprintf (stderr, "pthread_attr_getdetachstate: %s\n",strerror (error));
        return -1;
    }
    printf("分离状态:   %s\n",
        (detachstate == PTHREAD_CREATE_DETACHED) ? "分离线程" :
        (detachstate == PTHREAD_CREATE_JOINABLE) ? "可汇合线程" :
        "未知");

    int scope;
    if ((error = pthread_attr_getscope (attr, &scope)) != 0)
    {
        fprintf (stderr, "pthread_attr_getscope: %s\n",
            strerror (error));
        return -1;
    }
    printf ("竞争范围:   %s\n",
        (scope == PTHREAD_SCOPE_SYSTEM)  ? "系统级竞争" :
        (scope == PTHREAD_SCOPE_PROCESS) ? "进程级竞争" : "未知");

    int inheritsched;
    if ((error = pthread_attr_getinheritsched (attr,
```

```
        &inheritsched)) != 0)
    {
        fprintf (stderr, "pthread_attr_getinheritsched: %s\n",
            strerror (error));
        return -1;
    }
    printf ("继承特性:   %s\n",
        (inheritsched == PTHREAD_INHERIT_SCHED)  ? "继承调用属性" :
        (inheritsched == PTHREAD_EXPLICIT_SCHED) ? "显式调用属性" :
        "未知");

    int schedpolicy;
    if ((error = pthread_attr_getschedpolicy(attr,&schedpolicy)) != 0)
    {
        fprintf (stderr, "pthread_attr_getschedpolicy: %s\n",strerror (error));
        return -1;
    }
    printf ("调度策略:   %s\n",
        (schedpolicy == SCHED_OTHER) ? "普通" :
        (schedpolicy == SCHED_FIFO)  ? "先进先出" :
        (schedpolicy == SCHED_RR)    ? "轮转" : "未知");

    struct sched_param schedparam;
    if ((error = pthread_attr_getschedparam (attr, &schedparam)) != 0)
    {
        fprintf (stderr, "pthread_attr_getschedparam: %s\n",strerror (error));
        return -1;
    }
    printf ("调度优先级: %d\n", schedparam.sched_priority);

    size_t guardsize;
    if ((error = pthread_attr_getguardsize(attr, &guardsize)) != 0)
    {
        fprintf (stderr, "pthread_attr_getguardsize: %s\n",strerror (error));
        return -1;
    }
    printf ("栈尾警戒区: %u字节\n", guardsize);
    /*
    void* stackaddr;
    if ((error = pthread_attr_getstackaddr (attr, &stackaddr)) != 0)
    {
        fprintf (stderr, "pthread_attr_getstackaddr: %s\n",strerror (error));
        return -1;
    }
    printf ("栈地址:     %p\n", stackaddr);

    size_t stacksize;
    if ((error = pthread_attr_getstacksize (attr, &stacksize)) != 0)
    {
        fprintf (stderr, "pthread_attr_getstacksize: %s\n",strerror (error));
        return -1;
    }
    printf ("栈大小:     %u字节\n", stacksize);
```

```
    */
    void* stackaddr;
    size_t stacksize;
    if ((error = pthread_attr_getstack (attr, &stackaddr,&stacksize)) != 0)
    {
        fprintf (stderr, "pthread_attr_getstack: %s\n",strerror (error));
        return -1;
    }
    printf ("栈地址:     %p\n", stackaddr);
    printf ("栈大小:     %u字节\n", stacksize);

    printf("------------------------\n");

    return 0;
}

void* thread_proc (void* arg)
{
    pthread_attr_t attr;
    int error = pthread_getattr_np (pthread_self (), &attr);
    if (error)
    {
        fprintf (stderr, "pthread_getattr_np: %s\n", strerror (error));
        exit (EXIT_FAILURE);
    }

    if (printattrs (&attr) < 0)
        exit (EXIT_FAILURE);

    exit (EXIT_SUCCESS);

    return NULL;
}

int main (int argc, char* argv[])
{
    int error;
    pthread_attr_t attr, *pattr = NULL;

    if (argc > 1)
    {
        if (strcmp (argv[1], "-s"))
        {
            fprintf (stderr, "用法: %s [-s]\n", argv[0]);
            return -1;
        }

        if ((error = pthread_attr_init (&attr)) != 0)
        {
            fprintf (stderr, "pthread_attr_init: %s\n",strerror (error));
            return -1;
        }

        if ((error = pthread_attr_setdetachstate (&attr,PTHREAD_CREATE_DETACHED)) != 0)
        {
            fprintf (stderr, "pthread_attr_setdetachstate: %s\n",strerror (error));
```

```
            return -1;
        }

        if ((error = pthread_attr_setinheritsched (&attr,PTHREAD_EXPLICIT_SCHED)) != 0)
        {
            fprintf (stderr, "pthread_attr_setinheritsched: %s\n",strerror (error));
            return -1;
        }

        if ((error = pthread_attr_setstacksize (&attr, 4096*10)) != 0)
        {
            fprintf (stderr, "pthread_attr_setstack: %s\n",strerror (error));
            return -1;
        }

        pattr = &attr;
    }

    pthread_t tid;
    if ((error = pthread_create (&tid, pattr, thread_proc,NULL)) != 0)
    {
        fprintf (stderr, "pthread_create: %s\n", strerror (error));
        return -1;
    }

    if (pattr)
    {
        if ((error = pthread_attr_destroy (pattr)) != 0)
        {
            fprintf (stderr, "pthread_attr_destroy: %s\n",strerror (error));
            return -1;
        }
    }

    pause ();
    return 0;
}
```

注意：如果man手册查不到线程的相关函数，安装完整版gnu手册：sudo apt-get install glibc-doc。

二八、线程同步

1、什么是线程同步

同步就是协同步调，按预定的先后次序进行运行。如：你说完，我再说。“同”字从字面上容易理解为一起动作，其实不是，“同”字应是指协同、协助、互相配合。

如进程、线程同步，可理解为进程或线程A和B一块配合，A执行到一定程度时要依靠B的某个结果，于是停下来，示意B运行；B依言执行，再将结果给A，A再继续操作。

在多线程编程里面，一些敏感数据不允许被多个线程同时访问，此时就使用同步访问技术，保证数据在任何时刻，最多有一个线程访问，以保证数据的完整性。

```c
#include <stdio.h>
#include <pthread.h>

int num = 0;
void* run(void* arg)
{
    for(int i=0; i<1000000; i++)
    {
        num++;
    }
}

int main(int argc,const char* argv[])
{
    pthread_t tid1,tid2;
    pthread_create(&tid1,NULL,run,NULL);
    pthread_create(&tid2,NULL,run,NULL);
    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);
    printf("%d\n",num);
}
```

```
理想中的原子++:
-----------------+-----------------+------
       线程1      |       线程2     | 内存
--------+--------+--------+--------+------
 指  令 | 寄存器 | 指  令 | 寄存器 | g_cn
--------+--------+--------+--------+------
 读内存 |    0   |        |        |   0
 算加法 |    1   |        |        |   0
 写内存 |    1   |        |        |   1
        |        | 读内存 |    1   |   1
        |        | 算加法 |    2   |   1
        |        | 写内存 |    2   |   2
--------+--------+--------+--------+------

现实中的非原子++:
-----------------+-----------------+------
       线程1      |       线程2     | 内存
--------+--------+--------+--------+------
 指  令 | 寄存器 | 指  令 | 寄存器 | g_cn
--------+--------+--------+--------+------
 读内存 |    0   |        |        |   0
        |        | 读内存 |    0   |   0
 算加法 |    1   |        |        |   0
        |        | 算加法 |    1   |   0
 写内存 |    1   |        |        |   1
        |        | 写内存 |    1   |   1
--------+--------+--------+--------+------
```

2、互斥锁

```
注意：如果man手册中查不到这系列函数，可以安装以下内容：
    sudo apt-get install glibc-doc
    sudo apt-get install manpages-posix-dev

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
功能：定义并初始化互斥锁

int pthread_mutex_init (pthread_mutex_t* mutex,const pthread_mutexattr_t* mutexattr);
功能：初始化一互斥锁，会被初始化为非锁定状态

int pthread_mutex_lock (pthread_mutex_t* mutex);
功能：加锁，当互斥锁已经是锁定状态时，调用者会阻塞，直到互斥被解开，当前线程才会加锁成功并返回。

int pthread_mutex_unlock (pthread_mutex_t* mutex);
功能：解锁，解锁后等待加锁的线程才能加锁成功。
```

```

int pthread_mutex_destroy (pthread_mutex_t* mutex);
功能：销毁锁

int pthread_mutex_trylock (pthread_mutex_t *__mutex)
功能：加测试锁，如果不加锁刚立即返回

int pthread_mutex_timedlock(pthread_mutex_t *restrict mutex,
                            const struct timespec *restrict abs_timeout);
功能：倒计时加锁，如果超时还不加上则立即返回。
struct timespec{
    time_t tv_sec;        /* Seconds.  */
    long int tv_nsec;     /* Nanoseconds.*/ 1秒= 1000000000 纳秒
};
```

```c
#include <stdio.h>
#include <pthread.h>
/*
执行流程：
    1、互斥锁被初始化为非锁定状态
    2、线程1调用pthread_mutex_lock函数，立即返回，互斥量呈锁定状态；
    3、线程2调用pthread_mutex_lock函数，阻塞等待；
    4、线程1调用pthread_mutex_unlock函数，互斥量呈非锁定状态；
    5、线程2被唤醒，从pthread_mutex_lock函数中返回，互斥量呈锁定状态
*/

pthread_mutex_t mutex;
//pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int num = 0;
void* run(void* arg)
{
    for(int i=0; i<1000000; i++)
    {
        pthread_mutex_lock(&mutex);
        num++;
        pthread_mutex_unlock(&mutex);
    }
}

int main(int argc,const char* argv[])
{
    pthread_mutex_init(&mutex,NULL);
    pthread_t pid1,pid2;
    pthread_create(&pid1,NULL,run,NULL);
    pthread_create(&pid2,NULL,run,NULL);
    pthread_join(pid1,NULL);
    pthread_join(pid2,NULL);
    pthread_mutex_destroy(&mutex);
    printf("%d\n",num);
}
```

3、读写锁

```
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;
功能：定义并初始化读写锁

int pthread_rwlock_init(pthread_rwlock_t *restrict rwlock,
                        const pthread_rwlockattr_t *restrict attr);
功能：初始化读写锁

int pthread_rwlock_rdlock(pthread_rwlock_t *rwlock);
功能：加读锁，如果不能加则阻塞等待

int pthread_rwlock_wrlock(pthread_rwlock_t *rwlock);
功能：加写锁，如果不能加则阻塞等待

int pthread_rwlock_tryrdlock(pthread_rwlock_t *rwlock);
功能：尝试加读锁，如果不能加则立即返回

int pthread_rwlock_trywrlock(pthread_rwlock_t *rwlock);
功能：尝试加写锁，如果不能加则立即返回

int pthread_rwlock_timedrdlock(pthread_rwlock_t *restrict rwlock,
           const struct timespec *restrict abstime);
功能：带倒计时加读锁，超时则立即返回

int pthread_rwlock_timedwrlock(pthread_rwlock_t *restrict rwlock,
           const struct timespec *restrict abstime);
功能：带倒计时加写锁，超时则立即返回

int pthread_rwlock_destroy(pthread_rwlock_t *rwlock);
功能：销毁读写锁

使用读写锁的线程应根据后续的操作进行加锁，如果只对数据进行读取则只加读锁即可，只有对数据进行修改时才应该加写锁，与互斥锁的区别是，它能让只读的线程加上锁，使用原理与文件锁一样。
    线程A      线程B
    读锁       读锁   OK
    读锁       写锁   NO
    写锁       读锁   NO
    写锁       写锁   NO

```

```
#include <pthread.h>

// 创建读写锁
pthread_rwlock_t rwlock;

int num = 0;
void* run(void* arg)
{
    for(int i=0; i<1000000; i++)
    {
        // 加写锁
        pthread_rwlock_wrlock(&rwlock);
        num++;
        // 解锁
        pthread_rwlock_unlock(&rwlock);
```

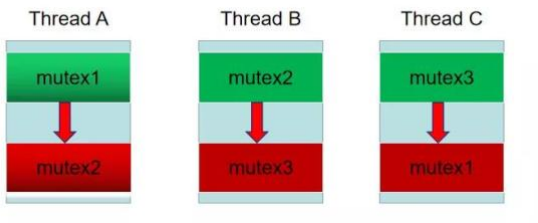
```
    }
}

int main(int argc,const char* argv[])
{
    // 初始化读写锁
    pthread_rwlock_init(&rwlock,NULL);
    pthread_t pid1,pid2;
    pthread_create(&pid1,NULL,run,NULL);
    pthread_create(&pid2,NULL,run,NULL);
    pthread_join(pid1,NULL);
    pthread_join(pid2,NULL);
    printf("%d\n",num);
}

```

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

// 创建读写锁
pthread_rwlock_t rwlock;

void* run1(void* arg)
{
    // 加读锁
    pthread_rwlock_rdlock(&rwlock);
    sleep(1);
    printf("%lu 线程1加读锁成功！\n",pthread_self());
    // 解锁
    pthread_rwlock_unlock(&rwlock);
}
void* run2(void* arg)
{
    // 加读锁
    pthread_rwlock_rdlock(&rwlock);
    sleep(1);
    printf("%lu 线程2加读锁成功！\n",pthread_self());
    // 解锁
    pthread_rwlock_unlock(&rwlock);
}


int main(int argc,const char* argv[])
{
    // 初始化读写锁
    pthread_rwlock_init(&rwlock,NULL);
    pthread_t pid1,pid2;
    pthread_create(&pid1,NULL,run1,NULL);
    pthread_create(&pid2,NULL,run2,NULL);
    pthread_join(pid1,NULL);
    pthread_join(pid2,NULL);
    // 销毁读写锁
    pthread_rwlock_destroy(&rwlock);
}
```

4、死锁问题

1、什么是死锁：

多个线程互相等待对方资源，在得到所需要的资源之前都不会释放自己的资源，然后造成循环等待的现象，称为死锁。



2、死锁产生四大必要条件：

1、资源互斥

2、占有且等待

3、资源不可剥夺

4、环路等待

以上四个条件缺一不可，只要有一个不满足就不能构成死锁。

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

// 创建三个互斥锁并初始化
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex3 = PTHREAD_MUTEX_INITIALIZER;

void* run1(void* arg)
{
    pthread_mutex_lock(&mutex1);
    usleep(100);
    pthread_mutex_lock(&mutex2);
    printf("没有构成死锁！！！\n");
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
}
void* run2(void* arg)
{
    pthread_mutex_lock(&mutex2);
    usleep(100);
    pthread_mutex_lock(&mutex3);
    printf("没有构成死锁！！！\n");
    pthread_mutex_unlock(&mutex3);
    pthread_mutex_unlock(&mutex2);
}
void* run3(void* arg)
{
    pthread_mutex_lock(&mutex3);
    usleep(100);
    pthread_mutex_lock(&mutex1);
```

```
    printf("没有构成死锁！！！\n");
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex3);
}


int main(int argc,const char* argv[])
{
    // 创建三个线程
    pthread_t tid1,tid2,tid3;
    pthread_create(&tid1,NULL,run1,NULL);
    pthread_create(&tid2,NULL,run2,NULL);
    pthread_create(&tid3,NULL,run3,NULL);

    // 主线程等待三个子线程结束
    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);
    pthread_join(tid3,NULL);
    return 0;
}
```

3、如休防止出现死锁

构成死锁的四个条件只有一个不成立，就不会产生死锁了。

1、破坏互斥条件，让资源能够共享使用(准备多份)。

2、破坏占有且等待的条件，一次申请完成它所有需要的资源，资源没有满足前不让它运行，一旦开始运行就一直归它所有， 缺点是系统资源会被浪费。

3、破坏不可剥夺的条件，当已经占有了一些资源，请求新的资源而获取不到，然后就释放已经获取到的资源，缺点是实现起来比较复杂，释放已经获取到的资源可能会造成前一阶段的工作浪费。

4、破坏循环等待的条件，采用顺序分配资源的方法，在系统中为资源进行编号，规定线程必须按照编号递增的顺序获取资源，缺点是资源必须相对稳定，这样就限制了资源的增加和减少。

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

// 创建三个互斥锁并初始化
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex3 = PTHREAD_MUTEX_INITIALIZER;

void* run1(void* arg)
{
    while(1)
    {
        pthread_mutex_lock(&mutex1);
        usleep(100);
        if(0 == pthread_mutex_trylock(&mutex2))
            break;
        pthread_mutex_unlock(&mutex1);

    }
```

```c

    printf("没有构成死锁！！！\n");
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
}
void* run2(void* arg)
{
    while(1)
    {
        pthread_mutex_lock(&mutex2);
        usleep(100);
        if(0 == pthread_mutex_trylock(&mutex3))
            break;
        pthread_mutex_unlock(&mutex2);
    }
    printf("没有构成死锁！！！\n");
    pthread_mutex_unlock(&mutex3);
    pthread_mutex_unlock(&mutex2);
}
void* run3(void* arg)
{
    while(1)
    {
        pthread_mutex_lock(&mutex3);
        usleep(100);
        if(0 == pthread_mutex_trylock(&mutex1))
            break;
        pthread_mutex_unlock(&mutex3);
    }
    printf("没有构成死锁！！！\n");
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex3);
}


int main(int argc,const char* argv[])
{
    // 创建三个线程
    pthread_t tid1,tid2,tid3;
    pthread_create(&tid1,NULL,run1,NULL);
    pthread_create(&tid2,NULL,run2,NULL);
    pthread_create(&tid3,NULL,run3,NULL);

    // 主线程等待三个子线程结束
    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);
    pthread_join(tid3,NULL);
    return 0;
}
```

4、检测死锁的方法

总体思路：观察+分析

方法1：阅读代码，分析各线程的加锁步骤。

方法2：使用strace追踪程序的执行流程。

方法3：查看日志观察程序的业务执行过程。

方法4：使用gdb调试，查看各线程的执行情况。

```
1、把断点打在线程创建完毕后
2、run
3、info threads 查看所有线程
4、thread n 进程指定的线程
5、bt 查看线程堆栈信息
6、配合s/n单步调试
```

5、原子操作

所谓的原子操作就是不可被拆分的操作，对于多线程对全局变量进行操作时，就再也不用再线程锁了，和pthread_mutex_t保护作用是一样的，也是线程安全的，有些编译器在使用时需要加-march=i686编译参数。

```
type __sync_fetch_and_add (type *ptr, type value);  // +
type __sync_fetch_and_sub (type *ptr, type value);  // -
type __sync_fetch_and_and (type *ptr, type value);  // &
type __sync_fetch_and_or (type *ptr, type value);   // |
type __sync_fetch_and_nand (type *ptr, type value); // ~
type __sync_fetch_and_xor (type *ptr, type value);  // ^
功能：以上操作返回的是*ptr的旧值

type __sync_add_and_fetch (type *ptr, type value);  // +
type __sync_sub_and_fetch (type *ptr, type value);  // -
type __sync_and_and_fetch (type *ptr, type value);  // &
type __sync_or_and_fetch (type *ptr, type value);   // |
type __sync_nand_and_fetch (type *ptr, type value); // ~
type __sync_xor_and_fetch (type *ptr, type value);  // ^
功能：以上操作返回的是*ptr与value计算后的值

type __sync_lock_test_and_set (type *ptr, type value);
功能：把value赋值给*ptr,并返回*ptr的旧值

__sync_lock_release(type *ptr);
功能：将*ptr赋值为0
```

```
#include <stdio.h>
#include <pthread.h>

int num = 0;
void* run(void* arg)
{
    for(int i=0; i<100000000; i++)
    {
        __sync_fetch_and_add(&num,1);
    }
}

int main(int argc,const char* argv[])
{
    pthread_t pid1,pid2;
    pthread_create(&pid1,NULL,run,NULL);
    pthread_create(&pid2,NULL,run,NULL);
```

```
    pthread_join(pid1,NULL);
    pthread_join(pid2,NULL);
    printf("%d\n",num);
}
```

1、原子操作的优点

1、速度贼快

2、不会产生死锁

2、原子操作的缺点

1、该功能并不通用，有些编译器不支持。

2、type只能是整数相关的类型，浮点型和自定义类型无法使用。

6、信号量

1、多线程使用的信号量

```
#include <semaphore.h>
sem_t sem;

int sem_init(sem_t *sem, int pshared, unsigned int value);
功能：给信号量设置初始值
pshared：信号量的使用范围
    0 线程间使用
    nonzero 进程之间使用

int sem_wait(sem_t *sem);
功能：信号量减1操作，如果信号量已经等于0，则阻塞

int sem_trywait(sem_t *sem);
功能：尝试对信号量减1操作，能减返回0成功，不能减返回-1失败，不会阻塞

int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
功能：带倒计时的对信号减1操作，能减返回0成功，不能减超时返回-1失败，阻塞abs_timeout一段时间

int sem_post(sem_t *sem);
功能：对信号量执行加1操作

int sem_getvalue(sem_t *sem, int *sval);
功能：获取信号量的值

int sem_destroy(sem_t *sem);
功能：销毁信号量
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

// 仓库最大容量
```

```c
#define STACK_MAX   50

// 仓库
char stack[STACK_MAX];
// 仓库的入口和出口
int top;

// 定义信号量
sem_t data_sem,null_sem;

// 显示仓库内容
void show_stack(const char* opt,char data)
{
    if(50 < top || 0 > top)
    {
        printf("爆仓了 %d\n",top);
        exit(0);
    }
    for(int i=0; i<top; i++)
    {
        printf("%c",stack[i]);
    }
    printf("%s%c\n",opt,data);
}

// 生产者
void* production(void* arg)
{
    for(;;)
    {
        // 空位置的数量减1
        sem_wait(&null_sem);
        // 生产数据
        char data = rand()%26+'A';
        // 显示仓库
        show_stack("<-",data);
        // 数据添加到仓库
        stack[top] = data;
        __sync_fetch_and_add(&top,1);
        usleep(rand()%100*100);
        // 数量的数量加1
        sem_post(&data_sem);
    }
}

// 消费者
void* consumption(void* arg)
{
    for(;;)
    {
        // 数据的数量减1
        sem_wait(&data_sem);
        // 从仓库中消费数据
        char data = stack[top];
        __sync_fetch_and_sub(&top,1);
        // 显示仓库内容
        show_stack("->",data);
        // 模拟消费数据所消耗的时间
```

```
        usleep(rand()%1000*10);
        // 空位置的数量加1
        sem_post(&null_sem);
    }
}

int main(int argc,const char* argv[])
{
    srand(time(NULL));

    // 初始化空位置的数量
    sem_init(&null_sem,0,STACK_MAX);
    // 初始化数据的数量
    sem_init(&data_sem,0,0);

    pthread_t tids[10];
    for(int i=0; i<10; i++)
    {
        if(i<5)
            pthread_create(tids+i,NULL,production,NULL);
        else
            pthread_create(tids+i,NULL,consumption,NULL);
    }

    for(int i=0; i<10; i++)
    {
        pthread_join(tids[i],NULL);
    }

    return 0;
}
```

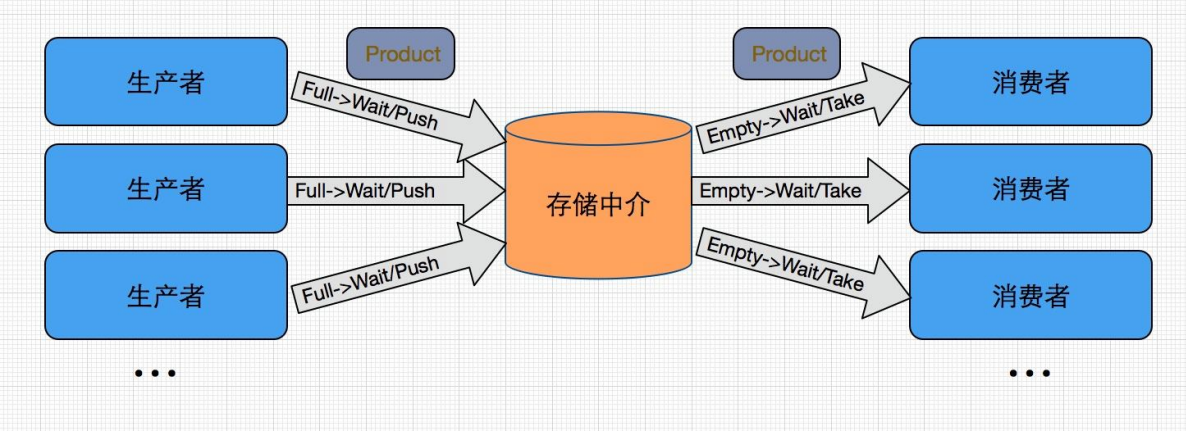
2、多进程使用的信号量：

```
sem_t *sem_open(const char *name, int oflag,mode_t mode, unsigned int value);
功能：在内核创建一个信号量对象
name：信号量的名字
oflag：
    O_CREAT 不存在则创建信号量，存在则获取
    O_EXCL  如果信号量已经存在，返回失败
mode：信号量的权限
value：信号量的初始值

sem_t *sem_open(const char *name, int oflag);
功能：获取信号，或相关属性

int sem_unlink(const char *name);
功能：删除信号量
```

```
// 进程A
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

```

```c
int main(int argc,const char* argv[])
{
    sem_t* sem =  sem_open("sem_name",O_CREAT,0644,0);
    for(;;)
    {
        sem_post(sem);
        sleep(1);
        int value = 0;
        sem_getvalue(sem,&value);
        printf("%d\n",value);
    }
    return 0;
}

// 进程B
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

int main(int argc,const char* argv[])
{
    sem_t* sem =  sem_open("sem_name",O_CREAT);
    for(;;)
    {
        sem_wait(sem);
        sleep(2);
        int value = 0;
        sem_getvalue(sem,&value);
        printf("%d\n",value);
    }

    return 0;
}
```

7、生产者与消费者模型



生产者：

生产数据的线程，这类的线程负责从用户端、客户端接收数据，然后把数据Push到存储中介。

消费者：

负责消耗数据的线程，对生产者线程生产的数据进行（判断、筛选、使用、响应、存储）处理。

存储中介：

也叫数据仓库，是生产者线程与消费者线程之间的数据缓冲区，用于平衡二者之间的生产速度与消耗速度不均衡的问题，通过缓冲区隔离生产者和消费者，与二者直连相比，避免相互等待，提高运行效率。

当生产快于消费：

缓冲区很快会被填满，消费者会被撑死。

解决方法：负责生产的线程通知负责消费的线程全速消费，然后进入休眠。

当消费快于生产：

缓冲区长期处理空的状态，消费者会被饿死。

解决方法：负责消费的线程通知负责生产的线程全速生产，然后进入休眠。

8、条件变量

条件变量是利用线程间共享的"全局变量"进行同步的一种机制，主要包括两个动作：

1、线程等待"条件变量的条件成立"而休眠；

2、等"条件成立"叫醒休眠的线程。

为了防止竞争，条件变量的使用总是和一个互斥锁结合在一起。

```
int pthread_cond_init (pthread_cond_t* cond,const pthread_condattr_t* attr);
//亦可pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

// 使调用线程睡入条件变量cond，同时释放互斥锁mutex
int pthread_cond_wait (pthread_cond_t* cond,pthread_mutex_t* mutex);

int pthread_cond_timedwait (pthread_cond_t* cond,
    pthread_mutex_t* mutex,
    const struct timespec* abstime);

struct timespec {
    time_t tv_sec;  // Seconds
    long   tv_nsec; // Nanoseconds [0 - 999999999]
};

// 从条件变量cond中唤出一个线程，
// 令其重新获得原先的互斥锁
int pthread_cond_signal (pthread_cond_t* cond);

注意：被唤出的线程此刻将从pthread_cond_wait函数中返回，
但如果该线程无法获得原先的锁，则会继续阻塞在加锁上。

// 从条件变量cond中唤醒所有线程
int pthread_cond_broadcast (pthread_cond_t* cond);

```

```c
// 销毁条件变量
int pthread_cond_destroy (pthread_cond_t* cond);
```

生产者与消费者模型的实现：

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
// 仓库最大容量
#define STACK_MAX   50

// 仓库
char stack[STACK_MAX];
// 仓库的入口和出口
int top;

// 保护仓库入口的互斥锁
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// 当仓库满时线程睡入的条件变量
pthread_cond_t full = PTHREAD_COND_INITIALIZER;
// 当仓库空时线程睡入的条件变量
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;

// 显示仓库内容
void show_stack(const char* opt,char data)
{
    if(50 < top || 0 > top)
    {
        printf("爆仓了 %d\n",top);
        exit(0);
    }
    for(int i=0; i<top; i++)
    {
        printf("%c",stack[i]);
    }
    printf("%s%c\n",opt,data);
}

// 生产者
void* production(void* arg)
{
    for(;;)
    {
        // 往仓库中push数据，加锁保护入口
        pthread_mutex_lock(&mutex);
        // 发现仓库满了
        while(top >= STACK_MAX)
        {
            // 叫睡所有消费线程全速消费，虽然暂时只能醒来一个消费者线程，但所有消费线程已经加入到争夺互斥锁的过程
            pthread_cond_broadcast(&empty);
            // 生产者线程睡入满仓条件变量
            pthread_cond_wait(&full,&mutex);
        }

        // 生产数据
```

```
        char data = rand()%26+'A';
        // 显示仓库
        show_stack("<-",data);
        // 数据添加到仓库
        stack[top++] = data;
        // 模拟生产数据消耗的时间
        usleep(rand()%5*10000);

        // 此时仓库已经不空了，叫醒一个空仓条件休眠的消费者线程
        pthread_cond_signal(&empty);

        // 生产完毕解锁入口
        pthread_mutex_unlock(&mutex);
    }
}

// 消费者
void* consumption(void* arg)
{
    for(;;)
    {
        // 从仓库中消费数据，加锁保护出口
        pthread_mutex_lock(&mutex);
        // 发现空仓
        while(0 == top)
        {

            // 叫醒所有生产线程全速生产，虽然暂时只能醒来一个生产者线程，但所有生产者线程都加入到争夺互斥锁的过程
            pthread_cond_broadcast(&full);
            // 消费者线程睡入空仓条件变量
            pthread_cond_wait(&empty,&mutex);
        }

        // 从仓库中消费数据
        char data = stack[top--];
        // 显示仓库内容
        show_stack("->",data);
        // 模拟消费数据所消耗的时间
        usleep(rand()%5*10000);

        // 此时已经不满，叫醒一个因为满仓而休眠的线程
        pthread_cond_signal(&full);

        // 消费完毕解锁出口
        pthread_mutex_unlock(&mutex);
    }
}

int main(int argc,const char* argv[])
{
    srand(time(NULL));

    pthread_t tids[10];
    for(int i=0; i<10; i++)
    {
        if(i%2)
            pthread_create(tids+i,NULL,production,NULL);
```

```
        else
            pthread_create(tids+i,NULL,consumption,NULL);
    }

    for(int i=0; i<10; i++)
    {
        pthread_join(tids[i],NULL);
    }
    return 0;
}
```

注意：使用互斥锁配合条件变量实现的生产者与消费者模型，能够平衡生产与消费的时间不协调，并且可以最大限度的节约运行资源。